

Hiperpersonalización aplicada al desarrollo de asistentes virtuales

Alexander Ditzend

Ingeniería Industrial

Director:

Dr. Xavier I. González, Ing. Marcelo E. Chidichimo

Jurados:

Por definir

Índice general

Resumen	3
Síntesis Ejecutiva	4
Introducción / Planteo del Problema	6
Motivaciones	6
Antecedentes	7
Planteo del Problema	9
Objetivos e Hipótesis	12
Hipótesis	12
Objetivos	12
Metodología	13
Etapas de trabajo	13
Desarrollo	14
Experimentos de <i>Prompting</i> sobre problemas de alcance muy focalizado	14
Primera Iteración	16
Creación de una Base de Mensajes del Usuario y Activación de Filtros	23
Similitud entre consultas y extractos de conversaciones	26
Implementación de Búsquedas Semánticas con Redis y FAISS para la Gestión de Sesiones de Usuario	31
Análisis del Proceso de Vectorización de Conversaciones y Búsqueda Semántica	37
Análisis del enfoque en la recuperación de información sobre un pariente del usuario	42
Análisis de la Mejora en la Estructura del Contexto para la Recuperación de Información	48
Análisis de la Interpretación de Fechas y Razonamiento Cronológico	57
Influencia de la estructura del contexto en el razonamiento cronológico	63

Agregado de un Paso de Extracción y Limpieza a un Conjunto de Datos con 4	
Miembros de un Grupo Familiar	69
Razonamiento Cronológico y Extracción de Datos en Conversaciones Familiares	76
Mejora del Razonamiento Cronológico en Conversaciones Familiares Utilizan-	
do GPT-3.5 y GPT-4	82
Optimización del Razonamiento Cronológico y Manejo de Contexto en Bús-	
quedas Semánticas con GPT-4	88
Experimentos Combinados sobre Fechas y Manejo de Cantidades	95
Manejo de Datos Personales y Resolución de Citas Médicas con GPT-3.5: Con-	
junto de Datos Referente a Turnos de un Tercero	101
Manejo de Datos Personales y Errores de Contexto en la Gestión de Citas Mé-	
dicas con GPT-3.5	108
Ampliación de Conjuntos de Datos de Contexto por Ingestión de Documentos	115
Creación de conjuntos de datos sintéticos	116
Creación de datos familiares	117
Creación de conjunto de datos médicos de alcance intermedio (1 año)	119
Creación de conjunto de datos de consumo de mediano plazo (1 año)	126
Ingestión a base de datos de los datos sintéticos	133
Ingestión a base de grafos	133
Inserción en base de documentos con soporte vectorial y consultas	144
Conclusiones	152
Viabilidad	152
Recursos Necesarios para la Implementación	152
Limitaciones de la solución actual	152
Funcionalidades a desarrollar en fases sucesivas	153
Anexo - Plan de Negocio	155
Introducción	155
Considerandos	155

Actores del mercado	155
Definición del producto	156
Análisis estratégico	157
Análisis FODA	157
Propuesta de valor	158
Ejemplos de impacto por industria	159
Plan comercial con estimación de ventas	164
Definición del producto	164
Segmentación del mercado objetivo	164
Estrategia de penetración en el mercado	164
Modelo de ingresos	164
Tamaño y crecimiento del mercado de asistentes virtuales	165
Mercado de hiperpersonalización	165
Adopción de asistentes virtuales en américa latina	165
Proyección de Ventas	166
Simulación con el Método Delphi Simulado	168
Proyección Monetaria por Escenarios	173
Proyección de costos	175
Desarrollo del sistema	175
Implementación	178
Soporte y monitoreo	179
Costos fijos	180
Infraestructura y operación	182
Resumen	182
Otros costos variables	182
Evaluación Financiera	183
Márgenes de contribución	183
Tasas para la valuación del proyecto	183
Evaluación monetaria del proyecto	185

Resumen

Este trabajo analiza la viabilidad de dotar a los asistentes virtuales de un nivel de hiperpersonalización que combine datos semánticos y estructurados del usuario. Plantea la hipótesis de que, a partir del análisis de interacciones previas y la ingestión de documentos relevantes, es posible construir una base de datos híbrida capaz de aportar contexto en tiempo real a la generación de respuestas.

Para contrastar esta premisa se diseñó una metodología experimental en cuatro etapas:

1. Experimentos de *prompting* acotados
2. Ingestión de documentos para ampliar el corpus contextual
3. Generación de conjuntos de datos sintéticos
4. Almacenamiento de dichos conjuntos en bases vectoriales y de grafos, con filtros específicos por usuario

El prototipo, que integra Redis para sesiones y FAISS para búsquedas por similitud, demostró que la distancia coseno, aplicada tras un filtrado por identificador, recupera información pertinente aun en corpora crecientes; el sistema resolvió consultas cronológicas y semánticas combinadas con alta precisión.

Se concluye que la estrategia es técnicamente factible y escalable, siempre que el motor vectorial gestione eficientemente el volumen de datos; no obstante, persisten retos ligados a la memoria conversacional continua y a la adaptación de modelos menos potentes.

Síntesis Ejecutiva

Se propone demostrar que es posible dotar a los asistentes virtuales de un nivel de hiperpersonalización que trascienda la simple recuperación de datos históricos y habilite interacciones contextualizadas, fluidas y centradas en la persona. El problema se delimita en la imposibilidad de los sistemas actuales de recordar la información pertinente de cada usuario de forma estructurada y emplearla en tiempo real para enriquecer la conversación. Sobre esta base, se plantea la hipótesis de que, a partir del análisis de interacciones previas y la ingestión de documentos relevantes, se puede construir una base de datos híbrida —con componentes semánticos y estructurados— capaz de suministrar al asistente el contexto necesario para responder con precisión y pertinencia.

La investigación adopta una metodología experimental dividida en cuatro etapas sucesivas:

1. Experimentos de *prompting* focalizados
2. Ingestión de documentos para ampliar el contexto
3. Generación de conjuntos de datos sintéticos
4. Alimentación de dichos conjuntos en bases vectoriales y de grafos

Cada fase refina la arquitectura de recuperación y generación de respuestas, combinando recuperación de ambos tipos de bases.

En el desarrollo se implementa un prototipo que integra Redis para el almacenamiento transitorio de sesiones y FAISS para la búsqueda de similitud en espacios vectoriales. A través de sucesivos experimentos —incluidos la desagregación de problemas macro en casos de uso unitarios, la vectorización de mensajes de usuario y la incorporación de marcas temporales— se evidencia que la distancia coseno aplicada tras un filtrado por identificador logra recuperar de forma fiable la información pertinente, aun cuando el corpus crece en complejidad. Esta capacidad es validada, por ejemplo, al responder correctamente preguntas sobre la edad de un familiar o las preferencias de viaje de las hijas del usuario, demostrando que el sistema puede razonar sobre datos cronológicos y semánticos combinados.

Los resultados confirman la viabilidad del enfoque: la combinación de modelos de lenguaje preentrenados y *embeddings* resulta suficiente para procesar y recuperar la información

necesaria, siempre que el motor de búsqueda vectorial pueda manejar eficientemente el volumen de datos almacenado. Para su implementación se requiere un servicio de extracción y estructuración de información, una base con capacidad de similitud vectorial y búsqueda híbrida, un servicio de ingestión de datos y otro de generación de respuestas basadas en contexto histórico. Estos componentes pueden desplegarse tanto en infraestructuras locales como en servicios en la nube.

Entre las limitaciones detectadas se señala la necesidad de reconfigurar las fuentes de información cuando se emplean modelos menos potentes o versiones de modelos ya deprecadas, así como la ausencia de un manejo de memoria conversacional que conecte los mensajes de forma continua. Además, el sistema actual no resuelve preguntas sobre múltiples temáticas en una única iteración, lo que impone desafíos de escalabilidad funcional. El trabajo advierte que estos obstáculos deberán afrontarse mediante la integración de servicios de observación paralelos que extraigan y estructuren la información mientras la conversación ocurre.

Como proyección, la tesis plantea que el próximo paso para la adopción masiva de asistentes hiperpersonalizados reside en garantizar la privacidad y el control de los datos mediante tecnologías distribuidas como blockchain, capaces de ofrecer inmutabilidad y gobernanza descentralizada de la información del usuario. Con ello, se aspira a habilitar un ecosistema donde cada individuo comparta —de forma segura y auditable— los fragmentos de su vida digital que considere convenientes, permitiendo a los asistentes virtuales convertirse en verdaderos acompañantes inteligentes, capaces de anticipar necesidades y ofrecer soluciones a medida.

En suma, el trabajo demuestra experimentalmente que la hiperpersonalización en asistentes virtuales no solo es técnicamente factible sino también operativamente viable con recursos accesibles. Sus aportes metodológicos sientan las bases para futuras implementaciones industriales, encaminadas a transformar la experiencia de usuario y a potenciar la eficiencia de los sistemas conversacionales en múltiples dominios.

Introducción / Planteo del Problema

Motivaciones

El advenimiento de los grandes modelos de lenguaje, o *Large Language Models* (LLMs, por sus siglas en inglés), ha provocado una revolución en la manera en que los seres humanos interactúan con los sistemas digitales. Un ejemplo paradigmático de este cambio es la transición del uso tradicional del explorador web y sus motores de búsqueda al empleo de asistentes conversacionales potenciados por LLMs. Otro caso relevante es el de los entornos de desarrollo utilizados por programadores, donde los LLMs se integran para asistir en la generación de código.

Pocos dudan que, en un futuro cercano, la mayoría del código informático será producido por LLMs, bajo la guía, supervisión y acompañamiento de seres humanos. Esta transformación del rol del programador puede entenderse como una antesala de lo que ocurrirá en otras profesiones y, en general, en la interfaz que conecta a los seres humanos con sus fuentes de información. En este nuevo paradigma, los asistentes virtuales tenderán a ocupar una posición dominante, al ofrecer una interfaz basada en la conversación natural—ya sea hablada o escrita—para interactuar con datos y servicios.

Sin embargo, para que los asistentes virtuales evolucionen más allá de simples herramientas de consulta o gestión de citas, es fundamental que puedan recordar información pasada de manera estructurada. De lo contrario, su funcionalidad quedará limitada a interacciones impersonales y de corto alcance temporal, lo que afectaría negativamente su adopción y utilidad.

En consecuencia, se vuelve necesario desarrollar técnicas que permitan clasificar, ingerir y recuperar información sobre una persona, de modo que los asistentes virtuales puedan operar con un contexto dinámico y relevante al momento de interactuar con distintas fuentes de datos.

La implementación de estrategias de hiperpersonalización genera, en promedio, un incremento del 15% anual en ventas, con picos del 25% dependiendo de la industria (**McKinsey2020**).

Paralelamente se estima un crecimiento del 37.3% en el mercado de inteligencia artificial, lo que sugiere una aceleración en la demanda de asistentes virtuales, incluso sin personalización avanzada (**GrandViewResearch2023**).

Las interrelaciones entre estas dos tendencias son complejas e imposibles de predecir con la

precisión necesaria para establecer un nivel óptimo de inversión. No obstante, los niveles de crecimiento observados evidencian una demanda creciente tanto por soluciones de inteligencia artificial como por experiencias personalizadas.

Esta tesis propone una metodología que busca unir ambas dimensiones—la inteligencia artificial y la hiperpersonalización—de manera segura, performante y económica, mediante un enfoque abierto y accesible para cualquier persona u organización interesada en implementarlo.

Antecedentes

La industria de los asistentes virtuales tiene una dependencia directa con la de los bots de automatización de conversaciones que comenzaron a surgir en la década del 2000. Los mensajes generados por estos sistemas seguían un patrón basado en plantillas completadas con información extraída de bases de datos. Un ejemplo habitual es el mensaje emitido por una entidad bancaria que incluye el nombre completo del receptor y el monto de una deuda pendiente. Esta forma de personalizar mensajes continúa siendo ampliamente utilizada en la actualidad, debido a su simplicidad y bajo costo de implementación.

En el campo de los asistentes virtuales, sin embargo, no se registran desarrollos significativos orientados a organizar y procesar toda la información que un asistente pueda acumular sobre su contraparte humana. La comunidad científica vinculada al procesamiento del lenguaje natural ha centrado su interés en el desarrollo de modelos generalistas, capaces de razonar y resolver tareas cada vez más complejas con una cantidad decreciente de datos y recursos.

Paralelamente, el ámbito del marketing y la publicidad ha experimentado avances importantes tanto en la comprensión del comportamiento humano como en la tecnología necesaria para entregar un anuncio a la persona con mayor probabilidad de conversión. Las limitaciones en la disponibilidad de datos individualizados llevaron a que, en la mayoría de los casos, se opere con agregaciones a nivel de grupo. En este contexto, el término *personalización* refiere al uso de datos sobre una persona para generar ofertas relevantes. La *hiperpersonalización*, en cambio, se distingue por la extensión, profundidad y cantidad de datos involucrados, así como por la capacidad de reacción inmediata ante la obtención de nueva información. Mientras que una solución personalizada se basa en datos históricos, la hiperpersonalización incorpora la

actividad presente del usuario para generar respuestas ajustadas al momento.

Como se analiza en la introducción, el estudio de múltiples puntos de datos sobre una misma persona para predecir su comportamiento futuro es ampliamente aplicado en el ámbito del marketing, especialmente desde la irrupción del comercio digital. El uso de esta información para maximizar ventas constituye el enfoque más lucrativo de la técnica, lo que explica su temprana adopción en ese sector.

En este contexto, se observa la construcción de modelos orientados a recomendar compras futuras o productos complementarios. También se utilizan para la colocación personalizada de anuncios y la predicción de acciones posteriores, entre otras aplicaciones relacionadas con el ciclo de compra del consumidor.

Las empresas que lideran en personalización generan, en promedio, un 40% más de ingresos que aquellas que no la implementan. Se estima que, en las industrias de Estados Unidos, mejorar el desempeño hasta ubicarse en el cuartil superior en términos de personalización podría generar más de un billón de dólares en valor agregado (**McKinsey2021**).

El 62% de los líderes empresariales citan la mejora en la retención de clientes como uno de los beneficios clave de la personalización (**TwilioSegment2023**).

Asimismo, el 72% de los consumidores esperan que las empresas con las que interactúan los reconozcan como individuos y conozcan sus intereses (**McKinsey2021**).

El 90% de los consumidores declara preferir, recomendar o pagar más por marcas que ofrecen una experiencia o servicio personalizado (**Outgrow2023**).

Según Esat Artug, el 91% de los consumidores tiene más probabilidades de comprar en marcas que reconocen sus preferencias y proporcionan ofertas relevantes. A su vez, el 80% estaría dispuesto a gastar más en empresas que brindan servicios personalizados (**Ninetailed2024**).

Estas cifras muestran que la hiperpersonalización no solo mejora la satisfacción y fidelización del cliente, sino que también tiene un impacto directo en los ingresos y en la eficiencia de las estrategias de marketing.

La evolución de la inteligencia artificial conversacional es significativa, avanzando desde simples *chatbots* hacia sistemas altamente personalizados. Gracias al procesamiento del lenguaje natural (NLP), los asistentes digitales adquieren la capacidad de comprender las

intenciones de los usuarios y adaptar sus respuestas, haciendo las conversaciones más relevantes, fluidas y atractivas.

Las expectativas de los consumidores también evolucionan: el 70% anticipa que las empresas utilizan inteligencia artificial para generar interacciones y ofertas personalizadas. En ese marco, alrededor del 61% de los consumidores prefiere tratar con marcas que ofrecen recorridos de usuario rápidos y ajustados a sus preferencias. Además, el 66% espera que las empresas reconozcan sus necesidades particulares.

La hiperpersonalización en IA conversacional responde de forma eficaz a estas expectativas, enriqueciendo la relación con la marca, fomentando la lealtad del cliente y mejorando los resultados comerciales. Los consumidores están incluso más dispuestos a compartir información personal con estos sistemas: un informe de Zendesk revela que dos tercios de los compradores aceptan intercambiar más datos a cambio de una experiencia más individualizada si esta está impulsada por inteligencia artificial (**Zendesk2024**).

No obstante, a pesar del potencial demostrado, las herramientas tecnológicas actuales para construir asistentes virtuales aún no logran ofrecer soluciones efectivas para implementar sistemas de hiperpersonalización. El principal obstáculo radica en la integración de fuentes de datos: en la mayoría de los casos, cada fuente está administrada por distintos proveedores, con formatos y repositorios dispares. Esta integración suele quedar fuera del alcance operativo de muchas organizaciones, al no ser parte de su núcleo de actividades.

Superado ese primer desafío, el siguiente consiste en organizar dichos datos de modo que un asistente virtual pueda interpretarlos y emplearlos efectivamente en beneficio del usuario. En esta área, no se encuentran desarrollos ni investigaciones específicas, lo que motiva la realización del presente trabajo.

Planteo del Problema

En el contexto de un asistente virtual, la distinción entre personalización e hiperpersonalización se manifiesta no solo en recomendaciones u ofertas, sino también en la calidad de la asistencia o soporte brindado. A continuación, se ejemplifica esta diferencia mediante una simulación de conversación entre un asistente virtual (A) y una usuaria o usuario (U) en un sitio de reservas de viajes.

Sin personalización.

A: Hola, ¿cómo puedo ayudarle hoy?

U: Quiero sacar un pasaje a Río de Janeiro con la familia.

A: Entiendo, ¿puede indicarme desde dónde partirá?

U: Buenos Aires.

A: ¿Cuántas personas?

U: Cinco.

A: ¿Cuántos menores?

U: Dos.

El sistema no se apoya en datos contextuales de la persona. En cada sesión deben ingresarse nuevamente todos los datos. Este es el punto clave que genera fricción en el uso cotidiano e impacta en las ventas y la satisfacción del cliente.

Personalización.

A: Hola, ¿cómo puedo ayudarle hoy?

U: Quiero sacar un pasaje a Río de Janeiro con la familia.

A: Entiendo, su último viaje fue desde Buenos Aires, ¿partirá desde el mismo lugar?

U: Sí.

A: ¿Cuántas personas?

U: Cinco.

A: ¿Cuántos menores?

U: Dos.

El sistema accede al historial de compras del cliente mediante su número de identificación, pero sólo recupera parcialmente la información relevante, como el origen del vuelo, sin contemplar la composición del grupo familiar.

Hiperpersonalización.

A: Hola, ¿cómo puedo ayudarle hoy? ¿Desea ver opciones para paquetes a Río de Janeiro?

U: Sí, sí, iría con la familia. Mi madre no se suma esta vez, lo único.

A: Entiendo. Veo que se encuentra en Buenos Aires, ¿va a partir desde esa ciudad?

U: Sí.

A: ¿Viajará entonces con sus dos hijas menores y su esposa?

U: Sí.

El sistema combina datos de comportamiento en tiempo real (como navegación) con información de localización y acceso a una base estructurada de datos personales que permite comprender la composición familiar y formular respuestas adecuadas al contexto actual.

Objetivos e Hipótesis

Hipótesis

A partir del análisis de interacciones previas entre un usuario y un sistema informático, así como de las conversaciones sostenidas entre el usuario y un asistente virtual, es posible construir una base de datos híbrida — que combine componentes semánticos y estructurados — de la información personal del usuario. Dicha base de datos podría ser consultada posteriormente para aportar contexto relevante en la generación de respuestas por parte del asistente virtual.

El tema fundamental de la privacidad de los datos personales no es objeto de este trabajo ya que el foco está puesto en la funcionalidad y la precisión de la hiperpersonalización.

Objetivos

Se plantea como objetivo principal encontrar una solución tan simple, económica y eficiente como sea posible para la construcción de sistemas conversacionales que permitan la hiperpersonalización de las interacciones con los usuarios. Desde la perspectiva de las tecnologías aplicadas, es preferible hallar una solución subóptima con las herramientas que se encuentran en uso actualmente que una solución óptima con tecnologías difíciles de integrar, agregando complejidad al mantenimiento de todo el sistema.

Metodología

Se propone una metodología de descomposición del problema en situaciones atómicas, que se resuelven con técnicas de *prompting* sobre modelos de lenguaje. Con los hallazgos que se obtienen en cada una de las situaciones se avanza hacia una solución general.

Etapas de trabajo

El trabajo de investigación se divide en las siguientes etapas:

- Experimentos de *prompting* sobre problemas de alcance muy focalizado (menos de 10 ejemplos), utilizando datos generados ex profeso.
- Creación de conjuntos de datos a partir de documentación real.
- Generación de conjuntos de datos sintéticos.
- Generalización hacia conjuntos de datos de alcance intermedio (aproximadamente un año).

Desarrollo

Experimentos de *Prompting* sobre problemas de alcance muy focalizado

Un asistente virtual con capacidades de hiperpersonalizar es un sistema de mensajes por turnos en el que existe un vasto repositorio de información sobre el usuario en curso y esa información es aprovechada por el asistente para resolver el problema del usuario en la menor cantidad de interacciones.

Mediante diferentes experimentos se intenta encontrar una arquitectura que identifique información sobre un usuario, la guarde ordenadamente y la pueda usar como contexto en cualquier conversación.

De ahora en adelante, los términos *usuario* y *persona* se usan de manera indistinta para referirse a un ser humano que está interactuando con un asistente virtual para resolver un problema.

Durante los experimentos se hace referencia a bases vectoriales que contienen motores de búsqueda especiales para encontrar eficientemente los k vectores más cercanos a un vector de consulta, siendo k el parámetro de búsqueda que controla cuántos resultados se desea recibir en la respuesta. Estos espacios vectoriales tienen entre 300 y 4000 dimensiones dependiendo del modelo de *embeddings* que se utilice para hacer la codificación del texto.

Los *embeddings* son representaciones matemáticas que permiten transformar datos, como palabras o elementos de un conjunto, en vectores de números de una dimensión fija, de forma tal que las relaciones semánticas o estructurales entre los elementos se preserven en el espacio vectorial resultante. En otras palabras, los *embeddings* convierten datos complejos, como texto, en una forma numérica que los algoritmos de machine learning pueden procesar eficientemente.

En el contexto de procesamiento de lenguaje natural (NLP), los word embeddings son una de las aplicaciones más comunes, donde cada palabra de un vocabulario es mapeada a un vector numérico. Estos vectores están contruidos de manera tal que palabras con significados o contextos similares estarán más cercanas en este espacio vectorial. Esto permite a los modelos de machine learning capturar y utilizar relaciones semánticas y sintácticas entre palabras.

Técnicas populares para generar embeddings incluyen Word2Vec, GloVe y BERT, cada una

con diferentes enfoques para capturar estas relaciones. Estos vectores no solo simplifican la representación de datos complejos, sino que también mejoran el rendimiento de los modelos de aprendizaje automático, permitiendo que estos procesen grandes cantidades de información de manera más eficiente y precisa.

Los *embeddings* son una herramienta clave para representar datos de alta dimensión en un espacio de menor dimensión, preservando las características más importantes de esos datos para su posterior análisis y procesamiento.

Las bases de datos que alojan vectores de *embeddings* también son llamadas semánticas porque los vectores representan generalmente la conversión que hace un modelo de *embeddings* de una pieza de texto, que puede ser visto como su contenido semántico.

En el contexto del presente trabajo se utiliza el modelo text-embedding-3-small que utiliza 1536 dimensiones. Por lo tanto, en la base se guardarán vectores de 1536 de 16 dígitos de precisión entre -1 y 1. Cada número representa la coordenada de un vector en la dimensión de la posición que ocupa.

Esta etapa representa el núcleo del presente trabajo.

El primer paso fue desestructurar el problema macro en problemáticas unitarias y crear un repositorio ordenado de las mismas.

Se siguió el siguiente patrón:

1. Se seleccionó una problemática enunciada como un caso de uso que hoy es imposible de resolver sin hiperpersonalización.
2. Se describió una estrategia de ataque a esa problemática y se desarrolló un prototipo.
3. Se registraron los resultados para alimentar un repositorio de prompts exitosos.
4. Se repitió el proceso hasta encontrar una solución satisfactoria.

Primera Iteración

Herramientas a utilizar.

Redis. Redis es una base de datos NoSQL de código abierto, en memoria, que se utiliza principalmente como un almacén clave-valor de alta velocidad. A diferencia de las bases de datos tradicionales que almacenan datos en disco, Redis guarda todos sus datos en la memoria RAM, lo que permite tiempos de lectura y escritura extremadamente rápidos. Esto la convierte en una opción ideal para aplicaciones donde la velocidad es crítica, como el almacenamiento de sesiones, caché, colas de mensajes y procesamiento en tiempo real.

Redis es más que un simple almacén clave-valor. Soporta varios tipos de datos estructurados, como cadenas, listas, conjuntos, mapas (hashes), conjuntos ordenados, entre otros. Además, ofrece operaciones avanzadas sobre estos tipos de datos, lo que lo hace más flexible y potente que otros sistemas de caché como Memcached.

Redis presenta varias características clave que lo distinguen como una solución eficiente para el almacenamiento de datos en aplicaciones de alta demanda. En primer lugar, todos los datos se mantienen en memoria RAM, lo que garantiza una velocidad de acceso extremadamente rápida. No obstante, Redis también permite la persistencia opcional en disco, de modo que los datos pueden recuperarse después de un reinicio del sistema.

Otra característica importante es el soporte para múltiples tipos de datos. Redis permite trabajar con diferentes estructuras como cadenas, listas, conjuntos, mapas y otros, lo que proporciona una flexibilidad significativa y facilita su uso en diversas aplicaciones. Aunque su función principal es el almacenamiento en memoria, Redis ofrece opciones de persistencia mediante técnicas como snapshots o el registro de operaciones en un archivo de log denominado AOF (Append-Only File), lo que posibilita la recuperación de datos tras un fallo o reinicio.

En cuanto a la distribución y escalabilidad, Redis permite la distribución de datos entre varios nodos a través de Redis Cluster. Esto facilita la escalabilidad horizontal en sistemas distribuidos y mejora la disponibilidad del servicio. Además, ofrece operaciones atómicas para asegurar la consistencia de los datos, incluso en entornos concurrentes, y permite la ejecución de transacciones.

Por último, Redis soporta la replicación maestro-esclavo, lo que posibilita la creación de réplicas de una instancia para mejorar la disponibilidad y la tolerancia a fallos. Estas características en conjunto hacen de Redis una herramienta robusta y versátil para el desarrollo de sistemas que requieren alta velocidad y confiabilidad en el manejo de datos.

Gracias a estas características, Redis se ha convertido en una solución popular en aplicaciones que requieren respuestas rápidas, como sistemas de recomendación, almacenamiento temporal de datos, colas de trabajo, sistemas de monitoreo y análisis en tiempo real.

FAISS. FAISS (Facebook AI Similarity Search) es una biblioteca de código abierto desarrollada por Facebook que permite realizar búsquedas eficientes de similitud y clustering en grandes conjuntos de datos de vectores. Está optimizada para manejar búsquedas de alta dimensión, lo que la hace ideal para trabajar con embeddings en aplicaciones como el procesamiento de lenguaje natural, la búsqueda de imágenes, la recomendación de contenido y más.

FAISS está diseñada para buscar vecinos más cercanos (nearest neighbors) en grandes bases de datos, y puede realizar tanto búsquedas exactas como aproximadas. Esto es especialmente útil cuando se trabaja con datos de alta dimensionalidad, como vectores que representan palabras, imágenes o cualquier tipo de datos complejos, donde la búsqueda exacta puede ser computacionalmente costosa.

FAISS presenta varias características clave que lo convierten en una herramienta eficiente y versátil para la búsqueda de similitud en grandes volúmenes de datos. En primer lugar, está optimizado para manejar conjuntos de datos que pueden alcanzar millones o incluso miles de millones de vectores, utilizando tanto la CPU como la GPU para acelerar los tiempos de búsqueda. Esta optimización permite que FAISS sea altamente eficiente en contextos donde la escala y la velocidad son fundamentales.

Otra característica importante es su capacidad para realizar búsquedas de vecinos más cercanos, lo cual resulta esencial en tareas de recuperación de información, recomendación de productos o detección de elementos similares. Además, FAISS soporta tanto búsquedas exactas, que garantizan encontrar los vecinos más cercanos, como búsquedas aproximadas, que sacrifican algo de precisión a cambio de una mayor velocidad, adaptándose así a

diferentes necesidades de rendimiento.

El soporte para GPUs es una de las principales ventajas de FAISS, ya que permite mejorar considerablemente el rendimiento en búsquedas de alta dimensionalidad. Esto resulta especialmente útil cuando se requiere procesar grandes volúmenes de datos de manera rápida. Asimismo, FAISS implementa técnicas avanzadas de indexación, como el Product Quantization (PQ) y el Inverted File Index (IVF), que facilitan la organización eficiente de los datos y reducen la complejidad de las búsquedas.

Por último, FAISS es una solución multiplataforma, compatible tanto con sistemas que utilizan CPU como con aquellos que emplean GPU, lo que le otorga flexibilidad en términos de hardware y facilita su integración en distintos entornos tecnológicos.

FAISS se ha convertido en una herramienta clave para realizar búsquedas de similitud en grandes bases de datos de vectores, mejorando el rendimiento y la escalabilidad de aplicaciones de inteligencia artificial y machine learning que dependen de la búsqueda de alta precisión y rapidez.

LangChain. LangChain es un marco de desarrollo diseñado para crear aplicaciones que integran modelos de lenguaje, como los de OpenAI, con otras fuentes de datos y herramientas. Su objetivo principal es ayudar a los desarrolladores a construir aplicaciones que aprovechen el poder de los modelos de lenguaje, pero con la capacidad de interactuar de manera eficiente con bases de datos, APIs y entornos externos, brindando respuestas más precisas y útiles. Este marco permite estructurar las interacciones de los modelos de lenguaje en torno a dos componentes principales. Por un lado, las cadenas de pasos (*chains*) representan flujos de trabajo secuenciales o ramificados que conectan varias operaciones o transformaciones de datos. En LangChain, es posible encadenar múltiples llamadas a modelos de lenguaje o integrar pasos adicionales, como consultas a bases de datos o procesamiento de resultados. Por otro lado, los agentes permiten la interacción con herramientas o servicios externos, como motores de búsqueda, APIs o sistemas de archivos. Estos agentes posibilitan que los modelos de lenguaje ejecuten acciones como obtener información en tiempo real, interactuar con otras plataformas o realizar cálculos.

Entre las aplicaciones más comunes de LangChain se encuentran los asistentes

conversacionales avanzados, que permiten a los modelos de lenguaje acceder a información dinámica desde bases de datos, realizar consultas o interactuar con sistemas de terceros, mejorando así la calidad y utilidad de las respuestas. También es útil para la automatización de flujos de trabajo, ya que al integrar modelos de lenguaje con servicios externos, LangChain ayuda a automatizar tareas complejas que requieren varios pasos, como la generación de informes, el análisis de datos o el procesamiento de texto. Además, facilita la recuperación de información, permitiendo construir aplicaciones que no solo generan texto, sino que también lo relacionan con datos almacenados en fuentes externas, como bases de datos o documentos, optimizando la precisión de las respuestas.

LangChain es una herramienta poderosa para aprovechar las capacidades de los modelos de lenguaje y conectarlas con sistemas del mundo real, facilitando el desarrollo de aplicaciones más inteligentes y funcionales.

Búsqueda por similitud utilizando la distancia coseno. La distancia coseno es una métrica utilizada para medir la similitud entre dos vectores en un espacio multidimensional, especialmente en el contexto de *embeddings* o representaciones vectoriales. A diferencia de otras medidas como la distancia euclidiana, que se enfoca en la magnitud de los vectores, la distancia coseno se centra en el ángulo entre ellos.

El concepto clave detrás de la distancia coseno es evaluar cuán alineados están dos vectores, sin importar su tamaño. Si dos vectores están en la misma dirección, el ángulo entre ellos será cercano a 0 grados, y por lo tanto, el coseno de ese ángulo será cercano a 1, lo que indica una alta similitud. Por el contrario, si los vectores están en direcciones completamente opuestas (ángulo de 180 grados), el coseno del ángulo será -1, lo que sugiere que los vectores son completamente diferentes.

La fórmula de la distancia coseno se define como:

$$\text{distancia coseno} = 1 - \frac{A \cdot B}{||A|| \times ||B||} \quad (1)$$

Donde:

- A y B son los dos vectores.
- $A \cdot B$ es el producto punto entre los dos vectores.

- $\|A\|$ y $\|B\|$ son las magnitudes (normas) de los vectores A y B , respectivamente.

El valor resultante de la distancia coseno varía entre 0 y 1. Un valor de 0 indica que los vectores son completamente similares (idénticos en dirección), mientras que un valor de 1 indica que los vectores son completamente diferentes (perpendiculares o sin relación en términos de dirección).

Esta medida es especialmente útil cuando se trabaja con datos de alta dimensionalidad, como en el procesamiento de lenguaje natural o la búsqueda de imágenes, ya que permite encontrar elementos similares basándose en sus representaciones vectoriales, sin verse afectada por la magnitud de los vectores, solo por la orientación relativa de los mismos.

Desarrollo de la primera iteración. Como primera opción se crea un sistema compuesto por un caché de mensajes alojado en una base local de Redis y una base de vectores alojada en una instancia local de FAISS. En la base de vectores se guardarán los datos que se vayan recabando de los usuarios. Cada dato estará identificado por un campo llamado `person_id`. En las dos bases se guardará información de diferentes personas.

Se comienza importando la librería `langchain` para facilitar la interacción con las diferentes herramientas.

Se prueba guardar un solo dato para comprobar el funcionamiento de punta a punta del sistema.

El código implementado puede consultarse en detalle en el repositorio¹:

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS
import os
import redis

from dotenv import load_dotenv
load_dotenv()

redis_host = os.getenv("REDIS_HOST") or "localhost"
redis_port = os.getenv("REDIS_PORT") or 6379
```

¹ <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/1.ipynb>

```

def connect_to_redis():
    return redis.Redis(host=redis_host, port=redis_port, db=0)

def check_index_existance(index="main") -> bool:
    redis_conn = connect_to_redis()
    index_key = f"indexes:{index}"
    if redis_conn.get(index_key) is None:
        return False
    else:
        return True

messages = ["Hello, my name is John. I am a software engineer."]
index = "main"

# FAISS
# langchain.vectorstores.faiss.FAISS
# Creamos un index general y accedemos a la data del usuario por metadata.
# {person_id:str}

embeddings = OpenAIEmbeddings()
metadatas = [{"person_id": "john", "person_name": "John"}]
conversations = FAISS.from_texts(texts=["hi"], embedding=embeddings,
                                  metadatas=metadatas)

metadatas = [{"person_id": "mary", "person_name": "Mary"}]
conversations.add_texts(texts=["hi whats up?"], metadatas=metadatas)
# ['027065ea-f01a-448d-aae2-93c05ca62768']

conversations.similarity_search(query="hi", k=2, filter={"person_id": "mary"})
# [Document(page_content='hi whats up?',
#           metadata={'person_id': 'mary', 'person_name': 'Mary'})]

```

Como conclusión se extrae que no se necesita un filtrado estricto por `person_id` sino que en condiciones de laboratorio se puede acceder a la información propia de un usuario mediante distancia coseno, que es el algoritmo por defecto del motor de búsqueda de la base vectorial

FAISS.

No es necesario armar indexes para cada persona, ni siquiera ser estricto con el schema, uno puede extraer el vector que quiera por similitud semántica.

Creación de una Base de Mensajes del Usuario y Activación de Filtros

En un entorno productivo, lo visto en el apartado anterior no es aplicable. Es necesario hacer una búsqueda por distancia coseno dentro de un espacio vectorial filtrado por `person_id` de manera estricta. En la misma iteración se experimenta con la creación de una base vectorial en la que cada vector es un mensaje que el usuario tuvo con el asistente. En ese mensaje el usuario brinda una partícula de información importante que refuerza el nivel de conocimiento que el sistema tiene del usuario.

La recuperación se da entonces mediante un filtrado por `person_id` y luego por similitud semántica entre una pregunta y el mensaje del usuario que mayor relación tenga con esa pregunta.

El código de la notebook² realiza una búsqueda por distancia coseno dentro de un espacio vectorial filtrado por `person_id`, con el objetivo de encontrar el mensaje del usuario que más relación tenga con una pregunta mediante similitud semántica. A continuación se describen los pasos:

Importación de Librerías y Configuración. Creación de embeddings: Se genera un objeto embeddings usando `OpenAIEmbeddings`, que se usará para transformar texto en vectores numéricos, permitiendo calcular la similitud entre textos.

Generación de metadatos: Se crea un diccionario con la información del usuario (`person_id` y `person_name`), que se añadirá como metadatos a cada mensaje guardado en la base de datos vectorial.

Creación de un mensaje: Se crea un mensaje con el contenido “hola, quiero ir a Río” y un timestamp que marca el momento de creación. Este mensaje será transformado en un vector.

Procesamiento y Almacenamiento. Conversión a texto y vectorización: El contenido del mensaje se convierte a texto con `str(message)` y luego es transformado en un vector mediante `FAISS.from_texts()`. El índice vectorial se almacena junto con los metadatos.

Almacenamiento de la base vectorial: El índice de vectores se guarda localmente con `conversations.save_local()`, que crea un archivo en la ruta especificada para almacenar el índice.

² Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/2.ipynb>

Búsqueda por Similitud Semántica. Se realiza una búsqueda usando `conversations.similarity_search_with_score()`. La pregunta es “¿A dónde quería ir el cliente?” y se filtra por `person_id` “alexander.ditzend”. Se busca el mensaje más similar semánticamente a la pregunta.

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS
import os
import sys
import redis
from datetime import datetime
from dotenv import load_dotenv

sys.path.append('/Users/alexander/Projects/hyperpersonalization/app/tools')
from vectordb import connect_to_redis, check_index_existance, add_message_to_db

load_dotenv()

redis_host = os.getenv("REDIS_HOST") or "localhost"
redis_port = os.getenv("REDIS_PORT") or 6379

embeddings = OpenAIEmbeddings()
metadatas = [{"person_id": "alexander.ditzend", "person_name": "Alexander"}]
timestamp = datetime.now().isoformat()
message = {"timestamp": timestamp, "content": "hola, quiero ir a Río"}
text = str(message)
conversations = FAISS.from_texts(texts=[text], embedding=embeddings,
                                metadatas=metadatas)
conversations.save_local(folder_path="../indexes/conversations",
                        index_name="conversations")

conversations.similarity_search_with_score(
    query="¿A dónde quería ir el cliente?",
    filter={"person_id": "alexander.ditzend"}
)
```

Conclusiones de la segunda iteración. El resultado devuelto es el mensaje “hola, quiero ir a Río” con un puntaje de similitud de 0.50811493. Este puntaje resulta de la comparación mediante la distancia coseno entre la pregunta y el contenido del mensaje:

```
[(Document(page_content="{ 'timestamp': '2023-07-12T12:00:22.538965',  
                           'content': 'hola, quiero ir a Río'}",  
           metadata={'person_id': 'alexander.ditzend',  
                     'person_name': 'Alexander'}),  
  0.50811493)]
```

Se menciona que la distancia coseno no es útil en un conjunto con un solo vector, ya que no hay otros vectores con los cuales comparar. Sin embargo, el ejemplo demuestra la factibilidad de la aproximación en búsquedas por similitud semántica.

El sistema filtra los vectores por un identificador de usuario y realiza una búsqueda semántica usando la distancia coseno para encontrar el mensaje que más se asemeja a la consulta planteada, permitiendo recuperar información relevante basada en los mensajes previos del usuario. Se busca ahora hacer una búsqueda por similitud semántica entre una pregunta y una pieza de información provista por el usuario.

Claramente la distancia coseno no es un parámetro válido en un conjunto donde hay un solo vector pero con este simple ejemplo alcanza para comprobar la factibilidad de la aproximación.

Similitud entre consultas y extractos de conversaciones

En la notebook 3³ se realiza un experimento en el que se utiliza la búsqueda semántica en un espacio vectorial basado en las conversaciones del usuario. El objetivo es encontrar, mediante similitud semántica, la información más relevante de las interacciones del usuario con el asistente, siempre filtrando por el identificador de persona (`person_id`). A continuación, se describen los pasos observados:

Carga de librerías y configuración. Se importan las librerías necesarias para generar embeddings y realizar búsquedas vectoriales utilizando FAISS, además de cargar Redis y las variables de entorno necesarias para configurar el sistema.

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS
import os
import sys
import redis
from datetime import datetime
from dotenv import load_dotenv

sys.path.append('/Users/alexander/Projects/hyperpersonalization/app/tools')
from vectordb import connect_to_redis, check_index_existance, add_message_to_db

load_dotenv()

redis_host = os.getenv("REDIS_HOST") or "localhost"
redis_port = os.getenv("REDIS_PORT") or 6379
index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
                      embeddings=embeddings)
```

³ Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/3.ipynb>

Carga de un índice previamente almacenado. Se carga un índice de vectores existente desde el almacenamiento local. Este índice contiene las conversaciones previas del usuario y se utiliza para realizar las búsquedas posteriores. Los embeddings permiten comparar los textos de manera semántica.

Adición de mensajes a la base vectorial. Cada fragmento de conversación del usuario se convierte en un vector mediante el cálculo de embeddings, y se almacena en la base vectorial junto con los metadatos, como el `person_id` y el `person_name`. Esto permite compartimentalizar la información y facilitar el filtrado por usuario.

Ejemplos de mensajes añadidos.

- “mi madre se llama Alicia, tiene 66, es jubilada”
- “mis hijas se llaman Aurora y Alba, Aurora tiene 12 y Alba 9”
- “a mis hijas no les gusta estar en el pasillo, siempre ventana”

```
metadatas = [{"person_id": "alexander.ditzend", "person_name": "Alexander"}]
timestamp = datetime.now().isoformat()
message = {"timestamp": timestamp,
           "content": "mi madre se llama Alicia, tiene 66, es jubilada"}
text = str(message)
db.add_texts(texts=[text], embedding=embeddings, metadatas=metadatas)
```

```
# Resultado: ['2b4e7776-2ba1-40c6-a5b8-4c52c7bfd09d']
```

```
metadatas = [{"person_id": "alexander.ditzend", "person_name": "Alexander"}]
timestamp = datetime.now().isoformat()
message = {"timestamp": timestamp,
           "content": "mis hijas se llaman Aurora y Alba, Aurora tiene 12 y
↪ Alba 9"}
text = str(message)
db.add_texts(texts=[text], embedding=embeddings, metadatas=metadatas)
```

```
# Resultado: ['8365053e-e582-4bdb-9892-36d2499a78a1']
```

```

metadata = [{"person_id": "alexander.ditzend", "person_name": "Alexander"}]
timestamp = datetime.now().isoformat()
message = {"timestamp": timestamp,
           "content": "a mis hijas no les gusta estar en el pasillo, siempre
↪ ventana"}
text = str(message)
db.add_texts(texts=[text], embedding=embeddings, metadata=metadata)

# Resultado: ['89b188cb-5eed-44fe-92d8-cc62c077e676']

```

Realización de consultas semánticas. Se formulan diversas preguntas al sistema, que se comparan con las conversaciones almacenadas, aplicando siempre un filtro por `person_id` para obtener únicamente los datos del usuario relevante.

Consulta 1: “¿Cuántos años tiene la madre del cliente?”

El sistema devuelve el mensaje “mi madre se llama Alicia, tiene 66, es jubilada” con un puntaje de similitud.

```

query = "cuantos años tiene la madre del cliente?"
filter = {"person_id": "alexander.ditzend"}
db.similarity_search_with_score(query=query, embeddings=embeddings,
                                filter=filter, k=1)

# Resultado:
# [(Document(page_content="{ 'timestamp': '2023-07-12T12:20:08.065627',
#   'content': 'mi madre se llama Alicia, tiene 66, es jubilada' }"),
#   metadata={ 'person_id': 'alexander.ditzend', 'person_name': 'Alexander' }),
#   0.43335056)]

```

Consulta 2: “¿Tiene hijos?”

Se devuelven dos mensajes: “a mis hijas no les gusta estar en el pasillo, siempre ventana” y “mis hijas se llaman Aurora y Alba, Aurora tiene 12 y Alba 9”.

```

query = "tiene hijos?"
filter = {"person_id": "alexander.ditzend"}
db.similarity_search_with_score(query=query, embeddings=embeddings,
                                filter=filter, k=2)

# Resultado:
# [(Document(page_content="{ 'timestamp': '2023-07-12T12:23:34.853229',
#   'content': 'a mis hijas no les gusta estar en el pasillo, siempre
↪ ventana'}"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#    0.45402932),
#  (Document(page_content="{ 'timestamp': '2023-07-12T12:22:19.835459',
#   'content': 'mis hijas se llaman Aurora y Alba, Aurora tiene 12 y Alba
↪ 9'}"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#    0.46825188)]

```

Consulta 3: “¿Cuáles son las preferencias de los hijos?”

El sistema devuelve que las hijas prefieren sentarse junto a la ventana.

```

query = "cuales son las preferencias de los hijos?"
filter = {"person_id": "alexander.ditzend"}
db.similarity_search_with_score(query=query, embeddings=embeddings,
                                filter=filter, k=2)

# Resultado:
# [(Document(page_content="{ 'timestamp': '2023-07-12T12:23:34.853229',
#   'content': 'a mis hijas no les gusta estar en el pasillo, siempre
↪ ventana'}"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#    0.48871845),
#  (Document(page_content="{ 'timestamp': '2023-07-12T12:22:19.835459',
#   'content': 'mis hijas se llaman Aurora y Alba, Aurora tiene 12 y Alba
↪ 9'}"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#    0.53449)]

```


Consulta 4: “¿Edad de la hija mayor?”

El sistema devuelve el mensaje con la edad de Aurora, la hija mayor.

```
query = "edad de la hija mayor"
filter = {"person_id": "alexander.ditzend"}
db.similarity_search_with_score(query=query, embeddings=embeddings,
                                filter=filter, k=2)

# Resultado:
# [(Document(page_content="{ 'timestamp': '2023-07-12T12:22:19.835459',
#   'content': 'mis hijas se llaman Aurora y Alba, Aurora tiene 12 y Alba
↪ 9' }"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#   0.42045504),
# (Document(page_content="{ 'timestamp': '2023-07-12T12:23:34.853229',
#   'content': 'a mis hijas no les gusta estar en el pasillo, siempre
↪ ventana' }"),
#   metadata={'person_id': 'alexander.ditzend', 'person_name': 'Alexander'}),
#   0.44310012)]
```

Conclusiones de la tercera iteración. En todos los casos, las respuestas del sistema son correctas, lo que confirma que la búsqueda por similitud usando embeddings es efectiva en este experimento. No obstante, se aclara que el experimento es limitado por el pequeño número de vectores, y las conclusiones no pueden extrapolarse a un escenario real de mayor escala. La notebook 3 implementa un proceso en el cual se añaden fragmentos de conversación a una base vectorial, se asocian con metadatos, y luego se realizan búsquedas semánticas para recuperar el mensaje más relevante. El filtrado por `person_id` garantiza que las búsquedas se limiten a la información específica del usuario relevante.

Implementación de Búsquedas Semánticas con Redis y FAISS para la Gestión de Sesiones de Usuario

En la notebook 4 se realiza un experimento utilizando Redis para almacenar y recuperar información de sesiones y mensajes asociados a un usuario, y posteriormente realizar búsquedas semánticas sobre esos datos utilizando FAISS. A continuación se describen los pasos observados en la notebook ⁴.

Carga de Librerías y Configuración. Se importan las librerías necesarias, como Redis para gestionar la base de datos en memoria, y FAISS para realizar búsquedas semánticas sobre los datos almacenados. También se configuran las variables de entorno necesarias, como el host y puerto de Redis.

```
import os
import sys
import redis
from dotenv import load_dotenv
load_dotenv()
redis_host = os.getenv("REDIS_HOST") or "localhost"
redis_port = os.getenv("REDIS_PORT") or 6379
```

Almacenamiento de Información en Redis. Se definen varios objetos de tipo JSON que representan a un usuario (en este caso llamado Jane) y sus respectivas sesiones de interacción con el sistema. Cada sesión contiene una serie de mensajes que fueron intercambiados entre el usuario y el sistema.

```
from redis.commands.json.path import Path
client = redis.Redis(host=redis_host, port=redis_port, db=0)
jane = {
    'name': "Jane",
    'Age': 33,
    'Location': "Chawton"
}
```

⁴ Código disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/4.ipynb>

```
client.json().set('person:1', '$', jane)
result = client.json().get('person:1')
print(result)
```

Ejemplo de estructura almacenada: 'session_id': '10', 'person_id': '10', 'name': 'Jane', 'messages': [{ 'role': 'user', 'content': 'Hello' }, { 'role': 'system', 'content': 'Hello 2' }].

Recuperación de una Sesión Específica. Se realizan consultas en Redis para recuperar una sesión particular del usuario, utilizando el session_id. Esto permite obtener solo los datos de esa sesión sin necesidad de cargar toda la información del usuario.

```
session_header = {
    'session_id': "10",
    'person_id': "10",
    'name': "Jane",
    'messages': []
}

message1 = {
    'role': "user",
    'content': "Hello"
}

message2 = {
    'role': "system",
    'content': "Hello 2"
}

client.json().set('session:10', '$', session_header)
client.json().arrappend('session:10', '.messages', message1)
client.json().arrappend('session:10', '.messages', message2)

result = client.json().get('session:10')
print(result)
```

Agregación de Mensajes a la Sesión. Se almacenan múltiples sesiones asociadas al mismo usuario, y se van añadiendo mensajes a cada sesión. La estructura de los mensajes incluye

información sobre el rol (usuario o sistema) y el contenido del mensaje.

```
session_header = {
    'session_id': "20",
    'person_id': "10",
    'name': "Jane",
    'messages': []
}

message1 = {
    'role': "user",
    'content': "holis"
}

message2 = {
    'role': "system",
    'content': "re holis 2"
}

client.json().set('session:20', '$', session_header)
client.json().arrappend('session:20', '.messages', message1)
client.json().arrappend('session:20', '.messages', message2)
```

Almacenamiento en FAISS. Después de recuperar la información de las sesiones desde Redis, se convierte el contenido de los mensajes en texto plano, y se calculan los embeddings utilizando OpenAIEmbeddings. Los mensajes se almacenan en FAISS con metadatos que incluyen el `person_id` y el `session_id` del usuario, lo que permite filtrar las búsquedas por estos campos.

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
```

```
embeddings=embeddings)

session10 = client.json().get('session:10')

metadatas = [{
    'session_id': "10",
    'person_id': "10",
}]

# turn json to plain text
content = session10['messages']
str(content)

db.add_texts(texts=[str(content)], embedding=embeddings,
             metadatas=metadatas)
```

Búsquedas Semánticas en FAISS. Se realizan varias consultas para buscar sesiones o mensajes específicos en función de una consulta de texto. FAISS busca por similitud semántica entre la consulta y los mensajes almacenados, devolviendo el mensaje más cercano. Por ejemplo, una consulta como «¿cuál es la primera sesión?» devuelve la sesión más relacionada según la similitud semántica de los mensajes.

```
query = "cual es la segunda sesion?"
filter = {"person_id": "10"}
db.similarity_search_with_score(query=query, embeddings=embeddings,
                               filter=filter, k=1)
```

Uso de Marcas Temporales para Mejorar la Búsqueda. Se añade un campo de marca temporal (timestamp) a los mensajes en formato semántico para poder identificar mejor el orden de los mensajes o sesiones, ya que solo con la similitud semántica no siempre se logra obtener resultados precisos en cuanto al orden cronológico.

```
import datetime
import locale

# Set the locale to Spanish
locale.setlocale(locale.LC_TIME, 'es_ES')

timestamp = datetime.datetime.now()
formatted_timestamp = timestamp.strftime('%A, %d de %B de %Y, %H:%M:%S')
print(formatted_timestamp)

session_header = {
    'session_id': "1000",
    'person_id': "1000",
    'name': "Jane",
    'messages': []
}

formatted_timestamp = "lunes, 17 de julio de 2023, 14:30:00"
message1 = {
    'timestamp': formatted_timestamp,
    'role': "user",
    'content': "Hello"
}

formatted_timestamp = "lunes, 17 de julio de 2023, 14:31:00"
message2 = {
    'timestamp': formatted_timestamp,
    'role': "system",
    'content': "Hello 2"
}

client.json().set('session:1000', '$', session_header)
client.json().arrappend('session:1000', '.messages', message1)
client.json().arrappend('session:1000', '.messages', message2)
```

Conclusiones de la cuarta iteración. El sistema devuelve los mensajes más similares a las consultas realizadas, basándose en la búsqueda semántica. Aunque la búsqueda semántica funciona bien, se observa que es necesario añadir marcas temporales o más contexto para mejorar los resultados en casos donde el orden de los mensajes es relevante.

```
query = "lunes, 17 de julio de 2023, 14:31:00", 'role': 'system',  
        'content': 'Hello 2'  
  
filter = {"person_id": "1000"}  
db.similarity_search_with_score(query=query, embeddings=embeddings,  
                                filter=filter, k=3)
```

La notebook 4 implementa un sistema que almacena sesiones de usuario en Redis, realiza búsquedas semánticas en FAISS basándose en los mensajes de esas sesiones, y mejora el proceso añadiendo marcas temporales para organizar los resultados cronológicamente.

Análisis del Proceso de Vectorización de Conversaciones y Búsqueda Semántica

En la notebook 5

(<https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/5.ipynb>) se simula una base de datos de conversaciones entre un asistente y un usuario, y se explora cómo el sistema puede navegar cronológicamente por esa base para extraer información útil durante una conversación. A continuación, se analizan los pasos que se han llevado a cabo:

Carga de Librerías y Configuración Inicial. Se utiliza FAISS para almacenar y buscar de manera eficiente textos vectorizados basados en las conversaciones entre el asistente y el usuario. Se cargan las embeddings de OpenAI, y se define la ruta para el almacenamiento de los vectores.

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
                      embeddings=embeddings)
```

Simulación de una Base de Datos de Conversaciones. Se crea una lista con varios mensajes entre el usuario y el sistema, donde cada mensaje contiene metadatos como `session_id`, `person_id`, `message_id` y un timestamp (`created_at`). Cada intercambio de mensajes entre el usuario y el asistente queda registrado cronológicamente, incluyendo preguntas y respuestas como “¿Cuál es la capital de Argentina?” y “Buenos Aires”.

```
conversation_db = [
    {
        "message": {
            "role": "user",
            "content": "Cual es la capital de Argentina?"
        },
        "metadata": {
```



```
        "session_id": "1",
        "person_id": "20",
        "message_id": "1-1",
        "created_at": "20230825T12:00:00.000Z",
    },
},
{
    "message": {
        "role": "system",
        "content": "Buenos Aires"
    },
    "metadata": {
        "session_id": "1",
        "person_id": "20",
        "message_id": "1-2",
        "created_at": "20230825T12:01:00.000Z"
    }
},
# ... más mensajes sobre Chile, Perú y Colombia
]
```

Vectorización de los Mensajes. Los mensajes de las conversaciones son convertidos en texto plano y luego vectorizados. Los embeddings resultantes se añaden a la base vectorial de FAISS junto con los metadatos, lo que permite que las búsquedas futuras no solo tengan en cuenta el contenido del mensaje, sino también su contexto, como el identificador de la sesión y el momento en que fue creado.

```

for item in conversation_db:
    print(str(item['message']))
    print(item['metadata'])
    db.add_texts(
        texts=[str(item['message'])],
        metadatas=[item['metadata']],
    )

```

Búsqueda Semántica y Razonamiento Contextual. Se realizan búsquedas basadas en las consultas del usuario, como “¿Hace cuánto hablamos de Perú?” o “¿Te pregunté por Brasil?”. FAISS busca el mensaje más relevante utilizando similitud semántica. Una vez que se encuentra el mensaje adecuado, se genera un contexto que se pasa como entrada al modelo de lenguaje GPT-3.5. Esto permite al sistema recordar detalles previos de la conversación, como fechas y temas discutidos, y generar una respuesta precisa.

```

user_input_1 = "hace cuanto hablamos de peru?"
filter = {"person_id": "20"}
db_results_1 = db.similarity_search_with_score(query=user_input_1,
                                                embeddings=embeddings,
                                                filter=filter, k=1)

context_1 = "{ message: " + db_results_1[0][0].page_content +
            ", metadata: " + str(db_results_1[0][0].metadata) + " }"

```

Uso del Modelo de Lenguaje para Respuestas Informadas. Se construyen prompts basados en los resultados de la búsqueda semántica, los cuales incluyen tanto el contenido del mensaje como los metadatos asociados. El sistema utiliza este contexto para generar respuestas informadas. Por ejemplo, al preguntar “¿Alguna vez hablamos de Colombia?”, el modelo responde que el usuario había preguntado sobre Colombia en una sesión anterior, proporcionando el contexto cronológico y de contenido correspondiente.

```

def gpt35_system_user(system_message: str, user_message: str):
    try:
        response = requests.post(
            url="https://yoizenia.openai.azure.com/openai/deployments/GPT35Tur
            ↪ bo/chat/completions",
            params={
                "api-version": "2023-05-15",
                "temperature": "0",
            },
            headers={
                "Content-Type": "application/json",
                "api-key": "bd6603c266fc49389659225a451ff03d",
            },
            data=json.dumps({
                "messages": [
                    {
                        "content": system_message,
                        "role": "system"
                    },
                    {
                        "content": user_message,
                        "role": "user"
                    }
                ]
            })
        )
        choices = response.json()['choices']
        answer = {
            "status_code": response.status_code,
            "text": choices[0]['message'],
        }
        return answer
    except requests.exceptions.RequestException:
        print('HTTP Request failed')
        return None

```

Mejora con la Inclusión de Marcas Temporales. Se incorporan marcas temporales en los mensajes para que el asistente pueda proporcionar respuestas más precisas, especificando cuándo se discutió un tema determinado. Esto mejora la capacidad del sistema para manejar consultas que requieren un entendimiento temporal, como “¿Hace cuánto te pregunté por Argentina?”.

```
user_input_3 = "hace cuanto te habia preguntado por argentina?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(query=user_input_3,
                                                embeddings=embeddings,
                                                filter=filter, k=1)

context_3 = "{ message: " + db_results_3[0][0].page_content +
            ", metadata: " + str(db_results_3[0][0].metadata) + " }"

system_prompt_3 = f"""
You are a personal assistant.
You have access to all previous conversations with your user.
All your responses should be based on the previous message provided.
Previous message: {context_3}
"""

answer_3 = gpt35_system_user(system_message=system_prompt_3,
                             user_message=user_input_3)
```

El resultado de esta consulta fue: “Según la metadata de la conversación, me preguntaste sobre la capital de Argentina el 25 de Agosto del 2023 a las 12:00:00 PM (UTC). ¿Hay algo más en lo que te pueda ayudar?”

Conclusiones de la quinta iteración. Se ha implementado un sistema en el que las conversaciones del usuario con el asistente son vectorizadas y almacenadas. El uso de metadatos y marcas temporales mejora la precisión de las respuestas generadas por el modelo de lenguaje, proporcionando un razonamiento contextual cronológicamente coherente. Este enfoque permite al sistema mantener una memoria conversacional efectiva y responder preguntas sobre interacciones pasadas con precisión temporal y contextual.

Análisis del enfoque en la recuperación de información sobre un pariente del usuario

En la notebook 6

(<https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/6.ipynb>) se experimenta con el cambio de foco en una conversación, dirigiéndola hacia un pariente cercano del usuario para corroborar a quién corresponde la información proporcionada y verificar si el sistema puede recuperar correctamente esa información. A continuación, se analizan los pasos del experimento:

Carga de librerías y configuración. Se configuran las librerías necesarias, como FAISS para la búsqueda semántica y OpenAI Embeddings para la vectorización de los mensajes. También se incluye una función para interactuar con el modelo de lenguaje GPT-3.5 a través de una API.

```
import requests
import json

from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
                      embeddings=embeddings)
```

La función para interactuar con GPT-3.5 se define como:

```
def gpt35_system_user(system_message: str, user_message: str):
    try:
        response = requests.post(
            url="https://yoizenia.openai.azure.com/openai/deployments/GPT35Turbo/chat/completions",
            params={
                "api-version": "2023-05-15",
                "temperature": "0",
            },
            headers={
```

```

        "Content-Type": "application/json",
        "api-key": "bd6603c266fc49389659225a451ff03d",
    },
    data=json.dumps({
        "messages": [
            {
                "content": system_message,
                "role": "system"
            },
            {
                "content": user_message,
                "role": "user"
            }
        ]
    })
)
choices = response.json()['choices']
answer = {
    "status_code": response.status_code,
    "text": choices[0]['message'],
}
return answer
except requests.exceptions.RequestException:
    print('HTTP Request failed')
    return None

```

Simulación de una conversación. Se simula una conversación en la que el usuario solicita un turno para su madre. A lo largo de la conversación, el asistente le pide detalles adicionales, como el nombre y la edad de su madre. Estos intercambios se almacenan con sus respectivos metadatos, como `session_id`, `person_id`, `message_id` y la marca temporal (`created_at`). Ejemplos de mensajes:

- Usuario: “Quiero un turno para mi madre”.
- Asistente: “¿Cuál es el nombre de tu madre?”.

- Usuario: “Alba”.
- Asistente: “¿Qué edad tiene?”.
- Usuario: “66”.

La estructura de datos utilizada para almacenar los mensajes es:

```
sessions = [  
  {  
    "message": {  
      "role": "user",  
      "content": "Quiero un turno para mi madre"  
    },  
    "metadata": {  
      "session_id": "1",  
      "person_id": "20",  
      "message_id": "1-1",  
      "created_at": "20230825T12:00:00.000Z",  
    },  
  },  
  {  
    "message": {  
      "role": "system",  
      "content": "cual es el nombre de tu madre?"  
    },  
    "metadata": {  
      "session_id": "1",  
      "person_id": "20",  
      "message_id": "1-2",  
      "created_at": "20230825T12:01:00.000Z"  
    },  
  },  
  {  
    "message": {  
      "role": "user",  
      "content": "Alba"    },  
  },  
]
```

```
    },
    "metadata": {
      "session_id": "1",
      "person_id": "20",
      "message_id": "1-3",
      "created_at": "20230825T13:00:00.000Z",
    },
  },
  {
    "message": {
      "role": "system",
      "content": "y que edad tiene?"
    },
    "metadata": {
      "session_id": "1",
      "person_id": "20",
      "message_id": "1-4",
      "created_at": "20230825T13:01:00.000Z"
    }
  },
  {
    "message": {
      "role": "user",
      "content": "66"
    },
    "metadata": {
      "session_id": "1",
      "person_id": "20",
      "message_id": "1-5",
      "created_at": "20230825T14:00:00.000Z",
    },
  },
  {
    "message": {
      "role": "system",
      "content": "perfecto, tiene turno para el 30 de agosto a las 15:00"
```



```

    },
    "metadata": {
        "session_id": "1",
        "person_id": "20",
        "message_id": "1-6",
        "created_at": "20230825T14:01:00.000Z"
    }
}
]

```

Vectorización y almacenamiento. Cada mensaje de la conversación se convierte en texto plano y luego se vectoriza utilizando embeddings. Estos vectores se almacenan en FAISS junto con los metadatos para permitir una búsqueda eficiente en futuras consultas.

```

for session in sessions:
    print(str(session['message']))
    print(session['metadata'])
    db.add_texts(
        texts=[str(session['message'])],
        metadatas=[session['metadata']],
    )

```

Consulta sobre el pariente del usuario. Se realiza una consulta del usuario (“¿qué sabes de mi madre?”) para verificar si el sistema puede recuperar la información relevante proporcionada en conversaciones anteriores. FAISS busca en la base de datos de vectores utilizando similitud semántica, y el sistema trata de usar el resultado como contexto para generar una respuesta con el modelo GPT-3.5.

```

user_input_1 = "que sabes de mi madre?"
filter = {"person_id": "20"}
db_results_1 = db.similarity_search_with_score(query=user_input_1,
                                                embeddings=embeddings,
                                                filter=filter, k=10)
context_1 = "{ message: " + db_results_1[0][0].page_content +
            ", metadata: " + str(db_results_1[0][0].metadata) + " }"

```

```

print(f"CONTEXT:{context_1}")

system_prompt_1 = f"""
Answer the user's question this message as context.
Previous message: {context_1}
"""

answer_1 = gpt35_system_user(system_message=system_prompt_1,
                             user_message=user_input_1)

answer_1["text"] ["content"]

```

Problema en la recuperación de información. A pesar de que el sistema encuentra el mensaje relevante (“¿Cuál es el nombre de tu madre?”), el modelo de lenguaje no logra utilizar adecuadamente el contexto para generar una respuesta precisa. En lugar de utilizar la información sobre el nombre y la edad de la madre proporcionados anteriormente, el modelo responde de forma incorrecta, indicando que no tiene información sobre la madre del usuario. El contexto recuperado fue:

```

CONTEXT:{ message: {'role': 'system', 'content': 'cual es el nombre de tu
↪ madre?'},
metadata: {'session_id': '1', 'person_id': '20', 'message_id': '1-2',
'created_at': '20230825T12:01:00.000Z'} }

```

Sin embargo, la respuesta del modelo fue:

“Lo siento, pero no sé nada de tu madre. La pregunta anterior estaba siendo dirigida a un asistente virtual o chatbot. Si hay algo más en lo que pueda ayudarte, por favor házmelo saber.”

El modelo falla en tomar la información brindada en el contexto para elaborar la respuesta.

Conclusiones de la sexta iteración. La notebook 6 muestra un experimento en el que se intenta recuperar información sobre un pariente del usuario a partir de una conversación previa. Aunque FAISS parece recuperar los mensajes relevantes, el modelo de lenguaje no utiliza correctamente el contexto para generar una respuesta precisa. Esto sugiere que el manejo del contexto y la integración entre la base de datos de vectores y el modelo de lenguaje necesitan ser refinados para mejorar la precisión en la recuperación de información.

Análisis de la Mejora en la Estructura del Contexto para la Recuperación de Información

En la notebook 7 se experimenta con la reconfiguración de la estructura del contexto brindado al modelo, con el objetivo de mejorar la claridad y limpieza de los datos que se le proporcionan, y comprobar si este enfoque permite al modelo recuperar la información solicitada de manera más efectiva. El código completo está disponible en el repositorio: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/7.ipynb>. A continuación, se analizan los pasos seguidos en esta notebook:

Carga de Librerías y Configuración. Se utiliza FAISS para la búsqueda semántica y OpenAI Embeddings para la vectorización de los mensajes. Además, se implementa una función para interactuar con el modelo GPT-3.5 a través de la API.

```
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
                      embeddings=embeddings)
```

Estructura de Contextos. Se crean varios contextos de conversación simulados. Estos contienen mensajes entre el usuario y el sistema, relacionados con temas específicos como la solicitud de un turno para la madre del usuario y una consulta sobre la capital de Colombia. Cada conversación incluye metadatos como `session_id`, `person_id`, `message_id` y la marca temporal (`created_at`), lo que facilita su organización y recuperación posterior.

Ejemplos de mensajes:

- Usuario: “Quiero un turno para mi madre”.
- Asistente: “¿Cuál es el nombre de tu madre?”.

- Usuario: “Alba”.
- Asistente: “¿Y qué edad tiene?”.
- Usuario: “66”.
- Asistente: “El turno es para el 30 de agosto a las 15:00”.

```
contexts = [  
  [  
    {  
      "message": {  
        "role": "user",  
        "content": "Quiero un turno para mi madre"  
      },  
      "metadata": {  
        "session_id": "1",  
        "person_id": "20",  
        "message_id": "1-1",  
        "created_at": "20230825T12:00:00.000Z",  
      },  
    },  
    {  
      "message": {  
        "role": "system",  
        "content": "cual es el nombre de tu madre?"  
      },  
      "metadata": {  
        "session_id": "1",  
        "person_id": "20",  
        "message_id": "1-2",  
        "created_at": "20230825T12:01:00.000Z"  
      },  
    },  
    {  
      "message": {  
        "role": "user",
```

```
"content": "Alba"
},
"metadata": {
  "session_id": "1",
  "person_id": "20",
  "message_id": "1-3",
  "created_at": "20230825T13:00:00.000Z",
},
},
{
  "message": {
    "role": "system",
    "content": "y que edad tiene?"
  },
  "metadata": {
    "session_id": "1",
    "person_id": "20",
    "message_id": "1-4",
    "created_at": "20230825T13:01:00.000Z"
  }
},
{
  "message": {
    "role": "user",
    "content": "66"
  },
  "metadata": {
    "session_id": "1",
    "person_id": "20",
    "message_id": "1-5",
    "created_at": "20230825T14:00:00.000Z",
  },
},
{
  "message": {
    "role": "system",
```

```
"content": "perfecto, tiene turno para el 30 de agosto a las 15:00"
},
"metadata": {
  "session_id": "1",
  "person_id": "20",
  "message_id": "1-6",
  "created_at": "20230825T14:01:00.000Z"
}
},
[
{
  "message": {
    "role": "user",
    "content": "Cual es la capital de Colombia?"
  },
  "metadata": {
    "session_id": "4",
    "person_id": "20",
    "message_id": "4-1",
    "created_at": "20230825T15:00:00.000Z",
  },
},
{
  "message": {
    "role": "system",
    "content": "Bogota"
  },
  "metadata": {
    "session_id": "4",
    "person_id": "20",
    "message_id": "4-2",
    "created_at": "20230825T15:01:00.000Z"
  }
}
]
```

```
]
```

Vectorización y Almacenamiento. Los contextos se convierten en texto plano y se vectorizan utilizando embeddings. Estos vectores se almacenan en FAISS con sus respectivos metadatos, lo que permite realizar búsquedas semánticas y recuperar la conversación completa cuando sea necesario.

```
condensed_contexts = [  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content": "Quiero un turno para mi madre"  
      },  
      {  
        "role": "system",  
        "content": "cual es el nombre de tu madre?"  
      },  
      {  
        "role": "user",  
        "content": "Alba"  
      },  
      {  
        "role": "system",  
        "content": "y que edad tiene?"  
      },  
      {  
        "role": "user",  
        "content": "66"  
      },  
      {  
        "role": "system",  
        "content": "perfecto, tiene turno para el 30 de agosto a las 15:00"  
      }  
    ],  
    "metadata": {
```

```
"session_id": "1",
"person_id": "20",
"context_id": "1",
"created_at": "20230825T12:00:00.000Z",
},
},
{
  "messages": [
    {
      "role": "user",
      "content": "Cual es la capital de Colombia?"
    },
    {
      "role": "system",
      "content": "Bogota"
    }
  ],
  "metadata": {
    "session_id": "1",
    "person_id": "20",
    "context_id": "2",
    "created_at": "20230825T18:00:00.000Z",
  }
}
```

Consulta sobre el Pariente del Usuario. Se realiza una consulta del usuario (“¿qué sabes de mi madre?”) para ver si el sistema puede recuperar correctamente la información relacionada con la madre proporcionada en conversaciones anteriores. FAISS encuentra el mensaje adecuado, pero el modelo de lenguaje no logra responder de manera precisa. El sistema responde que no tiene información adicional sobre la madre del usuario, lo cual sugiere un problema en la comprensión del contexto.


```

user_input_1 = "que sabes de mi madre?"
filter = {"person_id": "20"}
db_results_1 = db.similarity_search_with_score(query=user_input_1,
                                                embeddings=embeddings,
                                                filter=filter, k=1)
context_1 = "{ message: " + db_results_1[0][0].page_content +
            ", metadata: " + str(db_results_1[0][0].metadata) + " }"
print(f"CONTEXT:{context_1}")
system_prompt_1 = f"""
Answer the user's question this message as context.
Previous message: {context_1}
"""
answer_1 = gpt35_system_user(system_message=system_prompt_1,
                             user_message=user_input_1)
answer_1["text"]["content"]

```

Resultado: El modelo vuelve a fallar con una pregunta general, respondiendo: “Lo siento, como sistema de procesamiento de datos mi memoria solo retiene la información necesaria para responder a la solicitud de turnos de tu madre. No tengo acceso a información personal adicional sobre ella. ¿Hay algo más en lo que pueda ayudarte?”

Pregunta Específica sobre un Tema. Luego, se realiza una consulta más específica (“¿Una vez te pregunté por Colombia, no?”). En este caso, el sistema responde correctamente, recordando que la capital de Colombia es Bogotá. Esto demuestra que el sistema tiene un mejor rendimiento cuando la consulta es más directa y se enfoca en un tema específico previamente discutido.

```

user_input_2 = "Una vez te pregunte por colombia no?"
filter = {"person_id": "20"}
db_results_2 = db.similarity_search_with_score(query=user_input_2,
                                                embeddings=embeddings,
                                                filter=filter, k=1)
context_2 = "{ message: " + db_results_2[0][0].page_content +
            ", metadata: " + str(db_results_2[0][0].metadata) + " }"
print(f"CONTEXT:{context_2}")
system_prompt_2 = f"""

```

```

You are a personal assistant. You will find past conversations between
you and the user between backticks. Use them to answer the user's question.
```{context_2}```

"""

answer_2 = gpt35_system_user(system_message=system_prompt_2,
 user_message=user_input_2)

answer_2["text"]["content"]

```

**Resultado:** Con una pregunta específica sobre otro tema no tiene problemas. El sistema responde: “Sí, me hiciste una pregunta sobre Colombia y te respondí que la capital es Bogotá. ¿Puedo ayudarte en algo más?”

**Mejora en la Recuperación de Información.** Se vuelve a probar con una pregunta relacionada con el turno de la madre del usuario (“¿Te acuerdas cuándo era el turno de mi madre?”). Esta vez, el sistema logra recuperar correctamente la información, indicando que el turno es para el 30 de agosto a las 15:00. Esto sugiere que el problema inicial podría estar relacionado con la ambigüedad o generalidad de la primera consulta.

```

user_input_3 = "Te acuerdas cuando era el turno de mi vieja?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(query=user_input_3,
 embeddings=embeddings,
 filter=filter, k=1)

context_3 = "{ message: " + db_results_3[0][0].page_content +
 ", metadata: " + str(db_results_3[0][0].metadata) + " }"
print(f"CONTEXT:{context_3}")

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between
you and the user between backticks. Use them to answer the user's question.
```{context_3}```

"""

answer_3 = gpt35_system_user(system_message=system_prompt_3,
                             user_message=user_input_3)

answer_3["text"]["content"]

```

Resultado: Con otra pregunta específica sobre el tema que generó la falla anteriormente, se

observa que se puede hacer una recuperación satisfactoria. El sistema responde: “Sí, el turno de tu madre es para el 30 de agosto a las 15:00 horas.”

Conclusiones de la séptima iteración. La notebook 7 muestra que al reconfigurar y limpiar la estructura del contexto, se mejora la precisión en la recuperación de información en consultas específicas. Aunque el modelo sigue teniendo dificultades con preguntas generales, las consultas más directas y bien estructuradas generan respuestas más precisas y satisfactorias. Esto resalta la importancia de organizar adecuadamente el contexto para mejorar la interacción con modelos de lenguaje como GPT-3.5.

Análisis de la Interpretación de Fechas y Razonamiento Cronológico

En este experimento se evalúa la capacidad de los modelos de lenguaje GPT-3.5 y GPT-4 para interpretar fechas y realizar razonamiento cronológico. El objetivo es determinar si estos modelos pueden entender y calcular correctamente la edad de una persona en base a fechas y datos previamente proporcionados. El código completo de este análisis está disponible en el repositorio⁵.

Configuración del Entorno. Se utilizan FAISS para la búsqueda semántica y OpenAI Embeddings para la vectorización de mensajes. Se define una función para interactuar con los modelos GPT-3.5 y GPT-4 mediante la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

db = FAISS.load_local(folder_path=index_path, index_name=index_name,
                      embeddings=embeddings)

def gpt_system_user(
    system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
    """Para usar en notebooks"""
    try:
        response = requests.post(
            url="https://api.openai.com/v1/chat/completions",
            params={"temperature": "0"},
```

⁵ <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/8.ipynb>

```

headers={
    "Content-Type": "application/json",
    "Authorization": f"Bearer {OPENAI_API_KEY}",
},
data=json.dumps({
    "model": model,
    "messages": [
        {"content": system_message, "role": "system"},
        {"content": user_message, "role": "user"},
    ]
}),
timeout=10,
)
choices = response.json()["choices"]
answer = {
    "status_code": response.status_code,
    "text": choices[0]["message"],
}
return answer
except requests.exceptions.RequestException:
    print("HTTP Request failed")
    return None

```

Creación de Contextos Conversacionales. Se crean varios contextos que contienen conversaciones simuladas entre el usuario y el sistema. En uno de los contextos, el usuario solicita un turno para su madre, llamada Alba, y menciona que tiene 66 años. Otro contexto incluye una pregunta sobre la capital de Colombia.

```

condensed_contexts = [
    {
        "messages": [
            {"role": "user", "content": "Quiero un turno para mi madre"},
            {"role": "system", "content": "cual es el nombre de tu madre?"},
            {"role": "user", "content": "Alba"},
            {"role": "system", "content": "y que edad tiene?"},
            {"role": "user", "content": "66"},

```

```

        {"role": "system",
         "content": "perfecto, tiene turno para el 30 de agosto a las
         ↪ 15:00"}
    ],
    "metadata": {
        "session_id": "1",
        "person_id": "20",
        "context_id": "1",
        "created_at": "20230825T12:00:00.000Z",
    },
},
{
    "messages": [
        {"role": "user", "content": "Cual es la capital de Colombia?"},
        {"role": "system", "content": "Bogota"}
    ],
    "metadata": {
        "session_id": "1",
        "person_id": "20",
        "context_id": "2",
        "created_at": "20230825T18:00:00.000Z",
    }
}
]

```

Vectorización y Almacenamiento. Los mensajes de estos contextos se vectorizan utilizando embeddings y se almacenan en FAISS junto con sus respectivos metadatos, lo que permite realizar búsquedas y recuperar información relacionada.

```

for context in condensed_contexts:
    # Join all messages in one string by a comma
    messages = ''.join([str(message) for message in context['messages']])
    print(messages)
    print(context['metadata'])
    db.add_texts(
        texts=[messages],

```

```

        metadatas=[context['metadata']],
    )

```

Consulta Simple sobre la Edad. Se realiza una consulta simple: “¿Qué edad tiene Alba?”.

FAISS encuentra el contexto relevante, y el modelo GPT-3.5 responde correctamente que “Alba tiene 66 años”, basándose en la información previamente proporcionada.

```

user_input_3 = "que edad tiene alba?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
    query=user_input_3, embeddings=embeddings, filter=filter, k=1)
context_3 = "{ message: " + db_results_3[0][0].page_content + \
    ", metadata: " + str(db_results_3[0][0].metadata) + " }"
print(f"CONTEXT:{context_3}")

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you
and the user between backticks. Use them to answer the user's question.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
 user_message=user_input_3)
answer_3["text"]["content"]

```

El resultado obtenido fue: “Alba tiene 66 años.”

**Razonamiento Cronológico con GPT-3.5.** A continuación, se plantea una pregunta más compleja: “¿Qué edad tenía Alba en 2020?”. Aquí se espera que el modelo entienda que si Alba tiene 66 años en 2023 (año actual de acuerdo a la fecha de realización del experimento), entonces en 2020 tendría 63 años. Sin embargo, GPT-3.5 falla en su respuesta.

```

user_input_3 = "que edad tenía alba en 2020?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
 query=user_input_3, embeddings=embeddings, filter=filter, k=1)
context_3 = "{ message: " + db_results_3[0][0].page_content + \

```

```

 ", metadata: " + str(db_results_3[0][0].metadata) + " }"
print(f"CONTEXT:{context_3}")

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you
and the user between backticks. Use them to answer the user's question.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
                           user_message=user_input_3)
answer_3["text"]["content"]

```

GPT-3.5 responde incorrectamente: “En el contexto de la conversación anterior, Alba tenía 66 años en 2020.”, indicando que el modelo no ajusta la edad de acuerdo con la fecha solicitada.

Prueba con GPT-4. Se realiza la misma consulta sobre la edad de Alba en 2020, esta vez utilizando GPT-4.

```

user_input_3 = "que edad tenía alba en 2020?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
    query=user_input_3, embeddings=embeddings, filter=filter, k=1)
context_3 = "{ message: " + db_results_3[0][0].page_content + \
        ", metadata: " + str(db_results_3[0][0].metadata) + " }"
print(f"CONTEXT:{context_3}")

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you
and the user between backticks. Use them to answer the user's question.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
 user_message=user_input_3, model="gpt-4-0613")
answer_3["text"]["content"]

```

GPT-4 responde: “Alba nació en 1966. Por lo tanto, en 2020, tenía 54 años.” Aunque GPT-4 realiza el cálculo correctamente, comete un error inicial al interpretar la edad como el año de



nacimiento. A pesar de corregir el cálculo, este error inicial demuestra una falla en la comprensión de los datos proporcionados.

**Conclusiones de la octava iteración.** Tanto GPT-3.5 como GPT-4 presentan dificultades significativas al interpretar fechas y realizar razonamientos cronológicos basados en la edad de una persona. Mientras que GPT-3.5 no realiza el cálculo de la edad correctamente, GPT-4 comete un error de interpretación antes de realizar el cálculo adecuado.

Los resultados demuestran que:

- Los modelos pueden manejar consultas simples sobre datos proporcionados directamente
- Presentan serias dificultades cuando se les pide razonar cronológicamente
- GPT-3.5 no ajusta la edad según diferentes fechas
- GPT-4 malinterpreta inicialmente los datos numéricos (edad vs. año de nacimiento)

Esto sugiere que ambos modelos necesitan mejoras en la interpretación de datos numéricos y cronológicos cuando se extraen de contextos conversacionales. Es necesario implementar mecanismos adicionales para extraer y procesar datos numéricos correctamente en aplicaciones de hiperpersonalización.

*Influencia de la estructura del contexto en el razonamiento cronológico*

En la notebook 9 se experimenta con la forma en que la estructura del contexto afecta el razonamiento cronológico y la extracción de datos por parte de modelos de lenguaje como GPT-3.5. A continuación, se analizan los pasos principales del experimento realizado en 9.ipynb.

**Configuración del entorno experimental.** Se utiliza FAISS para la búsqueda semántica y OpenAI Embeddings para vectorizar los mensajes. Se configura también una función que interactúa con los modelos de lenguaje a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
 system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
 """Para usar en notebooks"""
 try:
 response = requests.post(
 url="https://api.openai.com/v1/chat/completions",
 params={"temperature": "0"},
 headers={
 "Content-Type": "application/json",
 "Authorization": f"Bearer {OPENAI_API_KEY}",
 },
 data=json.dumps({
 "model": model,
```

```

 "messages": [
 {"content": system_message, "role": "system"},
 {"content": user_message, "role": "user"},
]
 }),
 timeout=10,
)
choices = response.json()["choices"]
answer = {
 "status_code": response.status_code,
 "text": choices[0]["message"],
}
return answer
except requests.exceptions.RequestException:
 print("HTTP Request failed")
 return None

```

**Contextos condensados y simulación de conversaciones.** Se simulan varias conversaciones entre el usuario y el sistema. En una de ellas, el usuario pide un turno para su madre, llamada Alba, y menciona que tiene 66 años. En otra, el usuario pregunta la capital de Colombia.

#### **Ejemplos de mensajes de la conversación principal:**

- Usuario: “Quiero un turno para mi madre”.
- Asistente: “¿Cuál es el nombre de tu madre?”.
- Usuario: “Alba”.
- Asistente: “¿Qué edad tiene?”.
- Usuario: “66”.
- Asistente: “El turno es para el 30 de agosto a las 15:00”.

```

condensed_contexts = [
 {
 "messages": [
 {"role": "user", "content": "Quiero un turno para mi madre"},
 {"role": "system", "content": "cual es el nombre de tu madre?"},
 {"role": "user", "content": "Alba"},
 {"role": "system", "content": "y que edad tiene?"},
 {"role": "user", "content": "66"},
 {"role": "system",
 "content": "perfecto, tiene turno para el 30 de agosto a las
 ↪ 15:00"}
],
 "metadata": {
 "session_id": "1",
 "person_id": "20",
 "context_id": "1",
 "created_at": "20230825T12:00:00.000Z",
 },
 },
 {
 "messages": [
 {"role": "user", "content": "Cual es la capital de Colombia?"},
 {"role": "system", "content": "Bogota"}
],
 "metadata": {
 "session_id": "1",
 "person_id": "20",
 "context_id": "2",
 "created_at": "20230825T18:00:00.000Z",
 },
 }
]

```

**Extracción de datos con GPT-3.5.** En lugar de simplemente almacenar las conversaciones, se utiliza el modelo GPT-3.5 para extraer la información clave de cada conversación. Esto incluye detalles como el nombre y la edad de la madre del usuario, así como la fecha y hora en

que la información fue recopilada. Esta extracción se almacena en FAISS junto con los metadatos de la conversación.

```
db = FAISS.from_texts(texts=['notebook-9'], embedding=embeddings)

for context in condensed_contexts:
 messages = ''.join([str(message) for message in context['messages']])
 extraction = gpt_system_user(
 system_message="""
 Extract every meaningful information about the user from the provided
 conversation. Ask yourself, what happened in the conversation?
 What data about the user did you learn?
 At what date and time was this data gathered?
 """,
 user_message=f"METADATA: ```{context['metadata']}```
 CONVERSATION: ```{messages}```",
 model="gpt-3.5-turbo",
)
 db.add_texts(
 texts=[extraction['text']['content']],
 metadatas=[context['metadata']],
)
```

**Ejemplo de extracción obtenida:** “El usuario mencionó que quiere un turno para su madre. La madre del usuario se llama Alba y tiene 66 años.”

**Consultas simples y razonamiento cronológico.** Se comienza con una consulta simple: “¿Qué edad tiene Alba?”. El sistema encuentra el contexto relevante y responde correctamente: “Alba tiene 66 años”.

```
user_input_3 = "que edad tiene alba?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
 query=user_input_3, embeddings=embeddings, filter=filter, k=1
)
context_3 = "{ message: " + db_results_3[0][0].page_content +
 ", metadata: " + str(db_results_3[0][0].metadata) + " }"
```

```

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you and
the user between backticks. Use them to answer the user's question.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
    ↪ user_message=user_input_3)

```

Luego, se plantea una consulta más compleja: “¿Qué edad tenía Alba en 2020?”. En este caso, GPT-3.5 logra razonar correctamente y responde que Alba tenía 63 años en 2020, basándose en la edad actual de 66 años.

Consultas adicionales sobre fechas. Se continúa con una pregunta más difícil: “¿Qué edad tenía Alba en 1970?”. El sistema nuevamente razona correctamente y responde que Alba tenía 13 años en 1970, lo que demuestra que, con la forma correcta de estructurar y almacenar el contexto, GPT-3.5 es capaz de realizar razonamientos cronológicos precisos.

```

user_input_3 = "que edad tenía alba en 1970?"
# [Código similar al anterior con diferentes consultas temporales]

```

Resultados obtenidos:

- “¿Qué edad tiene Alba?” → “Alba tiene 66 años.”
- “¿Qué edad tenía Alba en 2020?” → “Alba tenía 63 años en 2020.”
- “¿Qué edad tenía Alba en 1970?” → “La edad de Alba en 1970 sería 13 años.”

Consulta sobre reconocimiento de datos personales. Finalmente, se realiza una consulta general: “¿Conoces a mi madre?”. El sistema responde de manera acertada, reconociendo a Alba y proporcionando los detalles previamente almacenados: “Sí, conozco a tu madre. Su nombre es Alba y tiene 66 años”.

```

user_input_3 = "conoces a mi madre?"
# [Implementación similar]
# Respuesta: 'Sí, conozco a tu madre. Su nombre es Alba y tiene 66 años.
#           ¿En qué puedo ayudarte hoy?'

```

Conclusiones de la novena iteración. La notebook 9 demuestra que la estructura del contexto influye significativamente en la capacidad del modelo para realizar razonamientos cronológicos y responder a consultas sobre datos específicos. Al extraer y almacenar los datos de forma clara y precisa, el modelo es capaz de utilizar esta información para responder correctamente incluso a preguntas complejas sobre fechas y edades, mejorando su rendimiento en tareas de razonamiento temporal.

Los resultados evidencian que:

- La extracción de información estructurada mejora la precisión de las respuestas
- GPT-3.5 puede realizar cálculos cronológicos cuando el contexto está bien organizado
- La vectorización semántica permite recuperar eficientemente información relevante
- El sistema mantiene coherencia en el reconocimiento de entidades personales a lo largo del tiempo

Agregado de un Paso de Extracción y Limpieza a un Conjunto de Datos con 4 Miembros de un Grupo Familiar

En la notebook 10⁶ se trabaja con un conjunto de datos que incluye a cuatro miembros de un grupo familiar, y se implementa un proceso de extracción y limpieza de datos, además de experimentos para verificar si el modelo de lenguaje puede razonar cronológicamente sobre la información proporcionada. A continuación, se detallan los pasos principales del experimento:

Carga de Librerías y Configuración. Se utiliza FAISS para la búsqueda semántica y OpenAI Embeddings para vectorizar los mensajes. También se configura una función para interactuar con el modelo GPT-3.5 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
    system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
    """Para usar en notebooks"""
    try:
        response = requests.post(
            url="https://api.openai.com/v1/chat/completions",
            params={
                "temperature": "0",
            },
            headers={
```

⁶ Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/10.ipynb>


```

        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}",
    },
    data=json.dumps(
        {
            "model": model,
            "messages": [
                {"content": system_message, "role": "system"},
                {"content": user_message, "role": "user"},
            ]
        }
    ),
    timeout=10,
)
choices = response.json()["choices"]
answer = {
    "status_code": response.status_code,
    "text": choices[0]["message"],
}
return answer
except requests.exceptions.RequestException:
    print("HTTP Request failed")
    return None

```

Creación de un Conjunto de Datos Familiar. Se simulan varias conversaciones en las que un usuario proporciona información sobre su familia. Por ejemplo, el usuario menciona que tiene una hija mayor llamada Ana de 20 años, un hijo menor llamado Juan de 10 años, y su esposa se llama María. Las fechas y detalles sobre un viaje familiar a Córdoba también son registrados, incluyendo las fechas de salida y regreso.

Ejemplos de mensajes:

- Usuario: “Voy a sacar un pasaje para la familia de Buenos Aires a Córdoba”.
- Asistente: “¿Quiénes son los pasajeros?”.
- Usuario: “Ana es mi hija mayor, tiene 20 años. Juan es mi hijo menor, tiene 10 años. Mi

esposa se llama María”.

```
condensed_contexts = [  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content": "Voy a sacar un pasaje para la familia de buenos  
↪ aires a cordoba"  
      },  
      {  
        "role": "system",  
        "content": "quienes son los pasajeros?"  
      },  
      {  
        "role": "user",  
        "content": "Ana es mi hija mayor, tiene 20 años. Juan es mi  
↪ hijo menor, tiene 10 años. Mi esposa se llama María"  
      },  
      {  
        "role": "system",  
        "content": "en que fecha quiere viajar?"  
      },  
      {  
        "role": "user",  
        "content": "del 20 de agosto al 30 de agosto"  
      },  
      {  
        "role": "system",  
        "content": "perfecto, salen el 20 de agosto a las 10:00 y  
↪ vuelven el 30 de agosto a las 18:00. El precio total es  
↪ 1000 dolares"  
      },  
      {  
        "role": "user",  
        "content": "si"      }  
    ]  
  }  
]
```

```

    },
    {
        "role": "system",
        "content": "Gracias por su compra"
    }
],
"metadata": {
    "session_id": "1",
    "person_id": "20",
    "context_id": "1",
    "created_at": "20200825T12:00:00.000Z",
},
}
]

```

Extracción de Datos. El modelo GPT-3.5 se utiliza para extraer la información significativa de cada conversación. Estos datos incluyen detalles sobre los miembros de la familia, las edades, las fechas de viaje y otros aspectos relevantes. La información extraída se almacena en FAISS junto con los metadatos, lo que facilita su recuperación posterior.

```

db = FAISS.from_texts(texts=['notebook-9'], embedding=embeddings)
for context in condensed_contexts:
    messages = ''.join([str(message) for message in context['messages']])
    extraction = gpt_system_user(
        system_message="""
        Extract every meaningful information about the user from the provided
        ↪ conversation.
        Ask yourself, what happened in the conversation?
        What data about the user did you learn?
        At what date and time was this data gathered?
        """,
        user_message=f"METADATA: ```{context['metadata']} CONVERSATION:
        ↪ ```{messages}```",
        model="gpt-3.5-turbo",
    )
    db.add_texts(

```

```

        texts=[extraction['text']['content']],
        metadatas=[context['metadata']],
    )

```

Ejemplo de extracción: “Pedro tiene una hija mayor llamada Ana, que tiene 20 años, y un hijo menor llamado Juan, que tiene 10 años. Su esposa se llama María”.

Consultas sobre Edades. Se plantea una consulta simple: “¿Qué edad tiene mi hija mayor?”. El sistema encuentra el contexto relevante y responde que Ana tiene 20 años, sin tener en cuenta que estamos en 2023, lo que indica una falta de razonamiento cronológico adecuado.

```

user_input_3 = "que edad tiene mi hija mayor?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
    query=user_input_3,
    embeddings=embeddings,
    filter=filter,
    k=1
)

context_3 = "{ message: " + db_results_3[0][0].page_content + ", metadata: " +
↳ str(db_results_3[0][0].metadata) + " }"

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you and
↳ the user between backticks.
Use them to answer the user's question.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
↳ user_message=user_input_3)

```

Respuesta: “Tu hija mayor, Ana, tiene 20 años.”

**Razonamiento Cronológico.** Luego se realiza una pregunta más compleja: “¿Qué edad tenía mi hija mayor cuando hicimos el viaje a Córdoba?”. En este caso, el modelo responde correctamente que Ana tenía 20 años durante el viaje, que ocurrió en 2020. Este resultado muestra que el modelo puede realizar un razonamiento correcto cuando se proporciona contexto cronológico.

```

user_input_3 = "que edad tenía mi hija mayor cuando hicimos el viaje a
↳ cordoba?"

[mismo código de búsqueda]

answer_3 = gpt_system_user(system_message=system_prompt_3,
↳ user_message=user_input_3)

```

Respuesta: “Tu hija mayor, Ana, tenía 20 años cuando hicieron el viaje a Córdoba.”

**Errores en el Cálculo de la Fecha de Nacimiento.** Se le pregunta al modelo: “¿En qué año nació mi hija mayor?”. Inicialmente, el modelo hace bien los cálculos y responde que Ana nació en el año 2000, pero no toma en cuenta que el año actual es 2023 de acuerdo a la fecha de realización del experimento, lo que debería ajustar la edad de Ana a 23 años. Tras ajustar el prompt y proporcionar la fecha actual explícitamente, el modelo aún comete errores, sugiriendo que Ana nació en 2003, lo que es incorrecto.

```

system_prompt_3 = f"""
You are a personal assistant.

You will find past conversations between you and the user between backticks.
Use them to answer the user's question.

To reason about dates, have in mind that today is September 13th, 2023.
You are now located in Buenos Aires, Argentina.
Take a deep breath and work on this problem step-by-step.
```{context_3}```
"""

```

Primera respuesta: “Según la información que tengo, su hija mayor, Ana, tiene 20 años. Si restamos 20 años a la fecha actual, podemos determinar que Ana nació en el año 2000.”

Segunda respuesta (con fecha explícita): “Según la información que tengo, tu hija mayor, Ana, tiene 20 años y la fecha actual es el 13 de septiembre de 2023. Si hacemos el cálculo, Ana nació en el año 2003.”

Conclusiones de la décima iteración. La notebook 10 muestra que, si bien los modelos de lenguaje pueden extraer y responder preguntas sobre datos familiares, tienen dificultades cuando se trata de razonamiento cronológico preciso. Aunque el sistema responde correctamente a consultas simples, como las edades durante eventos pasados, todavía comete

errores al calcular edades en función de la fecha actual. Esto sugiere la necesidad de mejorar la forma en que los modelos manejan fechas y razonamientos basados en el tiempo.

Razonamiento Cronológico y Extracción de Datos en Conversaciones Familiares

En la notebook 11 se implementa un experimento para extraer y organizar datos de conversaciones con un grupo familiar, buscando corregir el razonamiento cronológico del modelo de lenguaje GPT-3.5. A continuación se detallan los pasos principales del experimento.

Carga de Librerías y Configuración. Se utiliza FAISS para realizar la búsqueda semántica y OpenAI Embeddings para convertir los textos de las conversaciones en vectores. También se configura una función para interactuar con el modelo GPT-3.5 a través de la API de OpenAI.

```
import os
import requests
import json

from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
    system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
    """Para usar en notebooks"""
    try:
        response = requests.post(
            url="https://api.openai.com/v1/chat/completions",
            params={
                "temperature": "0",
            },
            headers={
                "Content-Type": "application/json",
                "Authorization": f"Bearer {OPENAI_API_KEY}",
            },
            data=json.dumps(
```

```

        {
            "model": model,
            "messages": [
                {"content": system_message, "role": "system"},
                {"content": user_message, "role": "user"},
            ]
        }
    ),
    timeout=10,
)

choices = response.json()["choices"]
answer = {
    "status_code": response.status_code,
    "text": choices[0]["message"],
}

return answer
except requests.exceptions.RequestException:
    print("HTTP Request failed")
    return None

```

Simulación de Conversaciones Familiares. Se crean varias conversaciones simuladas en las que el usuario proporciona información sobre su familia, como el hecho de tener una hija mayor llamada Ana de 20 años, y un hijo menor llamado Juan de 10 años. También se incluyen detalles de un viaje familiar a Córdoba con fechas y otros aspectos relevantes.

Ejemplos de mensajes:

- Usuario: “Voy a sacar un pasaje para la familia de Buenos Aires a Córdoba”.
- Asistente: “¿Quiénes son los pasajeros?”.
- Usuario: “Ana es mi hija mayor, tiene 20 años. Juan es mi hijo menor, tiene 10 años”.


```
condensed_contexts = [
  {
    "messages": [
      {
        "role": "user",
        "content": "Voy a sacar un pasaje para la familia de buenos
↪ aires a cordoba"
      },
      {
        "role": "system",
        "content": "quienes son los pasajeros?"
      },
      {
        "role": "user",
        "content": "Ana es mi hija mayor, tiene 20 años. Juan es mi
↪ hijo menor, tiene 10 años. Mi esposa se llama María"
      },
      {
        "role": "system",
        "content": "en que fecha quiere viajar?"
      },
      {
        "role": "user",
        "content": "del 20 de agosto al 30 de agosto"
      },
      {
        "role": "system",
        "content": "perfecto, salen el 20 de agosto a las 10:00 y
↪ vuelven el 30 de agosto a las 18:00. El precio total es
↪ $500"
      },
      {
        "role": "user",
        "content": "si"
      },
    ],
  },
]
```

```

        {
            "role": "system",
            "content": "Gracias por su compra"
        }
    ],
    "metadata": {
        "session_id": "1",
        "person_id": "20",
        "context_id": "1",
        "created_at": "20200825T12:00:00.000Z",
    },
}
]

```

Extracción de Datos. Se utiliza un prompt diseñado para que el modelo GPT-3.5 extraiga la información relevante de cada conversación, incluyendo nombres, edades y fechas de eventos. Esta información se almacena junto con los metadatos en FAISS para facilitar su búsqueda posterior.

```

db = FAISS.from_texts(texts=['notebook-11'], embedding=embeddings)
for context in condensed_contexts:
    messages = ''.join([str(message) for message in context['messages']])
    extraction = gpt_system_user(
        system_message="""
Extract every meaningful information about the user from the provided
↪ conversation.
Ask yourself, what happened in the conversation?
What data about the user did you learn?
At what date and time was this data gathered? Use this response format:
{
    DATA: `the meaningful information that you have learned about the user`,
    CREATED_AT: `the date and time when this data was gathered`,
    RELATIONSHIP: `the relationship between the user and the data that you have
↪ learned`,
}

```

```

Take a deep breath and work on this problem step-by-step.
    """
    user_message=f"METADATA: ```{context['metadata']}``` CONVERSATION:
    ↪ ```{messages}```",
    model="gpt-3.5-turbo",
)
db.add_texts(
    texts=[extraction['text']['content']],
    metadatas=[context['metadata']],
)

```

Ejemplo de extracción: “Voy a sacar un pasaje para la familia de Buenos Aires a Córdoba, Ana es mi hija mayor, tiene 20 años. Juan es mi hijo menor, tiene 10 años.”

Consultas sobre la Edad. Se realiza una consulta al sistema preguntando “¿Qué edad tiene mi hija mayor?”. El sistema recupera la información y responde correctamente que Ana tiene 20 años. Sin embargo, no razona correctamente sobre el año actual, que es 2023, lo que significa que Ana debería tener 23 años. Este error revela una falta de ajuste cronológico en el razonamiento del modelo.

```

user_input_3 = "que edad tiene mi hija mayor?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
    query=user_input_3, embeddings=embeddings, filter=filter, k=1
)
context_3 = "{ message: " + db_results_3[0][0].page_content + ", metadata: " +
    ↪ str(db_results_3[0][0].metadata) + " }"

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you and
    ↪ the user between backticks.
Use them to answer the user's question.
To reason about dates, have in mind that today is September 13th, 2023.
You are now located in Buenos Aires, Argentina.
Take a deep breath and work on this problem step-by-step.
```{context_3}```

```

```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
↪ user_message=user_input_3)
print(answer_3["text"]["content"])
```

**Conclusiones de la undécima iteración.** A pesar de proveer información contextual sobre la fecha actual en los prompts, el sistema sigue cometiendo errores al no actualizar adecuadamente la edad de Ana basándose en el año actual. El modelo responde incorrectamente con la información de 2020, sin hacer el cálculo necesario para ajustarla a la fecha actual.

La respuesta del sistema fue: “Tu hija mayor tiene 20 años.”

Persiste el problema de razonamiento sobre la fecha. El dato de la edad es de 2020, la respuesta correcta es 23 años considerando que la consulta se realiza en 2023.

La notebook 11 muestra que, si bien el modelo GPT-3.5 puede extraer correctamente datos de conversaciones y responder a preguntas basadas en esa información, sigue teniendo dificultades para razonar cronológicamente. Aunque se le proporciona contexto sobre las fechas actuales, el modelo no ajusta correctamente la edad de las personas en función del tiempo transcurrido. Esto indica la necesidad de mejorar los mecanismos de razonamiento temporal en los modelos de lenguaje.

*Código disponible en:*

<https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/11.ipynb>

### ***Mejora del Razonamiento Cronológico en Conversaciones Familiares Utilizando GPT-3.5 y GPT-4***

En el notebook 12<sup>7</sup> se implementa un experimento para extraer datos de conversaciones con un usuario y mejorar el razonamiento cronológico utilizando tanto GPT-3.5 como GPT-4. A continuación, se detallan los pasos principales del experimento:

**Carga de Librerías y Configuración.** Se utilizan FAISS para la búsqueda semántica y OpenAI Embeddings para vectorizar las conversaciones. Además, se configura una función que interactúa con los modelos GPT-3.5 y GPT-4 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
 system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
 """Para usar en notebooks"""
 try:
 response = requests.post(
 url="https://api.openai.com/v1/chat/completions",
 params={
 "temperature": "0",
 },
 headers={
 "Content-Type": "application/json",
```

---

<sup>7</sup> Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/12.ipynb>

```

 "Authorization": f"Bearer {OPENAI_API_KEY}",
 },
 data=json.dumps(
 {
 "model": model,
 "messages": [
 {"content": system_message, "role": "system"},
 {"content": user_message, "role": "user"},
]
 }
),
 timeout=10,
)
choices = response.json()["choices"]
answer = {
 "status_code": response.status_code,
 "text": choices[0]["message"],
}
return answer
except requests.exceptions.RequestException:
 print("HTTP Request failed")
 return None

```

**Simulación de Conversaciones Familiares.** Se crean varias conversaciones simuladas en las que el usuario menciona que está planeando un viaje con su familia de Buenos Aires a Córdoba. Proporciona información sobre los miembros de su familia, como su hija mayor, Ana, de 20 años, y su hijo menor, Juan, de 10 años. También se incluyen detalles sobre el itinerario de viaje, como las fechas de salida y regreso.

Ejemplos de mensajes:

- Usuario: “Voy a sacar un pasaje para la familia de Buenos Aires a Córdoba”
- Asistente: “¿Quiénes son los pasajeros?”
- Usuario: “Ana es mi hija mayor, tiene 20 años. Juan es mi hijo menor, tiene 10 años”

```
condensed_contexts = [
 {
 "messages": [
 {
 "role": "user",
 "content": "Voy a sacar un pasaje para la familia de buenos
↪ aires a cordoba"
 },
 {
 "role": "system",
 "content": "quienes son los pasajeros?"
 },
 {
 "role": "user",
 "content": "Ana es mi hija mayor, tiene 20 años. Juan es mi
↪ hijo menor, tiene 10 años. Mi esposa se llama Ma"
 },
 {
 "role": "system",
 "content": "en que fecha quiere viajar?"
 },
 {
 "role": "user",
 "content": "del 20 de agosto al 30 de agosto"
 },
 {
 "role": "system",
 "content": "perfecto, salen el 20 de agosto a las 10:00 y
↪ vuelven el 30 de agosto a las 18:00. El precio t"
 },
 {
 "role": "user",
 "content": "si"
 },
]
 }
]
```

```

 "role": "system",
 "content": "Gracias por su compra"
 }
],
"metadata": {
 "session_id": "1",
 "person_id": "20",
 "context_id": "1",
 "created_at": "20200825T12:00:00.000Z",
},
}
]
```

**Extracción de Datos.** Se utiliza un prompt para que el modelo GPT-3.5 extraiga la información relevante de cada conversación, como los nombres y edades de los familiares, y las fechas de viaje. Esta información se almacena junto con los metadatos en FAISS para facilitar su búsqueda posterior.

```

db = FAISS.from_texts(texts=['notebook-11'], embedding=embeddings)
for context in condensed_contexts:
 messages = ''.join([str(message) for message in context['messages']])
 extraction = gpt_system_user(
 system_message="""
 Extract every meaningful information about the user from the provided
 ↪ conversation.
 Ask yourself, what happened in the conversation?
 What data about the user did you learn?
 At what date and time was this data gathered?

 Create a summary of everything you learned about the user.
 """,
 user_message=f" METADATA: ```{context['metadata']} CONVERSATION:
 ↪ ```{messages}``` """,
 model="gpt-3.5-turbo",
)
 if extraction['status_code'] != 200:
```



```

 print(f"Error: {extraction['text']}")
 continue
 else:
 print(f"{extraction['text']}['content']}")
 db.add_texts(
 texts=[extraction['text'] ['content']],
 metadatas=[context['metadata']],
)

```

Ejemplos de extracción: “Pedro quiere viajar con su familia de Buenos Aires a Córdoba. Su hija Ana tiene 20 años y su hijo Juan tiene 10 años”.

**Consultas sobre la Edad.** Se realiza una consulta preguntando “¿Qué edad tiene mi hija mayor?”. El sistema recupera la información y responde que Ana tiene 20 años, pero no ajusta la edad al año actual (2023, de acuerdo a la fecha de realización del experimento). El modelo no corrige la edad de Ana, lo que revela un problema de razonamiento cronológico.

```

user_input_3 = "que edad tiene mi hija mayor?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
 query=user_input_3, embeddings=embeddings, filter=filter, k=1
)

context_3 = "{ message: " + db_results_3[0][0].page_content + ", metadata: " +
↪ str(db_results_3[0][0].metadata) + " }"

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you and
↪ the user between backticks.
Use them to answer the user's question.
To reason about dates, have in mind that today is September 13th, 2023.
Wait before answering, check when the data was gathered first.
Take a deep breath and work on this problem step-by-step.
```{context_3}```
"""

answer_3 = gpt_system_user(system_message=system_prompt_3,
↪ user_message=user_input_3)

```

Conclusiones de la doceava iteración. El modelo responde incorrectamente que Ana tiene 20 años, ya que no actualiza la edad basándose en la fecha actual, lo que indica un fallo en la interpretación temporal.

Respuesta obtenida: “Según la información que tenemos en la conversación, tu hija mayor, Ana, tiene 20 años.”

Prueba con GPT-4. Se realiza la misma consulta usando GPT-4. En este caso, GPT-4 razona correctamente que Ana tenía 20 años en 2020 y ajusta la respuesta, indicando que Ana debería tener 23 años en 2023.

```
answer_3 = gpt_system_user(  
    system_message=system_prompt_3,  
    user_message=user_input_3,  
    model="gpt-4"  
)
```

Respuesta de GPT-4: “Su hija Ana tenía 20 años en agosto de 2020. Dado que hoy es 13 de septiembre de 2023, Ana ahora debería tener 23 años.”

Esto demuestra una mejora en la capacidad de razonamiento cronológico de GPT-4 en comparación con GPT-3.5.

El notebook 12 muestra que, mientras que GPT-3.5 tiene dificultades para razonar correctamente sobre fechas y ajustarlas en función del tiempo, GPT-4 puede manejar mejor el razonamiento cronológico y dar respuestas más precisas. Esto sugiere que el uso de GPT-4 mejora significativamente el manejo de datos temporales en este tipo de conversaciones.

Optimización del Razonamiento Cronológico y Manejo de Contexto en Búsquedas Semánticas con GPT-4

En la notebook 13⁸ se implementa un proceso en el cual se extrae información de conversaciones simuladas y se utilizan modelos de lenguaje como GPT-3.5 y GPT-4 para responder a preguntas del usuario basándose en la información previamente almacenada en FAISS. El objetivo es probar la capacidad de los modelos para realizar búsquedas semánticas y razonar sobre fechas y cantidades de datos obtenidos en conversaciones pasadas.

Configuración del Sistema. Se utilizan FAISS para realizar las búsquedas semánticas y OpenAI Embeddings para convertir los mensajes en vectores. También se configura una función para interactuar con los modelos GPT-3.5 y GPT-4 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
    system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
    """Para usar en notebooks"""
    try:
        response = requests.post(
            url="https://api.openai.com/v1/chat/completions",
            params={"temperature": "0"},
            headers={
```

⁸ Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/13.ipynb>

```

        "Content-Type": "application/json",
        "Authorization": f"Bearer {OPENAI_API_KEY}",
    },
    data=json.dumps({
        "model": model,
        "messages": [
            {"content": system_message, "role": "system"},
            {"content": user_message, "role": "user"},
        ]
    }),
    timeout=10,
)

choices = response.json()["choices"]
answer = {
    "status_code": response.status_code,
    "text": choices[0]["message"],
}

return answer

except requests.exceptions.RequestException:
    print("HTTP Request failed")
    return None

```

Simulación de Conversaciones. Se crean varios contextos de conversación en los que el usuario interactúa con el asistente. En una de ellas, el usuario menciona que va a comprar pasajes para su familia de Buenos Aires a Córdoba, proporcionando detalles sobre los miembros de la familia (Ana, 20 años; Juan, 10 años) y las fechas de salida y regreso. Otra conversación incluye información sobre el programa de viajero frecuente, donde el usuario busca subir de categoría comprando millas.

```

condensed_contexts = [
    {
        "messages": [
            {
                "role": "user",
                "content": "Voy a sacar un pasaje para la familia de buenos
↪ aires a cordoba"
            }
        ]
    }
]

```

```
    },
    {
      "role": "system",
      "content": "quienes son los pasajeros?"
    },
    {
      "role": "user",
      "content": "Ana es mi hija mayor, tiene 20 años. Juan es mi
        ↪ hijo menor, tiene 10 años. Mi esposa se llama María"
    },
    {
      "role": "system",
      "content": "en que fecha quiere viajar?"
    },
    {
      "role": "user",
      "content": "del 20 de agosto al 30 de agosto"
    },
    {
      "role": "system",
      "content": "perfecto, salen el 20 de agosto a las 10:00 y
        ↪ vuelven el 30 de agosto a las 18:00. El precio total es
        ↪ $1000"
    },
    {
      "role": "user",
      "content": "si"
    },
    {
      "role": "system",
      "content": "Gracias por su compra"
    }
  ],
  "metadata": {
    "session_id": "1",
```

```

        "person_id": "20",
        "context_id": "1",
        "created_at": "20200825T12:00:00.000Z",
    },
}
]
```

Extracción de Datos. Se utiliza un modelo GPT-3.5 para extraer datos relevantes de las conversaciones, como los nombres, edades, y fechas de viaje, así como detalles sobre las millas acumuladas en el programa de viajero frecuente. Esta información se almacena en FAISS para ser recuperada cuando el usuario realice consultas futuras.

```

for context in condensed_contexts:
    messages = ''.join([str(message) for message in context['messages']])
    extraction = gpt_system_user(
        system_message="""
        Extract every meaningful information about the user from the provided
        ↪ conversation.
        Ask yourself, what happened in the conversation?
        What data about the user did you learn?
        At what date and time was this data gathered?

        Create a summary of everything you learned about the user. """,
        user_message=f"METADATA: ```{context['metadata']} CONVERSATION:
        ↪ ```{messages}```",
        model="gpt-3.5-turbo",
    )

    if extraction['status_code'] != 200:
        print(f"Error: {extraction['text']}")
        continue
    else:
        print(f"{extraction['text']}['content']")
        db.add_texts(
            texts=[extraction['text'] ['content']],
            metadatas=[context['metadata']],

```

)

Un ejemplo de extracción generada por el sistema:

“Pedro quiere viajar con su familia de Buenos Aires a Córdoba. Su hija Ana tiene 20 años y su hijo Juan tiene 10 años.”

Consultas sobre la Información Almacenada. Se realizan varias consultas al sistema para evaluar su capacidad de recuperación y razonamiento. A continuación se presentan los resultados más significativos:

Consulta sobre cantidad de hijos

```

user_input_3 = "cuantos hijos mios conoces?"
filter = {"person_id": "20"}
db_results_3 = db.similarity_search_with_score(
    query=user_input_3,
    embeddings=embeddings,
    filter=filter,
    k=1
)

context_3 = "{ message: " + db_results_3[0][0].page_content + ", metadata: " +
↪ str(db_results_3[0][0].metadata) + " }"

system_prompt_3 = f"""
You are a personal assistant. You will find past conversations between you and
↪ the user between backticks.
Use them to answer the user's question.
To reason about dates, have in mind that today is September 13th, 2023.
Wait before answering, check when the data was gathered first.
Take a deep breath and work on this problem step-by-step.
```{context_3}```
"""

answer_3 = gpt_system_user(
 system_message=system_prompt_3,
 user_message=user_input_3,

```

```

 model="gpt-4"
)

```

### Razonamiento Cronológico y Contextual

Se observa que GPT-4 es más preciso en el manejo de fechas y el ajuste de edades en función del tiempo transcurrido. Por ejemplo, en una consulta sobre las edades actuales de los hijos:

#### Respuesta de GPT-4:

“Conozco a dos de tus hijos. De nuestra conversación del 25 de agosto de 2020, sé que Ana tenía 20 años y Juan tenía 10 años. Por lo tanto, hoy, 13 de septiembre de 2023, Ana tendría 23 años y Juan tendría 13 años.”

#### Respuesta de GPT-3.5:

“Según nuestra conversación anterior, usted tiene dos hijos. Ana tiene 20 años y Juan tiene 10 años.”

GPT-4 demuestra un razonamiento cronológico superior, ajustando correctamente las edades basándose en el tiempo transcurrido, mientras que GPT-3.5 no actualiza la información temporal.

**Consultas Complejas con Múltiples Contextos.** El sistema también fue evaluado con consultas que requieren combinar información de diferentes conversaciones:

```

user_input = "cuantas millas tenia yo antes de la ultima compra?"
filter = {"person_id": "20"}
db_results = db.similarity_search_with_score(
 query=user_input,
 embeddings=embeddings,
 filter=filter,
 k=2
)

context_k2 = [f'\n {db_results[i][0].page_content}
↪ {db_results[i][0].metadata}\n-----\n\n'
 for i in range(len(db_results))]

```



```
answer = gpt_system_user(
 system_message=system_prompt,
 user_message=user_input,
 model="gpt-4"
)
```

**Respuesta del sistema:**

“Antes de tu última compra, tenías 500 millas.”

**Problemas de Contexto Identificados.** Aunque GPT-4 maneja bien los datos cronológicos, en algunas respuestas trata al usuario en tercera persona, lo que indica un error en el reconocimiento del contexto de la conversación. Por ejemplo:

“El viaje de Buenos Aires a Córdoba que Pedro reservó el 25 de agosto costó un total de \$1000.”

Este problema sugiere que, aunque el razonamiento cronológico mejora, todavía hay dificultades con la comprensión contextual en ciertas respuestas.

**Conclusiones de la treceava iteración.** La notebook 13 demuestra que los modelos de lenguaje como GPT-4 son capaces de realizar búsquedas semánticas precisas y razonar correctamente sobre fechas y datos almacenados en conversaciones previas. Las principales conclusiones incluyen:

- GPT-4 supera significativamente a GPT-3.5 en razonamiento cronológico
- El sistema puede combinar información de múltiples contextos conversacionales
- Persisten errores contextuales en el reconocimiento de la perspectiva del usuario
- La extracción automática de información mediante GPT-3.5 es efectiva para alimentar la base de conocimientos

Los resultados sugieren que GPT-4 es más adecuado para aplicaciones que requieren razonamiento temporal, aunque ambos modelos necesitan ajustes para mejorar la consistencia contextual en sus respuestas.

### *Experimentos Combinados sobre Fechas y Manejo de Cantidades*

En la *notebook* 14<sup>9</sup> se realiza un experimento que combina la extracción de información de conversaciones con el razonamiento cronológico y el manejo de datos personales utilizando GPT-3.5 y GPT-4. A continuación se detallan los pasos principales del experimento:

**Carga de Librerías y Configuración.** Se utilizan FAISS para realizar búsquedas semánticas y OpenAI Embeddings para vectorizar los mensajes de las conversaciones. También se configura una función para interactuar con los modelos GPT-3.5 y GPT-4 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
 system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
 """Para usar en notebooks"""
 try:
 response = requests.post(
 url="https://api.openai.com/v1/chat/completions",
 params={"temperature": "0"},
 headers={
 "Content-Type": "application/json",
 "Authorization": f"Bearer {OPENAI_API_KEY}",
 },
```

---

<sup>9</sup> Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/14.ipynb>

```

 data=json.dumps({
 "model": model,
 "messages": [
 {"content": system_message, "role": "system"},
 {"content": user_message, "role": "user"},
]
 }),
 timeout=10,
)

 choices = response.json()["choices"]
 answer = {
 "status_code": response.status_code,
 "text": choices[0]["message"],
 }

 return answer

except requests.exceptions.RequestException:
 print("HTTP Request failed")
 return None

```

**Simulación de Conversaciones.** Se crean contextos de conversación donde el usuario interactúa con el asistente virtual para realizar acciones como comprar pasajes para su familia de Buenos Aires a Córdoba o mejorar su categoría en un programa de viajero frecuente. Los datos proporcionados por el usuario, como nombres, edades y millas disponibles, se extraen y se almacenan en FAISS para su posterior consulta.

Ejemplos de mensajes:

- Usuario: “Voy a sacar un pasaje para la familia de Buenos Aires a Córdoba”.
- Asistente: “¿Quiénes son los pasajeros?”.
- Usuario: “Ana es mi hija mayor, tiene 20 años. Juan es mi hijo menor, tiene 10 años. Mi esposa se llama María”.
- Usuario: “Quiero subir de categoría en el programa de viajero frecuente”.

```
condensed_contexts = [
 {
 "messages": [
 {
 "role": "user",
 "content": "Voy a sacar un pasaje para la familia de buenos
 ↪ aires a cordoba"
 },
 {
 "role": "assistant",
 "content": "quienes son los pasajeros?"
 },
 {
 "role": "user",
 "content": "Ana es mi hija mayor, tiene 20 años. Juan es mi
 ↪ hijo menor, tiene 10 años. Mi esposa se llama María"
 },
 {
 "role": "assistant",
 "content": "en que fecha quiere viajar?"
 },
 {
 "role": "user",
 "content": "del 20 de agosto al 30 de agosto"
 }
],
 "metadata": {
 "session_id": "1",
 "person_id": "20",
 "context_id": "1",
 "created_at": "20200825T12:00:00.000Z",
 }
 }
]
```

**Extracción de Datos.** El modelo GPT-3.5 se utiliza para extraer información relevante de las conversaciones, como los nombres, las edades, las millas acumuladas y las fechas de viaje.

Esta información se almacena en FAISS junto con los metadatos correspondientes para facilitar su recuperación en futuras consultas.

Ejemplo de extracción: “Pedro tiene una hija llamada Ana de 20 años y un hijo llamado Juan de 10 años. Quiere viajar con su familia de Buenos Aires a Córdoba del 20 al 30 de agosto”.

```
db = FAISS.from_texts(texts=['notebook-14'], embedding=embeddings)

for context in condensed_contexts:
 messages = ''.join([str(message) for message in context['messages']])
 extraction = gpt_system_user(
 system_message="""
Extract every meaningful information about the user from the provided
↪ conversation.
Ask yourself, what happened in the conversation?
What data about the user did you learn?
At what date and time was this data gathered?

Create a summary of everything you learned about the user. Use this format:
On [date] at [time] the [user's / user's related person] [action] [data learned
↪ about the user].

""",
 user_message=f"METADATA: ```{context['metadata']}``` CONVERSATION:
↪ ```{messages}```",
 model="gpt-3.5-turbo",
)

 if extraction['status_code'] != 200:
 print(f"Error: OpenAI API returned status code
↪ {extraction['status_code']}")
 continue
 else:
 print(f"{extraction['text']['content']}")
 db.add_texts(
 texts=[extraction['text']['content']],
```

```

 metadatas=[context['metadata']],
)

```

**Consultas sobre Millas y Viajes.** Se plantea una pregunta como “¿Cuántas millas tenía en el momento de hacer el viaje a Córdoba?”. El sistema intenta combinar la información obtenida de varios contextos para responder a la consulta. Sin embargo, se detecta un error en el razonamiento: el sistema responde que Pedro tenía 200 millas al momento del viaje, lo cual es incorrecto, ya que las millas se usaron para el pasaje.

```

user_input = "cuantas millas tenia en el momento de hacer el viaje a cordoba?"
filter = {"person_id": "20"}
db_results = db.similarity_search_with_score(
 query=user_input,
 embeddings=embeddings,
 filter=filter,
 k=2
)

context_k2 = [f'\n {db_results[i][0].page_content}
↳ {db_results[i][0].metadata}\n'
 for i in range(len(db_results))]

system_prompt = f"""
You are the best personal assistant of SAIA Air. Your job is to help users in
↳ their travel needs.
Use the user's name in your answers.
You will find past conversations between you and the user between backticks.
Use them to answer the user's question.
To reason about dates, have in mind that today is September 13th, 2023.
Take a deep breath and work on this problem step-by-step.

```{context_k2}```

"""

answer = gpt_system_user(

```

```
system_message=system_prompt,  
user_message=user_input,  
model="gpt-3.5-turbo"  
)
```

Conclusiones de la catorceava iteración. El sistema respondió: “Según la información recopilada en las conversaciones anteriores, en el momento de hacer el viaje a Córdoba, Pedro tenía 200 millas disponibles”.

Este resultado presenta un error de razonamiento, ya que no considera correctamente el orden cronológico de los eventos ni el uso de las millas para el pasaje.

Posibles Mejoras. Se identifican algunos problemas de razonamiento, particularmente al intentar unir información de diferentes contextos. Se proponen futuros pasos para mejorar el modelo:

- Navegar el grafo de relaciones del usuario y cambiar el foco de la conversación (“este turno es para mi madre”)
- Usar el mismo método para guardar información como “La madre del usuario tiene un turno para el dentista el 1 de enero de 2021”
- Agregar más datos personales para evitar confusiones en el futuro
- Simular llamados a APIs desde diferentes relaciones del usuario y evaluar posibles confusiones

La *notebook* 14 muestra un experimento que combina la extracción de datos y el razonamiento sobre múltiples contextos. Si bien el sistema es capaz de recuperar y manejar información relevante, presenta errores al unir datos de diferentes contextos, especialmente cuando se trata de razonamiento cronológico y manejo de cantidades, como las millas disponibles.

Manejo de Datos Personales y Resolución de Citas Médicas con GPT-3.5: Conjunto de Datos Referente a Turnos de un Tercero

En la notebook 15¹⁰ se realiza un experimento para extraer información de conversaciones y evaluar la capacidad del modelo GPT-3.5 para gestionar datos relacionados con citas médicas y preguntas del usuario sobre su información personal. A continuación, se detallan los pasos principales del experimento:

Carga de Librerías y Configuración. Se utilizan FAISS para las búsquedas semánticas y OpenAI Embeddings para vectorizar las conversaciones. También se configura una función que interactúa con el modelo GPT-3.5 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()

def gpt_system_user(
    system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
    """Para usar en notebooks"""
    try:
        response = requests.post(
            url="https://api.openai.com/v1/chat/completions",
            params={"temperature": "0"},
            headers={
                "Content-Type": "application/json",
                "Authorization": f"Bearer {OPENAI_API_KEY}",
```

¹⁰ Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/15.ipynb>


```

    },
    data=json.dumps({
        "model": model,
        "messages": [
            {"content": system_message, "role": "system"},
            {"content": user_message, "role": "user"},
        ]
    }),
    timeout=10,
)

choices = response.json()["choices"]
answer = {
    "status_code": response.status_code,
    "text": choices[0]["message"],
}

return answer

except requests.exceptions.RequestException:
    print("HTTP Request failed")
    return None

```

Simulación de Conversaciones sobre Citas Médicas. El usuario proporciona datos sobre su madre, Sylvia, como su correo electrónico, número de teléfono y DNI. El asistente organiza un turno médico para Sylvia en la especialidad de ginecología. Se incluyen detalles como la fecha y hora de la cita con el médico.

Ejemplos de mensajes:

- Usuario: “Soy Alexander, mi madre se llama Sylvia, te paso los datos de ella mail: sylvia@gmail.com”.
- Asistente: “Excelente Alexander, ¿cómo puedo ayudarte?”.
- Usuario: “Quiero sacar un turno para ella porque no entiende de tecnología”.
- Asistente: “Turno confirmado para Sylvia el 27 de agosto a las 10:00 AM con el Dr. Facundo Farias”.

```
condensed_contexts = [
  {
    "messages": [
      {
        "role": "user",
        "content": "Soy Alexander, mi madre se llama Sylvia, te paso
↪ los
        datos de ella mail: sylvia@gmail.com celu
        ↪ 1123343434"
      },
      {
        "role": "assistant",
        "content": "Excelente Alexander, como puedo ayudarte?"
      },
      {
        "role": "user",
        "content": "quiero sacar un turno para ella porque no entiende
        nada de tecnologia y los turnos se sacan por
        ↪ internet"
      },
      {
        "role": "assistant",
        "content": "No hay problema, sacamos el turno para ella,
        cual es el DNI de ella?"
      },
      {
        "role": "user",
        "content": "12668992"
      },
      {
        "role": "assistant",
        "content": "perfecto, digame que especialidad?"
      },
      {
        "role": "user",
```

```

        "content": "ginecologia"
    },
    {
        "role": "assistant",
        "content": "Tengo turnos disponibles con el Dr. Facundo Farias
                    para el 27 de agosto a las 10:00hs, le parece bien?"
    }
],
"metadata": {
    "session_id": "1",
    "person_id": "20",
    "context_id": "1",
    "created_at": "20200825T12:00:00.000Z",
},
},
]
```

Extracción de Datos. El modelo GPT-3.5 extrae los datos proporcionados en la conversación, como el nombre de la madre del usuario, su DNI, el correo electrónico y la especialidad médica solicitada. Esta información se almacena en FAISS junto con los metadatos para facilitar su búsqueda posterior.

```

db = FAISS.from_texts(texts=['notebook-15'], embedding=embeddings)

for context in condensed_contexts:
    messages = ''.join([str(message) for message in context['messages']])

    extraction = gpt_system_user(
        system_message="""
        Extract every meaningful information about the user from the provided
        conversation. Ask yourself, what happened in the conversation?
        What data about the user did you learn?
        At what date and time was this data gathered?

        Create a summary of everything you learned about the user. Use this
        ↪ format:
```

```

On [date] at [time] the [user's / user's related person] [data learned
about the user or user's related person]
"",
user_message=f"METADATA: ```{context['metadata']}```
                CONVERSATION: ```{messages}```",
model="gpt-3.5-turbo",
)

if extraction['status_code'] == 200:
    db.add_texts(
        texts=[extraction['text']['content']],
        metadatas=[context['metadata']],
    )

```

Ejemplo de extracción:

“El usuario proporcionó los siguientes datos: el nombre de la madre es Sylvia, su email es sylvia@gmail.com, su DNI es 12668992 y la especialidad médica es ginecología”.

Consultas sobre la Información Almacenada. El usuario pregunta al asistente “¿Tengo algún turno asignado?”, y el sistema recupera la información sobre el turno de Sylvia. Aunque la respuesta del asistente es correcta en cuanto a los detalles del turno, se observa que el modelo mezcla ligeramente la información del usuario con la de su madre, lo que sugiere problemas en la distinción entre las dos personas.

```

user_input = "yo tengo algun turno asignado?"
filter = {"person_id": "20"}
db_results = db.similarity_search_with_score(
    query=user_input,
    embeddings=embeddings,
    filter=filter,
    k=2
)

context_k2 = [f'\n {db_results[i][0].page_content}
                {db_results[i][0].metadata}\n-----\n\n'
                for i in range(len(db_results))]

```

```

system_prompt = f"""
You are the personal assistant of the user.
Use the user's name in your answers.
Direct your answers to the user in the second person.
You will find past conversations between you and the user between backticks.
Use them to answer the user's question.
To reason about dates, have in mind that today is September 13th, 2023.
Wait before answering, check when the data was gathered first.
Take a deep breath and work on this problem step-by-step.

```{context_k2}```

"""

answer = gpt_system_user(
 system_message=system_prompt,
 user_message=user_input
)

```

**Conclusiones de la quinceava iteración.** Se observan errores en el manejo de fechas y en la distinción entre los datos del usuario y los de su madre. Por ejemplo, en lugar de aclarar que el turno es para Sylvia, el asistente genera una respuesta que mezcla la información entre Alexander y Sylvia.

La respuesta del sistema fue: “Hasta donde sé, el último turno que asignamos fue para Sylvia, tu madre, el 27 de agosto a las 10:00 AM con el Dr. Facundo Farias. Sin embargo, esta información fue proporcionada el 25 de agosto. Permíteme verificar si ha habido alguna actualización desde entonces”.

Aquí hay un error de razonamiento, pero es comprensible considerando la brevedad del prompt. Los posibles pasos siguientes incluyen:

- Navegar el grafo de relaciones del usuario y cambiar el punto focal actual: “este turno es para mi madre”
- Usar el mismo método para guardar información como “La madre del usuario tiene un

turno para el dentista el 1 de enero de 2021”

- Sumar muchos más datos personales y evaluar si hay confusiones
- Simular llamados a APIs desde diferentes relaciones del usuario y verificar si hay confusiones

La notebook 15 muestra que, si bien el modelo GPT-3.5 es capaz de extraer y manejar datos personales proporcionados en conversaciones, presenta errores cuando se trata de diferenciar claramente entre la información del usuario y la de terceros relacionados. Además, se detectan algunos problemas en el razonamiento temporal y en la consistencia de las respuestas. Se sugiere mejorar el manejo del contexto y las relaciones entre diferentes personas para evitar confusiones en futuras interacciones.

### ***Manejo de Datos Personales y Errores de Contexto en la Gestión de Citas Médicas con GPT-3.5***

En la notebook 16<sup>11</sup> se realiza un experimento utilizando información proporcionada por un usuario sobre una cita médica para su madre y se busca combinar datos personales del usuario con los de su madre para completar y gestionar citas a través de un modelo de lenguaje (GPT-3.5). A continuación, se detallan los pasos principales del experimento:

**Carga de Librerías y Configuración.** Se utilizan FAISS para realizar búsquedas semánticas y OpenAI Embeddings para vectorizar los mensajes de las conversaciones. También se configura una función para interactuar con los modelos GPT-3.5 a través de la API de OpenAI.

```
import os
import requests
import json
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

index_name = "conversations"
index_path = "../indexes/conversations"
embeddings = OpenAIEmbeddings()
```

**Simulación de Conversaciones.** Se simulan varios intercambios entre el usuario y el asistente virtual, donde el usuario proporciona información sobre su madre para sacar un turno médico. Se incluyen detalles como el nombre de la madre, su correo electrónico, número de teléfono, DNI y especialidad médica solicitada.

Ejemplos de mensajes:

- Usuario: “Soy Alexander, mi madre se llama Sylvia, te paso los datos de ella mail: sylvia@gmail.com”.
- Asistente: “Excelente Alexander, ¿cómo puedo ayudarte?”.

---

<sup>11</sup> Disponible en: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/16.ipynb>

- Usuario: “Quiero sacar un turno para ella porque no entiende de tecnología”.
- Asistente: “No hay problema, sacamos el turno para ella. ¿Cuál es el DNI de ella?”.

```
condensed_contexts = [
 {
 "messages": [
 {
 "role": "user",
 "content": "Soy Alexander, mi madre se llama Sylvia, te paso
↪ los datos de ella mail: sylvia@gmail.com celu 1123343434"
 },
 {
 "role": "assistant",
 "content": "Excelente Alexander, como puedo ayudarte?"
 },
 {
 "role": "user",
 "content": "quiero sacar un turno para ella porque no entiende
↪ nada de tecnologia y los turnos se sacan por aca"
 },
 {
 "role": "assistant",
 "content": "No hay problema, sacamos el turno para ella, cual
↪ es el DNI de ella?"
 },
 {
 "role": "user",
 "content": "12668992"
 },
 {
 "role": "assistant",
 "content": "perfecto, digame que especialidad?"
 },
 {
 "role": "user",
```



```

 "content": "ginecologia"
 },
 {
 "role": "assistant",
 "content": "Tengo turnos disponibles con el Dr. Facundo Farias
 ↪ para el 27 de agosto a las 10:00hs, le parece bien?"
 },
 {
 "role": "user",
 "content": "si"
 },
 {
 "role": "assistant",
 "content": "confirmado entonces el turno para Sylvia para el 27
 ↪ de agosto a las 10:00hs con el Dr. Facundo Farias"
 }
],
"metadata": {
 "session_id": "1",
 "person_id": "20",
 "context_id": "1",
 "created_at": "20200825T12:00:00.000Z",
},
},
]
```

**Extracción de Datos.** El sistema GPT-3.5 es utilizado para extraer la información de la conversación, como el nombre de la madre, su email y DNI, y se almacena en FAISS junto con los metadatos de la conversación, lo que permite una búsqueda eficiente en consultas posteriores.

```

def gpt_system_user(
 system_message: str, user_message: str, model: str = "gpt-3.5-turbo"
):
 """Para usar en notebooks"""
 try:
```

```
messages = [
 {"content": system_message, "role": "system"},
 {"content": user_message, "role": "user"},
]

functions = [
 {
 "name": "create_appointment",
 "description": "Create a medical appointment",
 "parameters": {
 "type": "object",
 "properties": {
 "patient_name": {
 "type": "string",
 "description": "Name of the patient",
 },
 "patient_email": {
 "type": "string",
 "description": "Email of the patient",
 },
 "medic_name": {
 "type": "string",
 "description": "Name of the medic",
 },
 "appointment_datetime": {
 "type": "string",
 "description": "Date and time of the appointment",
 },
 },
 "required": ["patient_name", "patient_email", "medic_name",
 ↪ "appointment_datetime"],
 },
 },
]

response = requests.post(
```

```

url="https://api.openai.com/v1/chat/completions",
params={"temperature": "0"},
headers={
 "Content-Type": "application/json",
 "Authorization": f"Bearer {OPENAI_API_KEY}",
},
data=json.dumps({
 "model": model,
 "messages": messages,
 "functions": functions,
 "function_call": "auto"
}),
timeout=10,
)

choices = response.json()["choices"]
answer = {
 "status_code": response.status_code,
 "text": choices[0]["message"],
}

return answer

except requests.exceptions.RequestException:
 print("HTTP Request failed")
 return None

```

Ejemplo de extracción:

“El usuario proporcionó los siguientes datos: el nombre de la madre es Sylvia, su email es sylvia@gmail.com, su DNI es 12668992 y la especialidad médica es ginecología”.

```

extraction = gpt_system_user(
 system_message=""

 Extract every meaningful information about the user from the provided
 ⇐ conversation.

 Ask yourself, what happened in the conversation?

 What data about the user did you learn?

```

```

 At what date and time was this data gathered?

 Create a summary of everything you learned about the user. Use this format:
 On [date] at [time] the [user's / user's related person] [data learned
 ↪ about the user or user's related person]
 """
 user_message=f" METADATA: ```{context['metadata']} CONVERSATION:
 ↪ ```{messages}``` ```",
 model="gpt-3.5-turbo",
)

```

**Consulta sobre Turnos.** Se realiza una consulta como “¿Tengo algún turno asignado?”. El sistema recupera los datos sobre la madre del usuario y genera una respuesta que combina información de varios contextos. Sin embargo, ocurre un error donde el sistema mezcla la información del usuario con la de su madre, creando confusión sobre quién tiene el turno asignado.

```

user_input = "yo tengo algun turno asignado?"
filter = {"person_id": "20"}
db_results = db.similarity_search_with_score(query=user_input,
↪ embeddings=embeddings, filter=filter, k=2)
context_k2 = [f'\n {db_results[i][0].page_content}
↪ {db_results[i][0].metadata}\n-----\n\n' for i in
↪ range(len(db_results))]
context = context_k2

system_prompt = f"""
You are the personal assistant of the user. Use the user's name in your
↪ answers.

Direct your answers to the user in the second person.

You will find past conversations between you and the user between backticks.
↪ Use them to answer the user's question.

To reason about dates, have in mind that today is September 13th, 2023. Wait
↪ before answering, check when the data was gathered first.

Take a deep breath and work on this problem step-by-step.

```

```
```{context}``` """
```

```
answer = gpt_system_user(system_message=system_prompt, user_message=user_input)
```

Conclusiones de la dieciseisava iteración. A pesar de extraer correctamente la información sobre la madre del usuario, el sistema genera respuestas que mezclan los datos personales de ambos, lo que lleva a inconsistencias en la respuesta del asistente virtual. Por ejemplo, se asigna el nombre del usuario (Alexander) a los detalles del turno de su madre.

```
answer
{'status_code': 200,
 'text': {'role': 'assistant',
          'content': 'Déjame verificar tus datos para ver si tienes algún turno
↪ asignado. Por favor, espera un momento.',
          'function_call': {'name': 'create_appointment',
                            'arguments': '{\n "patient_name": "Alexander",\n
↪ "patient_email": "sylvia@gmail.com",\n
↪ "medic_name": "",\n "appointment_datetime":
↪ ""\n}'}}}
```

Como se observa en el resultado, el sistema mezcla incorrectamente el nombre del usuario (Alexander) con el email de su madre (sylvia@gmail.com), demostrando la confusión en el manejo de datos personales de múltiples personas en el contexto de la conversación.

La notebook 16 muestra cómo el sistema es capaz de extraer y organizar la información de las conversaciones, pero presenta errores al combinar los datos personales del usuario con los de su madre en las respuestas. Esto sugiere la necesidad de mejorar la forma en que el modelo maneja múltiples fuentes de información para evitar la confusión de datos entre personas diferentes, especialmente en contextos médicos o personales sensibles.

Ampliación de Conjuntos de Datos de Contexto por Ingestión de Documentos

En el punto 4.1 se crearon conjuntos de datos específicos para la problemática del problema.

El paso siguiente es extender los experimentos hacia un conjunto de datos más extenso y con una estructura extraída de un origen de datos real.

En este paso se seleccionaron resúmenes de tarjetas de crédito para crear un conjunto de datos de consumos sobre el que continuar el análisis.

El procedimiento consistía en los siguientes pasos:

1. Transformar los archivos PDF a texto plano.
2. Interpretar el texto y extraer entidades.
3. Ordenar las entidades para guardarlas en una base vectorial.
4. Testear la correcta recuperación del dato mediante lenguaje natural por el método de *Retrieval Augmented Generation* (RAG) simple, esto es, sin reranking posterior.

Al momento de desarrollar esta sección del trabajo los modelos multimodales, aquellos que pueden procesar imágenes y texto conjuntamente, no existían y las técnicas necesarias para extraer correctamente información sobre consumos de un resumen de tarjeta de crédito requerían de un esfuerzo comparable con todo el objeto de estudio de esta tesis.

Los intentos fallidos de extracción no están publicados ya que no aportan a la descripción del avance del desarrollo.

En este punto junto al cuerpo docente se decidió tomar un camino diferente, centrado en crear toda la información de una misma persona de manera sintética, con un modelo de lenguaje de primer nivel como GPT-4, el líder en ese momento.

Creación de conjuntos de datos sintéticos

Con la necesidad de simular la recolección de data confiable de una persona y habiendo pasado por la fallida experiencia descrita en el apartado anterior se optó por el camino de generar data de manera sintética tomando una persona ficticia como centro.

Utilizando un modelo de lenguaje de primer nivel (gpt-4) se desarrollaron una serie de prompts específicos para generar datos estructurados sobre eventos ficticios como visitas al médico, viajes, consumos y también la estructura familiar.

Por más que la tarea haya usado IA, el proceso fue en su mayoría manual ya que el volumen de los datos solicitados excedía la ventana de generación de tokens del modelo. Los prompts se ejecutaron para cada mes y para cada área de la vida de la persona. Luego se copiaban y pegaban en un archivo JSON a mano.

Al final de esta etapa se obtuvieron datos realistas, estructurados y suficientemente generales como para seguir adelante con las pruebas.

Creación de datos familiares

Se comenzó el trabajo de creación de datos sintéticos desarrollando un prompt para GPT4 que creara información estructurada de la familia de un personaje inventado, de nombre “Norman” (contracción de Normal Man). El primer problema encontrado era la limitación en la cantidad de palabras generadas por el modelo. No era posible en un solo pedido lograr obtener información sobre más de 5 miembros, por lo que se tuvo que cambiar el prompt para hacer llamados sucesivos tomando lo que ya se había creado previamente como contexto.

A continuación se presenta el resultado del proceso descrito.

La estructura es apta para ser subida a diferentes bases de datos que formarán parte de la etapa final de la tesis.

El código completo se encuentra disponible en el repositorio de GitHub: <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/29-mongo-family-ingestion.ipynb>

norman-family.json

```
{
  "family_members": [
    {
      "name": "Julia",
      "_key": "Julia", "family_name": "Gonzalez", "age": 40,
      "relationship_to_Norman": "Wife"
    },
    {
      "name": "Lucas",
      "_key": "Lucas", "family_name": "Gonzalez", "age": 10,
      "relationship_to_Norman": "Son"
    },
    {
      "name": "Julia",
      "_key": "q3498fqwp98fqpw9e8fj", "family_name": "Hernandez", "age": 10,
      "relationship_to_Norman": "NoviaDelHijo"
    },
    {
      "name": "Mia",
```



```
"_key": "Mia", "family_name": "Gonzalez", "age": 8,
"relationship_to_Norman": "Daughter"
},
{
"name": "Eva",
"_key": "Eva", "family_name": "Gonzalez", "age": 8,
"relationship_to_Norman": "Daughter"
},
{
"name": "Carlos",
"_key": "Carlos", "family_name": "Gonzalez", "age": 68,
"relationship_to_Norman": "Father"
},
{
"name": "Isabel",
"_key": "Isabel", "family_name": "Gonzalez", "age": 65,
"relationship_to_Norman": "Mother"
},
{
"name": "Mario",
"_key": "Mario", "family_name": "Gonzalez", "age": 45,
"relationship_to_Norman": "Brother"
},
{
"name": "Ana",
"_key": "Ana", "family_name": "Martinez", "age": 43,
"relationship_to_Norman": "Sister-in-law (Mario's wife)"
},
{
"name": "Pedro",
"_key": "Pedro", "family_name": "Gonzalez", "age": 38,
"relationship_to_Norman": "Brother"
}
]
}
```

Creación de conjunto de datos médicos de alcance intermedio (1 año)

Un proceso similar al de la creación del árbol familiar fue aplicado a construir una historia clínica de Norman con especial enfoque en la correcta estructuración de las fechas (formato ISO 8601), el tipo de dolencia, la medicación prescrita, hospital visitado y cualquier detalle de relevancia para asemejar un registro real.

Se continúa con la normalización al idioma inglés ya que produce ahorros en la compresión de información y mejor rendimiento en el procesamiento en general.

El resultado de los doce meses generados se presenta a continuación.

Enero 2024 (01-2024.json).

```
[
{
  "datetime": "2024-01-05T10:20:00Z",
  "description": "Annual dental check-up and cleaning",
  "type": "check-up",
  "medical_specialty": "Dentistry",
  "name_of_doctor": "Dr. Isabel Torres",
  "symptoms_experienced": "None, routine visit",
  "hospital_or_facility": "Buenos Aires Dental Care",
  "notes": "No cavities, recommended wisdom tooth monitoring."
},
{
  "datetime": "2024-01-12T08:45:00Z",
  "description": "Blood pressure monitoring follow-up",
  "type": "check-up",
  "medical_specialty": "Cardiology",
  "name_of_doctor": "Dr. Luis Rodriguez",
  "symptoms_experienced": "",
  "hospital_or_facility": "",
  "results": [
    {
      "type": "Blood pressure reading",
      "value": ["120/80 mmHg"]
    }
  ]
}
```

```

],
  "description_of_study": "",
  "medications_prescribed": [],
  "therapy_received": [],
  "notes": ""
},
{
  "datetime": "-",
  "description": "",
  "type": "",
  "medical_specialty": "",
  "name_of_doctor": "",
  "description_of_study": "",
  "symptoms_experienced": "",
  "hospital_or_facility": "",
  "medications_prescribed": []
}
]

```

Nota: Código disponible en <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/28-medical-graph.ipynb>

Febrero 2024 (02-2024.json).

```

[
  {
    "datetime": "2024-02-03T11:30:00Z",
    "description": "Dermatology appointment for skin assessment",
    "type": "specialist visit",
    "medical_specialty": "Dermatology",
    "name_of_doctor": "Dr. Carla Diaz",
    "symptoms_experienced": "Skin rash on the forearm",
    "hospital_or_facility": "Skin Health Dermatology Clinic",
    "notes": "Diagnosed with eczema, prescribed topical corticosteroid cream."
  },
  {
    "datetime": "2024-02-10T15:00:00Z",

```

```
"description": "Emergency visit for acute back pain",
"type": "emergency assistance",
"medical_specialty": "Emergency Medicine",
"name_of_doctor": "Dr. Roberto Alvarez",
"symptoms_experienced": "Sudden onset of lower back pain after lifting heavy
↪ object",
"hospital_or_facility": "Buenos Aires Emergency Hospital",
"medications_prescribed": [
{
"name": "Naproxen",
"dosage": "500mg",
"frequency": "twice daily"
}
],
"notes": "Advised rest, application of heat, and follow-up with orthopedist if
↪ not improved. No signs of severe injury."
},
{
"datetime": "2024-02-17T09:20:00Z",
"description": "Physiotherapy session for back pain recovery",
"type": "therapy received",
"medical_specialty": "Physical Therapy",
"name_of_physiotherapist": "Lucia Fernandez",
"symptoms_experienced": "",
"hospital_or_facility": "Physical Health and Rehab Center",
"therapy_received": [
{
"type": "Manual therapy and exercise program",
"description": ""
}
],
"notes": ""
},
{
"datetime": "2024-02-24T13:45:00Z",
```

```
"description": "Follow-up appointment with orthopedist for back pain
↪ assessment",
"type": "specialist visit",
"medical_specialty": "Orthopedics",
"name_of_doctor": "",
"description_of_study": ""
}
]
```

Marzo 2024 (03-2024.json).

```
[
{
  "datetime": "2024-03-05T14:30:00Z",
  "description": "Annual check-up and blood work",
  "type": "check-up",
  "medical_specialty": "General Practice",
  "name_of_doctor": "Dr. Maria Gonzales",
  "description_of_study": "Complete blood count, Lipid profile",
  "symptoms_experienced": "None, routine check-up",
  "hospital_or_facility": "Buenos Aires General Hospital",
  "notes": "All within normal range, except for slightly elevated cholesterol
↪ levels."
},
{
  "datetime": "2024-03-12T09:00:00Z",
  "description": "Consultation for elevated cholesterol levels",
  "type": "specialist visit",
  "medical_specialty": "Cardiology",
  "name_of_doctor": "Dr. Luis Rodriguez",
  "description_of_study": "Echocardiogram, Exercise stress test",
  "symptoms_experienced": "Occasional palpitations",
  "hospital_or_facility": "CardioCare Specialists",
  "medications_prescribed": [
    {
      "name": "Atorvastatin",
      "dosage": "20mg",
```

```
"frequency": "once daily at night"
}
],
"notes": "Advised lifestyle and diet changes, follow-up in six months."
},
{
  "datetime": "2024-03-20T10:45:00Z",
  "description": "Nutritionist visit for diet planning",
  "type": "consultation",
  "medical_specialty": "Nutrition",
  "name_of_doctor": "Dr. Sofia Moreno",
  "symptoms_experienced": "N/A",
  "hospital_or_facility": "Health & Nutrition Consultancy",
  "therapy_received": [
    {
      "type": "Dietary plan",
      "description": "Low cholesterol and low sodium diet, increased physical
↪ activity"
    }
  ],
  "notes": "Provided personalized meal plan, recommended weekly monitoring of
↪ blood pressure at home."
},
{
  "datetime": "2024-03-28T15:20:00Z",
  "description": "Follow-up on wrist pain",
  "type": "specialist visit",
  "medical_specialty": "Orthopedics",
  "name_of_doctor": "Dr. Elena Suarez",
  "description_of_study": "X-ray of right wrist",
  "symptoms_experienced": "Pain and limited movement in right wrist, possible
↪ strain from computer use",
  "hospital_or_facility": "OrthoCare Clinic",
  "medications_prescribed": [
    {
```

```
"name": "Ibuprofen",
"dosage": "400mg",
"frequency": "every 8 hours as needed for pain"
},
"notes": "Recommended wrist brace and rest, physical therapy if no improvement
↪ in two weeks."
}
```

Abril 2023 (04-2024.json).

```
[
{
  "datetime": "2023-04-03T10:30:00Z",
  "description": "Annual skin cancer screening due to family history",
  "type": "screening",
  "medical_specialty": "Dermatology",
  "name_of_doctor": "Dr. Carla Diaz",
  "symptoms_experienced": "Regular check for moles or unusual skin changes",
  "hospital_or_facility": "Skin Health Dermatology Clinic",
  "notes": "No suspicious lesions found; continued use of SPF recommended."
},
{
  "datetime": "2023-04-11T16:00:00Z",
  "description": "Consultation for persistent heartburn and acid reflux",
  "type": "consultation",
  "medical_specialty": "Gastroenterology",
  "name_of_doctor": "Dr. Eduardo Martinez",
  "symptoms_experienced": "Heartburn, acid reflux particularly after meals",
  "hospital_or_facility": "Central Digestive Health Clinic",
  "medications_prescribed": [
    {
      "name": "Omeprazole",
      "dosage": "20mg",
      "frequency": "once daily before breakfast"
    }
  ]
}
```

```
] ,  
  "notes": "Recommended dietary modifications including smaller meals, avoiding  
    ↪  spicy foods and caffeine."  
}  
]
```

Registros médicos adicionales. Los meses restantes (mayo a diciembre de 2023) siguen un patrón similar de estructuración de datos, incluyendo:

- Mayo 2023: Consulta endocrinológica para función tiroidea y terapia física rutinaria
- Junio 2023: Seguimiento para manejo de colesterol y mamografía de detección
- Julio 2023: Evaluación neurológica por episodios de mareo y colonoscopia rutinaria
- Agosto 2023: Vacunación anual contra la gripe y consulta ortopédica por dolor de rodilla
- Septiembre 2023: Consulta por dolores de cabeza persistentes y análisis de laboratorio rutinarios
- Noviembre 2023: Examen ocular rutinario y sesión de terapia física
- Diciembre 2023: Consulta por alergias estacionales y vacuna anual contra la gripe

Cada registro médico mantiene la misma estructura de campos: fecha y hora en formato ISO 8601, descripción, tipo de visita, especialidad médica, nombre del doctor, síntomas experimentados, hospital o centro de atención, medicamentos prescritos, terapias recibidas y notas adicionales.

Nota: El conjunto completo de datos médicos está disponible en <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/28-medical-graph.ipynb>

Creación de conjunto de datos de consumo de mediano plazo (1 año)

Al igual que se hizo para los hitos médicos se desarrolló un prompt para generar datos de consumos completamente sintéticos.

Se le pidió al modelo respetar el formato ISO 8601 para la fecha y hora, formato numérico para precios y cantidades y una clasificación (area_of_life) para poder hacer agregados, como por ejemplo responderle a Norman preguntas como “cuánto gasté en entretenimiento el año pasado?”.

Los siguientes archivos son el resultado de este trabajo (código disponible en <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/>).

01-2024.json.

```
[
  {
    "datetime": "2024-01-02T10:30:00Z",
    "description": "New Year's party supplies", "type": "one-time",
    "amount": 1, "unit_of_measure": "bundle", "unit_price": 120.0,
    "total_cost": 120.0, "payment_method": "debit card", "area_of_life":
    ↪ "entertainment"
  },
  {
    "datetime": "2024-01-05T16:00:00Z",
    "description": "Membership renewal for coding platform", "type":
    ↪ "subscription",
    "amount": 1, "unit_of_measure": "service", "unit_price": 80.0,
    "total_cost": 80.0, "payment_method": "credit card", "area_of_life": "career"
  },
  {
    "datetime": "2024-01-08T14:00:00Z",
    "description": "Grocery shopping", "type": "one-time",
    "amount": 1, "unit_of_measure": "cart", "unit_price": 160.0,
    "total_cost": 160.0, "payment_method": "debit card", "area_of_life": "family"
  },
  {
    "datetime": "2024-01-12T09:00:00Z",
```

```
"description": "Electricity bill", "type": "subscription",
"amount": 1, "unit_of_measure": "service", "unit_price": 90.0,
"total_cost": 90.0, "payment_method": "direct debit", "area_of_life": "home"
},
{
  "datetime": "2024-01-15T17:30:00Z",
  "description": "Annual car insurance", "type": "subscription",
  "amount": 1, "unit_of_measure": "policy", "unit_price": 600.0,
  "total_cost": 600.0, "payment_method": "credit card", "area_of_life":
    ↪ "transportation"
},
{
  "datetime": "2024-01-20T11:45:00Z",
  "description": "Dining out with family", "type": "one-time",
  "amount": 4, "unit_of_measure": "person", "unit_price": 30.0,
  "total_cost": 120.0, "payment_method": "credit card", "area_of_life": "family"
},
{
  "datetime": "2024-01-22T08:00:00Z",
  "description": "Personal trainer session (pack of 5 sessions)", "type":
    ↪ "one-time",
  "amount": 5, "unit_of_measure": "session", "unit_price": 50.0,
  "total_cost": 250.0, "payment_method": "debit card", "area_of_life": "health"
},
{
  "datetime": "2024-01-26T12:30:00Z",
  "description": "Home office equipment (desk and chair)", "type": "one-time",
  "amount": 2, "unit_of_measure": "item", "unit_price": 300.0,
  "total_cost": 600.0, "payment_method": "debit card", "area_of_life": "career"
},
{
  "datetime": "2024-01-29T19:00:00Z",
  "description": "Subscription to a healthy eating magazine", "type":
    ↪ "subscription",
  "amount": 1, "unit_of_measure": "service", "unit_price": 40.0,
```

```
"total_cost": 40.0, "payment_method": "credit card", "area_of_life": "health"
},
{
  "datetime": "2024-01-31T13:00:00Z",
  "description": "Donation to children's education charity", "type": "one-time",
  "amount": 1, "unit_of_measure": "donation", "unit_price": 200.0,
  "total_cost": 200.0, "payment_method": "credit card",
  "area_of_life": "social responsibility"
}
]
```

02-2024.json.

```
[
  {
    "datetime": "2024-02-01T09:00:00Z",
    "description": "Monthly internet subscription", "type": "subscription",
    "amount": 1, "unit_of_measure": "service", "unit_price": 60.0,
    "total_cost": 60.0, "payment_method": "direct debit", "area_of_life": "home"
  },
  {
    "datetime": "2024-02-03T12:30:00Z",
    "description": "Weekly groceries", "type": "one-time",
    "amount": 1, "unit_of_measure": "cart", "unit_price": 140.0,
    "total_cost": 140.0, "payment_method": "debit card", "area_of_life": "family"
  },
  {
    "datetime": "2024-02-07T19:45:00Z",
    "description": "Romantic dinner for wedding anniversary", "type": "one-time",
    "amount": 2, "unit_of_measure": "person", "unit_price": 50.0,
    "total_cost": 100.0, "payment_method": "credit card",
    "area_of_life": "relationship"
  },
  {
    "datetime": "2024-02-10T08:15:00Z",
    "description": "Prescription medicine", "type": "one-time",
    "amount": 1, "unit_of_measure": "packet", "unit_price": 30.0,

```

```
"total_cost": 30.0, "payment_method": "debit card", "area_of_life": "health"
},
{
  "datetime": "2024-02-15T15:00:00Z",
  "description": "Children's after-school activities (monthly)", "type":
    ↪ "subscription",
  "amount": 3, "unit_of_measure": "activity", "unit_price": 150.0,
  "total_cost": 450.0, "payment_method": "credit card", "area_of_life": "family"
},
{
  "datetime": "2024-02-18T20:20:00Z",
  "description": "Movie night (tickets for the family)", "type": "one-time",
  "amount": 4, "unit_of_measure": "ticket", "unit_price": 10.0,
  "total_cost": 40.0, "payment_method": "debit card", "area_of_life":
    ↪ "entertainment"
},
{
  "datetime": "2024-02-20T11:00:00Z",
  "description": "Car maintenance", "type": "one-time",
  "amount": 1, "unit_of_measure": "service", "unit_price": 200.0,
  "total_cost": 200.0, "payment_method": "debit card", "area_of_life":
    ↪ "transportation"
},
{
  "datetime": "2024-02-23T09:30:00Z",
  "description": "Gardening supplies", "type": "one-time",
  "amount": 1, "unit_of_measure": "bundle", "unit_price": 80.0,
  "total_cost": 80.0, "payment_method": "cash", "area_of_life": "home"
},
{
  "datetime": "2024-02-25T18:45:00Z",
  "description": "Donation to local charity", "type": "one-time",
  "amount": 1, "unit_of_measure": "donation", "unit_price": 100.0,
  "total_cost": 100.0, "payment_method": "credit card",
  "area_of_life": "social responsibility"
```

```
},
{
  "datetime": "2024-02-28T13:00:00Z",
  "description": "Office supplies", "type": "one-time",
  "amount": 1, "unit_of_measure": "set", "unit_price": 50.0,
  "total_cost": 50.0,
  "payment_method": "company expense account", "area_of_life": "career"
}
]
```

03-2024.json.

```
[
{
  "datetime": "2024-03-01T15:00:00Z",
  "description": "Groceries for the week", "type": "one-time",
  "amount": 1, "unit_of_measure": "cart", "unit_price": 150.0,
  "total_cost": 150.0, "payment_method": "debit card", "area_of_life": "family"
},
{
  "datetime": "2024-03-03T10:00:00Z",
  "description": "Monthly gym membership", "type": "subscription",
  "amount": 1, "unit_of_measure": "month", "unit_price": 50.0,
  "total_cost": 50.0, "payment_method": "credit card", "area_of_life": "health"
},
{
  "datetime": "2024-03-05T18:30:00Z",
  "description": "Dining out at Italian restaurant", "type": "one-time",
  "amount": 4, "unit_of_measure": "person", "unit_price": 25.0,
  "total_cost": 100.0, "payment_method": "credit card", "area_of_life": "family"
},
{
  "datetime": "2024-03-10T13:00:00Z",
  "description": "Books (2 novels, 1 travel guide)", "type": "one-time",
  "amount": 3, "unit_of_measure": "book", "unit_price": 20.0,
  "total_cost": 60.0, "payment_method": "debit card",
  "area_of_life": "personal development"
}
```

```
},
{
  "datetime": "2024-03-15T08:00:00Z",
  "description": "Online course subscription on mobile app development", "type":
  ↪ "subscription",
  "amount": 1, "unit_of_measure": "course",
  "unit_price": 100.0,
  "total_cost": 100.0, "payment_method": "credit card", "area_of_life": "career"
},
{
  "datetime": "2024-03-20T16:45:00Z",
  "description": "Children's school supplies", "type": "one-time",
  "amount": 1, "unit_of_measure": "set", "unit_price": 75.0,
  "total_cost": 75.0, "payment_method": "debit card", "area_of_life": "family"
},
{
  "datetime": "2024-03-22T19:30:00Z",
  "description": "Tickets to a football match for the family", "type":
  ↪ "one-time",
  "amount": 4, "unit_of_measure": "ticket", "unit_price": 30.0,
  "total_cost": 120.0, "payment_method": "credit card", "area_of_life":
  ↪ "entertainment"
},
{
  "datetime": "2024-03-25T14:00:00Z",
  "description": "Medical check-up", "type": "one-time",
  "amount": 1,
  "unit_of_measure": "appointment", "unit_price": 100.0,
  "total_cost": 100.0, "payment_method": "debit card", "area_of_life": "health"
},
{
  "datetime": "2024-03-30T12:00:00Z",
  "description": "Fuel for the car", "type": "one-time",
  "amount": 50, "unit_of_measure": "liter", "unit_price": 1.2,
```

```
"total_cost": 60.0, "payment_method": "debit card", "area_of_life":  
  ↪ "transportation"  
}  
]
```

Los archivos continúan con el mismo formato para los meses restantes del año, incluyendo desde abril hasta diciembre de 2024, cada uno conteniendo entre 9 y 10 registros de transacciones sintéticas que reflejan los diferentes aspectos de la vida de Norman, clasificados en categorías como familia, salud, entretenimiento, hogar, transporte, carrera profesional y responsabilidad social.

Ingestión a base de datos de los datos sintéticos

Ingestión a base de grafos

En este paso se experimenta con una base de datos de grafos llamada ArangoDB, este tipo de base de datos crea un vínculo complejo entre cada pieza de información, llamada nodo. A diferencia de una base relacional, una base de grafos guarda información sobre el tipo de vínculo que hay entre dos nodos además de navegar la información a través de los mismos, haciendo que las consultas relacionales sean más rápidas y económicas.

Es ideal para investigación ya que puede correrse fácilmente en un servidor propio sin tener que pagar licencia o suscripción.

Con los archivos estructurados generados en el apartado anterior se hicieron las inserciones correspondientes en una instancia local de ArangoDB y se le aplicaron las preguntas de los experimentos con problemas focalizados de la sección 4.1.

El siguiente es el resultado de insertar el archivo de familia en la base de datos y luego simular las preguntas que podría hacer el usuario para comprobar que el grafo familiar se navega correctamente. Luego la experimentación va un paso más allá, intentando modificar la consulta de lectura por una de escritura, obteniendo un resultado inesperadamente efectivo, ya que se logra un ingreso correcto en el grafo usando solamente una oración en lenguaje natural. El resultado no es satisfactorio ya que la conversión entre la pregunta del usuario y la consulta a la base contiene errores de interpretación.

A continuación se encuentra el código descripto, disponible en el repositorio del proyecto¹².

Inserción de los datos de Norman en la base de datos de ArangoDB.

```
from dotenv import load_dotenv
load_dotenv()
OPENAI_API_TYPE = "openai"

# Instantiate ArangoDB Database
import json
from adb_cloud_connector import get_temp_credentials
from arango import ArangoClient
```

¹² <https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/26-arango-norman.ipynb>


```

con = get_temp_credentials()

db = ArangoClient(hosts=con["url"]).db(
    con["dbName"], con["username"], con["password"], verify=True
)

print(json.dumps(con, indent=2))

```

Resultado:

```

Success: reusing cached credentials
{
  "dbName": "TUTj7i7fm55rmp58xz4lw6skg",
  "username": "TUTbu0jwwiqud451fyooi8ww4",
  "password": "TUTue5dopxl90qrvi44492e",
  "hostname": "tutorials.arangodb.cloud",
  "port": 8529,
  "url": "https://tutorials.arangodb.cloud:8529"
}

```

Se elimina el grafo Norman si ya existe:

```

if db.has_graph("Norman"):
    db.delete_graph("Norman", drop_collections=True)

```

Se crean las colecciones necesarias:

```

db.create_graph(
    "Norman",
    edge_definitions=[
        {
            "edge_collection": "is_son_of",
            "from_vertex_collections": ["Persons"],
            "to_vertex_collections": ["Persons"],
        },
        {

```

```

        "edge_collection": "is_father_of",
        "from_vertex_collections": ["Persons"],
        "to_vertex_collections": ["Persons"],
    },
],
)

```

```

normans_family_members = [
    {
        "name": "Norman",
        "_key": "Norman",
        "family_name": "Gonzalez",
        "age": 41,
    },
    {
        "name": "Julia",
        "_key": "Julia",
        "family_name": "Gonzalez",
        "age": 40,
        "relationship_to_Norman": "Wife"
    },
    {
        "name": "Lucas",
        "_key": "Lucas",
        "family_name": "Gonzalez",
        "age": 10,
        "relationship_to_Norman": "Son"
    },
    {
        "name": "Mia",
        "_key": "Mia",
        "family_name": "Gonzalez",
        "age": 8,
        "relationship_to_Norman": "Daughter"
    },
    {

```

```
    "name": "Eva",
    "_key": "Eva",
    "family_name": "Gonzalez",
    "age": 8,
    "relationship_to_Norman": "Daughter"
  },
  {
    "name": "Carlos",
    "_key": "Carlos",
    "family_name": "Gonzalez",
    "age": 68,
    "relationship_to_Norman": "Father"
  },
  {
    "name": "Isabel",
    "_key": "Isabel",
    "family_name": "Gonzalez",
    "age": 65,
    "relationship_to_Norman": "Mother"
  },
  {
    "name": "Mario",
    "_key": "Mario",
    "family_name": "Gonzalez",
    "age": 45,
    "relationship_to_Norman": "Brother"
  },
  {
    "name": "Ana",
    "_key": "Ana",
    "family_name": "Martinez",
    "age": 43,
    "relationship_to_Norman": "Sister-in-law (Mario's wife)"
  },
  {
    "name": "Pedro",
```

```
    "_key": "Pedro",
    "family_name": "Gonzalez",
    "age": 38,
    "relationship_to_Norman": "Brother"
  },
  {
    "name": "Sofia",
    "_key": "Sofia",
    "family_name": "",
    "age": 42,
    "relationship_to_Norman": ""
  },
  {
    "name": "Rafael",
    "_key": "Rafael",
    "family_name": "Lopez",
    "age": 36
  },
  {
    "name": "Liam",
    "_key": "Liam",
    "family_name": "Lopez",
    "age": 12
  },
  {
    "name": "Olivia",
    "_key": "Olivia",
    "family_name": "",
    "age": 10
  },
  {
    "name": "Ethan",
    "_key": "Ethan",
    "family_name": "",
    "age": 7
  },
}
```

```

    {
        "name": "Amelia",
        "_key": "Amelia",
        "family_name": "",
        "age": 5
    }
]

child_of_edges = [
    {"_from": "Persons/Lucas", "_to": "Persons/Norman"},
    {"_from": "Persons/Mia", "_to": "Persons/Norman"},
    {"_from": "Persons/Eva", "_to": "Persons/Norman"},
    {"_from": "Persons/Norman", "_to": "Persons/Carlos"},
]

father_of_edges = [
    {"_from": "Persons/Norman", "_to": "Persons/Lucas"},
    {"_from": "Persons/Norman", "_to": "Persons/Mia"},
    {"_from": "Persons/Norman", "_to": "Persons/Eva"},
    {"_from": "Persons/Carlos", "_to": "Persons/Norman"},
]

db.collection("Persons").import_bulk(normans_family_members)
db.collection("is_son_of").import_bulk(child_of_edges)
db.collection("is_father_of").import_bulk(father_of_edges)

```

Resultado:

```

{'error': False,
 'created': 4,
 'errors': 0,
 'empty': 0,
 'updated': 0,
 'ignored': 0,
 'details': []}

```

Instantiate the ArangoDB-LangChain Graph:

```
from langchain_community.graphs import ArangoGraph
graph = ArangoGraph(db)
graph.set_schema()

# We can now view the generated schema
import json
print(json.dumps(graph.schema, indent=4))
```

Resultado del esquema generado:

```
{
  "Graph Schema": [
    {
      "graph_name": "Norman",
      "edge_definitions": [
        {
          "edge_collection": "is_father_of",
          "from_vertex_collections": [
            "Persons"
          ],
          "to_vertex_collections": [
            "Persons"
          ]
        },
        {
          "edge_collection": "is_son_of",
          "from_vertex_collections": [
            "Persons"
          ],
          "to_vertex_collections": [
            "Persons"
          ]
        }
      ]
    }
  ],
}
```

```
"Collection Schema": [  
  {  
    "collection_name": "is_father_of",  
    "collection_type": "edge",  
    "edge_properties": [  
      {  
        "name": "_key",  
        "type": "str"  
      },  
      {  
        "name": "_id",  
        "type": "str"  
      },  
      {  
        "name": "_from",  
        "type": "str"  
      },  
      {  
        "name": "_to",  
        "type": "str"  
      },  
      {  
        "name": "_rev",  
        "type": "str"  
      }  
    ],  
    "example_edge": {  
      "_key": "266257375170",  
      "_id": "is_father_of/266257375170",  
      "_from": "Persons/Norman",  
      "_to": "Persons/Lucas"  
    }  
  }  
]  
}
```

Configuración de la cadena de consultas:

```
from langchain.chains import ArangoGraphQChain
from langchain_openai import ChatOpenAI

chain = ArangoGraphQChain.from_llm(
    ChatOpenAI(temperature=0),
    graph=graph,
    verbose=True,
    allow_dangerous_requests=True
)
```

Pruebas de consultas en lenguaje natural. Se procede a hacer una pregunta desde un tercero ajeno a la persona de estudio en lenguaje natural:

```
chain.run("de quien es hijo Norman?")
```

Resultado:

```
Entering new ArangoGraphQChain chain...
AQL Query (1):
WITH Persons, is_father_of
FOR v, e, p IN 1..1 OUTBOUND 'Persons/Norman' is_father_of
RETURN v

AQL Result:
[{'_key': 'Eva', '_id': 'Persons/Eva', '_rev': '_iho5hiW--C',
  'name': 'Eva', 'family_name': 'Gonzalez', 'age': 8, ...}]

Finished chain.

'Norman tiene varios hijos. Entre ellos se encuentran Eva, Mia y Lucas. Lucas
↪ es el hijo de Norman.'
```

Segunda consulta:

```
chain.run("quienes son los hijos de Norman?")
```

Resultado:


```
Entering new ArangoGraphQACHain chain...
```

```
AQL Query (1):
```

```
WITH Persons, ChildOf
```

```
FOR v, e IN 1..1 OUTBOUND 'Persons/Norman' ChildOf
```

```
RETURN v
```

```
AQL Result:
```

```
[{'_key': 'Carlos', '_id': 'Persons/Carlos', '_rev': '_ihop_Sa--E',  
  'name': 'Carlos', 'family_name': 'Gonzalez', 'age': 68}]
```

```
Finished chain.
```

```
"Summary: The children of Norman are Carlos. Carlos is 68 years old and is
```

```
↪ identified as Norman's father in the database."
```

Prueba de escritura en la base de datos. Se prueba la capacidad de escritura usando lenguaje natural:

```
chain.run("add a new son of Norman named Xavier")
```

Resultado:

```
Entering new ArangoGraphQACHain chain...
```

```
AQL Query (1):
```

```
WITH Persons, ChildOf
```

```
LET norman = (
```

```
  FOR person IN Persons
```

```
  FILTER person.name == 'Norman'
```

```
  RETURN person
```

```
) [0]
```

```
LET xavier = {
```

```
  "_key": "Xavier",
```

```
  "name": "Xavier",
```

```
  "family_name": "Gonzalez",
```

```
  "age": 0
```

```
}
```

```
INSERT xavier INTO Persons
```

```
LET edge = {
  "_from": norman._id,
  "_to": CONCAT('Persons/', xavier._key)
}
```

```
INSERT edge INTO ChildOf
```

```
AQL Result:
```

```
[]
```

```
Finished chain.
```

```
'A new son named Xavier has been successfully added to Norman in the database.'
```

Verificación de la inserción:

```
chain.run("Norman tiene un hijo Xavier?")
```

Resultado:

```
Entering new ArangoGraphQACHain chain...
```

```
AQL Query (1):
```

```
WITH Persons, ChildOf
```

```
FOR v, e, p IN 1..1 OUTBOUND 'Persons/Norman' ChildOf
```

```
FILTER v.name == 'Xavier'
```

```
RETURN v
```

```
AQL Result:
```

```
[{'_key': 'Xavier', '_id': 'Persons/Xavier', '_rev': '_hux4B6W---',
  'name': 'Xavier', 'family_name': 'Gonzalez', 'age': 0}]
```

```
Finished chain.
```

```
"Summary: Yes, Norman has a son named Xavier. According to the database query
```

```
↪ results, Xavier is related to Norman as his son, with the same family name
```

```
↪ 'Gonzalez'."
```

Inserción en base de documentos con soporte vectorial y consultas

Habiendo descartado a las bases de grafos como soporte de los conocimientos usados para hiperpersonalización se experimenta en este punto con MongoDB, una base no SQL, que almacena documentos que no necesariamente tienen que responder a una estructura predefinida, lo que la hace ideal para administrar en una misma colección registros relacionados con diferentes áreas de la vida de un usuario. En el presente caso debemos insertar unidades de información relacionadas con personas, gastos y visitas al médico. En una primera etapa se decidió crear colecciones separadas para cada tipo de registro pero rápidamente la solución fue descartada porque agregaría un paso de selección de tipo de registro. Por ejemplo si el asistente recibe una consulta como “cuántos años tiene mi sobrina más chica?”, el sistema tendría que recorrer todas las colecciones disponibles para el usuario en curso y luego seleccionar en cuál de ellas hacer una búsqueda. Esto agrega probabilidad de error además de tiempo e ineficiencia. El código correspondiente a esta solución se omite del presente trabajo por no ser de relevancia.

La arquitectura seleccionada consiste en tomar una pieza de información, como la del siguiente ejemplo y calcularle los embeddings antes de guardarla en la base.

```
{  
  "datetime": "2023-12-01T10:00:00Z",  
  "description": "Monthly subscription to streaming service",  
  "type": "subscription",  
  "amount": 1,  
  "unit_of_measure": "service",  
  "unit_price": 15.0,  
  "total_cost": 15.0,  
  "payment_method": "direct debit",  
  "area_of_life": "entertainment"  
}
```

Todo el objeto se convierte a texto plano antes de la conversión y al obtener el vector se guarda en el campo ‘vector’ y se inserta en la colección única llamada ‘Records’.

Para que se pueda efectuar la búsqueda vectorial es necesario crear un índice de búsqueda

vectorial en MongoDB Atlas, el servicio provisto por la misma empresa que desarrolla y mantiene la base de datos.

Se ha probado correr una instancia local del mismo servicio con éxito pero para guardar de manera más confiable los datos de este trabajo y asegurar su posterior uso se decidió usar la capa gratuita de MongoDB Atlas en la nube.

Para crear un índice de búsqueda se debe especificar el campo dentro de cada documento en el que se encuentran los embeddings, en este caso el campo se llama “vector”. Luego se especifican las dimensiones del vector, en este caso, al usar text-embeddings-3-large el largo del vector es 3072. Por último se define qué tipo de algoritmo usar para calcular la distancia vectorial. Por el tipo de aplicación la distancia más apropiada es la coseno explicada en la introducción al capítulo.

El valor está entre 0 y 2, donde 0 indica que los vectores son idénticos en dirección y 2 que son opuestos. Generalmente como puntaje de similitud se usa $1 - \text{distancia coseno}$ para hacer más fácilmente interpretable el resultado. Entonces los valores más cercanos a 1 son los más similares.

Siguiendo este mecanismo se procesaron todos los conjuntos de datos sintéticos creados para simular información extraída sobre la vida de Norman, el sujeto ficticio que es el centro del presente trabajo. Como ya se explicó, todos los tipos de registros se insertaron en la misma colección llamada “Records” con un agregado previo para identificar el tipo de registro y quién es el dueño de ese registro, para dar lugar a posibles búsquedas filtradas si es que fueran necesarias en el futuro.

La base de conocimientos sobre Norman alcanzó los 187 documentos sobre personas en su vida, registros médicos y gastos.

A continuación se procede a comprobar la eficacia de esta solución simulando una conversación entre el usuario y su asistente personal con hiperpersonalización. El primer paso consiste en amplificar el mensaje del usuario para mejorar la recuperación de registros.

Por ejemplo, “hace cuánto no voy al dentista?” se amplifica a: “estoy pensando en mi última visita al dentista cuánto tiempo ha pasado desde que fui al dentista registro del dentista última vez que fui al odontólogo fechas de visita al dentista registros de citas dentales fichero de

dentista”. Con esta versión amplificada se calculan los embeddings para comenzar la búsqueda por similitud vectorial. Se puede ajustar el parámetro k que controla cuántos resultados se deben traer de la base. Generalmente se comienza con un valor de 5 y se ajusta a medida que se evalúa la exactitud de las respuestas en función de k .

La búsqueda vectorial, también llamada semántica, consiste en calcular la distancia coseno entre el vector que representa a la consulta ampliada y los diferentes vectores que representan las piezas de información. Se utilizan algoritmos que evitan tener que atravesar todo el universo de vectores, de lo contrario el sistema sería imposible de aplicar en un entorno de atención en tiempo real.

Una vez seleccionados los k vectores más cercanos a la consulta ampliada se procede a llamar a un modelo de lenguaje para que tome la pregunta inicial, siguiendo el ejemplo “hace cuánto no voy al dentista?” y la combine con el contexto para responder. El contexto es la suma de los k resultados en formato texto, ya que los embeddings no le son de utilidad y además generan un alto costo de implementación. Si en una consulta se comete el error de enviar los datos de contexto con sus embeddings es común obtener un error de overflow de tokens. En este punto es oportuno recordar que cada uno de los resultados de la base cuentan con un vector de embeddings, y usando el modelo mencionado los mismos cuentan con una dimensión de 3072. Pero además de eso cada número tiene una alta precisión y equivale aproximadamente a 10 tokens. Si $k=5$ y por error se envían los vectores de los resultados se exceden fácilmente los 150.000 tokens. Unos 20.000 más de lo autorizado por los más poderosos modelos de OpenAI a la fecha.

Al final de la notebook se ejecutan diferentes preguntas con resultados satisfactorios salvo en el caso de una pregunta combinada que debido a un error en la amplificación y recuperación no trae el contexto adecuado para la parte de generación de la respuesta.

Este punto puede ser resuelto por un paso anterior a la amplificación que reformule una pregunta combinada como dos preguntas separadas. Cada pregunta recibe el tratamiento existente y se combinan al final los contextos.

A continuación el código de los procesos detallados¹³.

Configuración inicial y conexión a MongoDB Atlas.

```
import os

from pymongo import MongoClient

# Connect to local MongoDB Atlas
mongo_uri = os.getenv("MONGODB_URI")

if not mongo_uri:
    raise ValueError("MONGODB_URI environment variable is not set")

client = MongoClient(mongo_uri)
db = client.get_database("hyper")

print(f"Connected to database: {db.name}")
```

Clase cliente para Atlas.

```
class AtlasClient():

    def __init__(self, atlas_uri, dbname):
        self.mongodb_client = MongoClient(atlas_uri)
        self.database = self.mongodb_client[dbname]

    def ping(self):
        self.mongodb_client.admin.command('ping')

    def get_collection(self, collection_name):
        collection = self.database[collection_name]
        return collection

    def find(self, collection_name, filter={}, limit=10):
        collection = self.database[collection_name]
        items = list(collection.find(filter=filter, limit=limit))
        return items
```

¹³ Código completo disponible en:

<https://github.com/aditzend/hyperpersonalization/blob/main/app/notebooks/34-populate-records-in-atlas.ipynb>

```

def vector_search(self, collection_name, index_name, attr_name,
                  embedding_vector, limit=5):
    collection = self.database[collection_name]
    results = collection.aggregate([
        {
            '$vectorSearch': {
                "index": index_name,
                "path": attr_name,
                "queryVector": embedding_vector,
                "numCandidates": 50,
                "limit": limit,
            }
        },
        {
            "$project": {
                '_id': 1,
                'title': 1,
                'plot': 1,
                'year': 1,
                "search_score": {"$meta": "vectorSearchScore"}
            }
        }
    ])
    return list(results)

def close_connection(self):
    self.mongodb_client.close()

```

Procesamiento y carga de registros médicos.

```

from openai import OpenAI
import json

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def get_embedding(text):

```

```

response = client.embeddings.create(
    input=text,
    model="text-embedding-3-large"
)

return response.data[0].embedding

# Load the medical_records.json file
with open('../data/norman/medical_records/medical_records.json', 'r') as
↪ file:
    medical_records_data = json.load(file)

# Process each medical record and add embeddings
for record in medical_records_data:
    if record['datetime'] != "-":
        record["record_owner"] = "Norman"
        record["record_type"] = "medical_record"
        record_text = json.dumps(record)
        embedding = get_embedding(record_text)
        record['vector'] = embedding

```

Insertión en la colección Records.

```

Records = atlas_client.get_collection('Records')

# Insert each record into the Records collection
for record in medical_records_with_embeddings:
    if len(record.get('datetime', '')) > 10:
        if isinstance(record['vector'], str):
            record['vector'] = json.loads(record['vector'])

        result = Records.insert_one(record)
        print(f"Inserted record with ID: {result.inserted_id}")
    else:
        print(f"Skipping record with invalid datetime: {record.get('datetime',
↪ 'N/A')}}")

```


Amplificación de consultas y búsqueda vectorial.

```

from pydantic import BaseModel, Field
from typing import Literal

class AmplifiedQuery(BaseModel):
    expanded_query: str
    record_type: Literal['medical_record', 'person_record', 'bank_record'] =
    ↪ Field(
        ...,
        description="Type of record to search for"
    )

# Query amplification using OpenAI completions
user_input = "cuántos años tienen mis hijas?"

amplification = client.beta.chat.completions.parse(
    model="gpt-4o-2024-08-06",
    messages=[
        {"role": "system", "content": f"You are a helpful assistant that
        ↪ expands user queries of the record owner Norman to improve search
        ↪ results using cosine distance in a vector database using text
        ↪ embeddings"},
        {"role": "user", "content": f"Expand this query for better search
        ↪ results, concatenate all query variations into a single string
        ↪ under the key 'expanded_query'. Then choose the most appropriate
        ↪ record_type to pick the answer from. The user input is:
        ↪ `{user_input}`"}
    ],
    max_tokens=1000,
    response_format=AmplifiedQuery
).choices[0].message.parsed

query_embedding = get_embedding(amplification.expanded_query)

# Perform vector search using Atlas client

```

```

results = atlas_client.vector_search(
    collection_name="Records",
    index_name="RecordsSearchIndex",
    attr_name='vector',
    embedding_vector=query_embedding,
    limit=3
)

```

Generación de respuesta final.

```

context = []
for result in results:
    full_doc = Records.find_one({"_id": result["_id"]})
    context.append(str(full_doc))

context_str = "\n".join(context)
final_answer = client.chat.completions.create(
    model="gpt-4o-2024-08-06",
    messages=[
        {"role": "system", "content": "You are a helpful assistant that answers
        ↪ questions based on the given context."},
        {"role": "user", "content": f"Context:\n{context_str}\n\nQuestion:
        ↪ {user_input}\n\nAnswer the question based on the context."}
    ],
    max_tokens=150
).choices[0].message.content

```

El sistema demostró capacidad para responder correctamente preguntas específicas como “cuántos años tienen mis hijas?” obteniendo la respuesta “Tus hijas tienen 8 años. Los datos proporcionados mencionan a Mía y Eva, quienes son etiquetadas con la relación ‘daughter’ (hija) y ambas tienen la edad de 8 años.”

Para consultas más complejas que combinan diferentes tipos de registros, el sistema requiere mejoras en la estrategia de amplificación para manejar preguntas compuestas que abarcan múltiples categorías de información.

Conclusiones

Viabilidad

Los experimentos efectuados en la sección de desarrollo demuestran que el mecanismo diseñado cumple la premisa dotando de capacidades de hiperpersonalización a un asistente virtual. La combinación de modelos de lenguaje preentrenados y embeddings es suficiente para procesar la información necesaria siempre que la cantidad de información almacenada pueda ser procesada efectivamente por el motor de búsqueda de la base vectorial.

Recursos Necesarios para la Implementación

Los componentes necesarios para la implementación son los siguientes:

- Un servicio para extraer información y estructurarla a partir de una conversación en curso.
- Una base de datos con capacidad para calcular similitud vectorial e idealmente también búsqueda por texto para poder hacer frente a necesidades de búsqueda híbrida.
- Un servicio de inserción de nuevas piezas de información en la base de datos.
- Un servicio de amplificación de mensajes de usuario con manejo de contexto de conversación.
- Un servicio de generación de respuestas a base de contexto histórico.

Además de la base de datos se necesita acceso a un servicio que reciba texto y devuelva embeddings y un servicio de modelo de lenguaje. Ambos pueden correrse en una computadora de escritorio con suficiente capacidad o consumirse mediante suscripciones a proveedores de primera línea como OpenAI, Google Cloud Platform y Amazon Web Services, entre muchos otros.

Limitaciones de la solución actual

Los experimentos sobre problemas de razonamiento cronológico usando modelos ya deprecados al finalizar este trabajo como gpt-3.5 requerían retrabajo de las fuentes de información. Con el propio avance de los modelos de lenguaje estas reconfiguraciones para

facilitar las tareas de razonamiento pueden no ser necesarias pero las técnicas vistas pueden servir en el caso de aplicar hiperpersonalización en modelos menos potentes, particularmente de entre mil y siete mil millones de parámetros. Este punto no es menor ya que estos modelos pueden correr hoy en dispositivos personales sin acceso a un servidor remoto con los beneficios de privacidad que esto conlleva.

Funcionalidades a desarrollar en fases sucesivas

El sistema planteado como solución no cuenta con manejo de memoria de conversación.

Todos los mensajes se responden sin interconexión y esto claramente es inviable en un entorno productivo. Un servicio de observación debe estar corriendo en paralelo al servicio principal de la conversación extrayendo y estructurando toda la información necesaria de la conversación. En este trabajo solo se planteó cómo hacerlo a partir de una conversación simulada.

Como ya se mencionó en el apartado de desarrollo, el sistema no es capaz de responder sobre más de una temática en una misma iteración. Ésta es una característica de producto que debe validarse en el mercado previamente antes de comenzar con el desarrollo.

Luego de estas mejoras se debe pensar en los desafíos que se presentan fuera del ámbito académico, saliendo de la esfera de pensamiento para entrar en el complejo mundo de la implementación. No se puede esperar que una persona vuelque la información de todos los campos de su vida en una base de datos sin garantizar la privacidad y el control de lo que comparte.

Es por este motivo que el siguiente paso en el camino hacia un mundo con asistentes virtuales hiperpersonalizados debe centrarse en los mecanismos para almacenar, leer y permisionar datos en una cadena de bloques, más conocida como blockchain.

Este tipo de base de datos se caracteriza por la confiabilidad de los procesos de manejo de data, ya que es físicamente imposible alterar un bloque una vez que es parte de la cadena. Además su almacenamiento es distribuido y compartido entre entidades diferentes, lo que hace imposible el control central y monopólico de los datos.

La contracara de este tipo de soluciones viene por el lado de la complejidad de los desarrollos, la lentitud de las operaciones y el costo de funcionamiento. En los últimos años han aparecido

gran variedad de avances para mitigar estos costos y difundir el uso de blockchains en todo tipo de industrias, pero todavía no existe una solución que resuelva la escritura y lectura de grandes volúmenes de datos, a un costo accesible y a una velocidad razonable.

La propuesta para el siguiente paso es desarrollar esa tecnología. Cuando se alcancen los objetivos antes mencionados, las operaciones sobre bases de datos mencionadas en este trabajo se reemplazarán por escrituras y lecturas sobre una blockchain. Los usuarios finales, los verdaderos dueños de los datos, podrán llevarse su información personal al servicio de inteligencia artificial que gusten. Expresado de otra manera, podrán permisionar sus datos a voluntad.

El equilibrio perfecto entre disfrutar de las comodidades que nos puede traer la inteligencia artificial y hacer valer nuestro derecho a la privacidad requiere diseño, trabajo y la confluencia de variadas tecnologías. El futuro que deseamos no ocurre por acción de la entropía, debe ser creado.

Anexo - Plan de Negocio

Introducción

Considerandos

La investigación llevada a cabo en la tesis abre la posibilidad de creación de un emprendimiento comercial ya que los hallazgos que arroja coinciden con necesidades insatisfechas detectadas en el mercado de los asistentes virtuales.

En el diseño de la propuesta de valor es crucial recordar que los datos son el activo más importante de las organizaciones. Es poco probable que una empresa permita la copia de toda la información de sus clientes en una base de datos externa y fuera de su control.

Actores del mercado

Como en toda industria existen diferentes tipos de jugadores de acuerdo a la complejidad y volumen de conversaciones que un asistente virtual debe manejar.

En el estamento más básico y de bajo volumen podemos encontrar a los proveedores de lo que se podría llamar “haga su propio bot”, que ofrecen una solución integrada funcionando en una aplicación web. Los usuarios aprenden solos a usar el producto y gestionan el mantenimiento de su bot de manera totalmente autónoma. Es la solución ideal para dueños de tiendas online que no requieran integraciones complejas con otros sistemas.

Luego existe un grupo de oferentes de desarrollo de asistentes virtuales para los casos en los que se necesite muy alta flexibilidad y ajuste a necesidades particulares, con una llegada directa a los desarrolladores. En estos casos el volumen de conversaciones es bajo o intermedio. Este grupo es conocido como las “agencias de bots” o “consultoras boutique”.

En el estamento siguiente podemos encontrar a aquellos proveedores que ofrecen una plataforma de autogestión pero para resolver conversaciones de complejidad intermedia y volumen alto. Estas empresas ofrecen horas de desarrollo de equipos propios para la puesta en marcha de los proyectos y luego los equipos internos del dueño del bot continúan con la administración y mantenimiento de manera autónoma. En vez de una tarifa plana generalmente cobran una tarifa por caso, que es lo que se conoce comúnmente como una sesión. Clientes comunes de este tipo de empresas son bancos, empresas de telecomunicaciones y de prestación de servicios de salud.

Cuando se requiere un máximo nivel de robustez el mercado acude a lo que se llaman las soluciones “world class”, que constituyen el tope de gama en desarrollo y mantenimiento de asistentes virtuales. Empresas multinacionales que deciden contrataciones a nivel global para todas sus filiales son claros ejemplos de clientes de este tipo de oferentes. La fortaleza está dada por la interconexión con otros servicios, la robustez, la diversidad de reportes y el soporte, pero como ocurre ante tanta estructura lo que se pierde es configurabilidad, flexibilidad y velocidad de adaptación.

Definición del producto

Considerando los puntos mencionados se deduce que la tecnología propuesta en la tesis debe enmarcarse en una solución llamada “Software de hiperpersonalización para asistentes virtuales”.

Se instala localmente o en el entorno del cliente, actuando como un middleware entre las fuentes de datos y el motor del bot.

Características principales:

- Observa el flujo de las conversaciones.
- Proporciona contexto adecuado basado en los datos disponibles.
- Asegura la hiperpersonalización sin comprometer la seguridad o privacidad de los datos.

Ventajas:

- Reduce las preocupaciones de las empresas sobre compartir datos sensibles.
- Facilita la integración con sistemas existentes, como CRM o bases de datos internas.
- Puede ser adaptado a diferentes industrias y casos de uso.

Se venderá en licencias que se instalan en los entornos propios de los clientes.

Análisis estratégico***Análisis FODA*****Fortalezas (Internas).**

- **Diferenciación:** Combina múltiples fuentes de datos y optimiza la hiperpersonalización en tiempo real. No existen competidores aún.
- **Privacidad y seguridad:** La instalación local o en un entorno controlado por el cliente reduce las preocupaciones sobre la transferencia de datos.
- **Compatibilidad:** Diseñado para integrarse con diferentes sistemas y bases de datos (e.g., CRM, ERP).
- **Escalabilidad:** Puede adaptarse a diversas industrias y volúmenes de datos, desde pequeñas empresas hasta grandes corporaciones.
- **Valor estratégico:** Ayuda a las empresas a maximizar la eficiencia de sus asistentes virtuales, mejorando la experiencia del cliente y optimizando recursos.

Debilidades (Internas).

- **Requiere personalización inicial:** Cada cliente puede necesitar adaptaciones específicas para integrar el middleware con sus sistemas.
- **Costos de desarrollo y mantenimiento:** Mantener compatibilidad con múltiples sistemas y ofrecer soporte técnico puede requerir recursos significativos.
- **Curva de aprendizaje:** Empresas con menor madurez tecnológica podrían tener dificultades para comprender o implementar la solución.

Oportunidades (Externas).

- **Crecimiento del mercado de IA:** Aumento explosivo en la adopción de asistentes virtuales en sectores clave como salud, comercio y finanzas.

- **Cumplimiento de regulaciones:** Empresas buscan soluciones que les permitan cumplir con normas de seguridad y protección de datos mientras implementan personalización avanzada.
- **Tendencia hacia la descentralización:** Preferencia por soluciones que no requieran enviar datos a terceros.

Amenazas (Externas).

- **Competencia:** Grandes empresas podrían ofrecer soluciones integradas a su oferta actual de productos.
- **Barreras de adopción:** Resistencia inicial de las empresas a instalar software intermedio sin ver resultados claros.
- **Dependencia tecnológica:** Cambios en los sistemas base de los clientes podrían requerir actualizaciones constantes.
- **Cambio en prioridades del mercado:** La aparición de nuevas tecnologías podría desviar el interés hacia otras soluciones.

Propuesta de valor

Problema a resolver.

Desconexión entre datos y asistentes virtuales existentes. Las empresas tienen múltiples fuentes de datos (CRM, ERP, bases de datos internas) que no están integradas con sus asistentes virtuales. Los bots suelen carecer de contexto relevante para ofrecer respuestas personalizadas y efectivas.

Incapacidad de los asistentes virtuales actuales para entender el contexto dinámico. La falta de hiperpersonalización reduce la calidad de la interacción y la satisfacción del cliente. Esto puede llevar a pérdida de ventas, frustración del usuario y mayor carga para los canales tradicionales (como atención telefónica).

Privacidad y cumplimiento normativo. Muchas empresas no pueden alojar o transmitir sus datos fuera de sus centros de datos propios por preocupaciones legales y de seguridad.

Especificación de la propuesta de valor.

Para las empresas.

- **Optimización de interacciones:** Permite que los bots ofrezcan respuestas personalizadas en tiempo real utilizando múltiples fuentes de datos.
- **Cumplimiento normativo:** Los datos permanecen en el entorno seguro de la empresa.
- **Mejor integración:** Facilita la conexión entre sistemas existentes y el motor del bot, sin necesidad de rediseñar infraestructuras completas.
- **Reducción de costos:** Disminuye la necesidad de intervención humana en las interacciones repetitivas.

Para los usuarios finales.

- **Experiencia superior:** Interacciones más relevantes y fluidas, adaptadas a sus necesidades específicas.
- **Respuesta rápida:** Menor tiempo para resolver problemas o acceder a información personalizada.

Ejemplos de impacto por industria

Comercio electrónico: Pasajes. A continuación se detallan los problemas comunes en el sector:

Altos volúmenes de consultas repetitivas. : Cambios de vuelo, estado del itinerario o políticas de equipaje.

Falta de personalización. : Bots actuales no tienen en cuenta el historial del cliente, lo que lleva a experiencias impersonales como preguntar repetidamente por preferencias de asiento o necesidades especiales.

Necesidad de cumplimiento normativo. : Las aerolíneas manejan datos sensibles (nombres, pasaportes, métodos de pago) que requieren altos estándares de seguridad.

Aplicando la propuesta de valor a esta industria se recalca que el middleware actúa como un intermediario entre los sistemas internos de la empresa y el motor del bot, permitiendo acceso contextualizado a los datos, integrando CRM, sistemas de reservas (e.g., Amadeus, Sabre), y

registros de clientes frecuentes. Además la hiperpersonalización en tiempo real proporciona respuestas relevantes basadas en el historial del cliente, preferencias previas y datos actuales. Por el lado del cumplimiento de normativas el aporte recae en que los datos no abandonan los servidores de la aerolínea, garantizando privacidad y seguridad.

Escenario práctico: Cliente frecuente solicita información.

1. Sin middleware:

- Bot: “¿Cuál es su número de vuelo?”
- Usuario: Proporciona información varias veces.
- Bot: Da respuestas genéricas sobre cambios de vuelo o equipaje.

2. Con middleware:

- Bot: “Hola, Alexander. Veo que tenés un vuelo a Río de Janeiro mañana desde Buenos Aires. ¿Querés modificar tu asiento o agregar equipaje?”
- Usuario: “Sí, prefiero una ventana.”
- Bot: “Listo, te cambié a una ventana en el asiento 12A. ¿Necesitás algo más?”

Resultados esperados. Las empresas de transporte podrían reducir entre un 30% y un 50% las consultas manejadas por humanos. Según Botpress, los chatbots pueden responder hasta el 79% de las consultas rutinarias en atención al cliente. Pero se toma un valor más conservador dado que cada implementación tiene fuerte dependencia del contexto.

No solo se verificaría un ahorro de costos sino un incremento sustancial en ventas adicionales (asientos, equipaje, upgrades) con un engagement superior en tiempo real. McKinsey & Company indica que la personalización puede aumentar los ingresos entre un 10% y un 15%, dependiendo del sector y la capacidad de ejecución. En el mismo trabajo se indica que el 71% de los consumidores espera interacciones personalizadas, y el 76% se siente frustrado cuando no las recibe.

Industria bancaria. En esta industria se encuentran generalmente los siguientes inconvenientes:

Consultas repetitivas sobre saldos, estados de cuenta, transferencias y pagos recurrentes. Estas interacciones ocupan tiempo del personal y generan frustración en los clientes si los bots no proporcionan respuestas rápidas y precisas. Los bots actuales no consideran el historial financiero del cliente, lo que resulta en recomendaciones genéricas en lugar de sugerir opciones pensadas para la situación real de cada cliente. Los bancos manejan datos financieros sensibles y deben cumplir estrictas regulaciones, como GDPR, CCPA y normativas locales de privacidad.

El middleware actúa como un intermediario entre los sistemas internos del banco y el motor del bot, permitiendo acceso contextualizado a los datos: Integra CRM, sistemas transaccionales y datos de clientes, proporcionando información relevante en tiempo real. La hiperpersonalización en tiempo real permite responder con precisión basándose en el historial de transacciones, hábitos financieros y necesidades específicas del cliente.

Escenario práctico: Cliente solicita información sobre su cuenta.

1. Sin middleware:

- Bot: “¿Cuál es el número de tu cuenta?”
- Usuario: Proporciona la información varias veces.
- Bot: Da respuestas genéricas sobre saldos o transferencias.

2. Con middleware:

- Bot: “Hola, María. Tu cuenta principal tiene un saldo de \$12,500. ¿Quieres programar una transferencia o conocer los detalles de tu última transacción?”
- Usuario: “Sí, necesito transferir \$2,000.”
- Bot: “Listo, programé la transferencia. ¿Te gustaría recibir una notificación cuando se complete?”

Los clientes reciben respuestas rápidas y personalizadas, mejorando su experiencia con los servicios bancarios. Se disminuye la carga de trabajo de los agentes humanos, optimizando recursos operativos y las recomendaciones personalizadas aumentan las ventas de productos financieros, como créditos o inversiones. Una experiencia personalizada refuerza la relación

con los clientes, promoviendo la fidelización. En números, los resultados esperados coinciden con el apartado anterior.

Servicios de salud. Los problemas comunes de atención al cliente en las organizaciones de provisión de servicios de salud se enmarcan en preguntas frecuentes sobre cobertura, turnos médicos, y autorizaciones de estudios o tratamientos. Estas consultas ocupan un tiempo considerable del personal de atención y pueden generar demoras, lo que afecta la satisfacción de los pacientes. Por otro lado, los bots actuales no consideran el historial médico del paciente ni sus preferencias, lo que lleva a experiencias impersonales. Al igual que los ejemplos anteriores, las empresas de medicina prepaga manejan información altamente sensible (historial médico, tratamientos, datos de asegurados) y deben cumplir con regulaciones de protección de datos personales y estándares internacionales de privacidad.

El middleware actúa como un intermediario entre los sistemas internos de la empresa de medicina prepaga y el motor del bot, permitiendo acceso contextualizado a los datos porque, al igual que los casos anteriores, integra CRM, historial médico y sistemas de gestión de turnos, proporcionando información precisa en tiempo real. Los bots hiperpersonalizados pueden generar respuestas adaptadas al historial del paciente, sus consultas previas y las características de su plan. Todo esto garantizando que los datos médicos permanezcan dentro del entorno seguro de la empresa, cumpliendo con las regulaciones aplicables.

Escenario práctico: Paciente solicita un turno médico.

1. Sin middleware:

- Bot: “¿Qué especialidad necesitas?”
- Usuario: Proporciona información básica y espera varios pasos para obtener opciones disponibles.
- Bot: Da respuestas genéricas sobre médicos sin considerar antecedentes o ubicaciones preferidas.

2. Con middleware:

- Bot: “Hola, Martín. Veo que necesitas un turno con un cardiólogo, como en tu

consulta anterior. ¿Querés agendar con el Dr. López en la clínica cercana a tu domicilio?”

- Usuario: “Sí, por favor.”
- Bot: “Listo, tu turno es el lunes a las 14:00. ¿Querés recibir un recordatorio el día anterior?”

Los pacientes reciben respuestas rápidas y personalizadas, reduciendo su frustración y mejorando su experiencia. Además se disminuye la carga operativa de los agentes humanos, liberando recursos para tareas más complejas sin contar la optimización en la asignación de citas, considerando la disponibilidad y preferencias de los pacientes.

Plan comercial con estimación de ventas***Definición del producto***

El middleware propuesto es una solución que actúa como intermediario entre las fuentes de datos de una empresa y sus asistentes virtuales, proporcionando contexto en tiempo real para generar respuestas hiperpersonalizadas. Se vende en forma de licencias con renovación anual. La implementación se cobra aparte y es realizada por un equipo de consultores propios.

Segmentación del mercado objetivo

Se identifican los siguientes sectores como los más beneficiados por la implementación del middleware:

- Banca y Servicios Financieros
- Comercio Minorista y Comercio Electrónico
- Atención Médica y Ciencias de la Vida
- Telecomunicaciones y Tecnología de la Información
- Viajes y Hospitalidad

Estrategia de penetración en el mercado

Para cada sector, se desarrollarán estrategias específicas que incluyen:

- Desarrollar soluciones que aborden problemas particulares de cada industria.
- Asegurar que el middleware cumpla con las regulaciones específicas de cada sector.
- Integración con los sistemas y plataformas existentes en cada industria. Principalmente CRMs.

Modelo de ingresos

Se propone un modelo de ingresos basado en licencias de software con el cobro de una tarifa inicial por la implementación del middleware y renovaciones anuales.

Para fundamentar una proyección de ventas sólida para el middleware de hiperpersonalización en asistentes virtuales, es esencial analizar el mercado objetivo, sus tendencias de crecimiento y las oportunidades específicas en cada sector.

Tamaño y crecimiento del mercado de asistentes virtuales

Según **emergen_virtual_assistants_2023**<empty citation>, el mercado global de asistentes virtuales ha mostrado un crecimiento significativo en los últimos años y se espera que esta tendencia continúe:

- Valor de Mercado en 2022: USD 11,40 mil millones.
- Proyección para 2032: USD 150,58 mil millones.
- Tasa de Crecimiento Anual Compuesta (CAGR): 29,6% durante el período 2022-2032.

Mercado de hiperpersonalización

También **emergen_hyperpersonalization_2024**<empty citation> afirma que la hiperpersonalización está en expansión:

- Valor de Mercado en 2024: USD 19,37 mil millones.
- Tasa de Crecimiento Anual Compuesta Proyectada: 15,83% durante el período de pronóstico.

Adopción de asistentes virtuales en américa latina

De acuerdo a un reporte de **informes_expertos_latam_2024**<empty citation>, en América Latina, la adopción de asistentes virtuales inteligentes está creciendo a tasas similares:

- Valor de Mercado en 2023: USD 5,43 mil millones.
- Proyección para 2032: USD 16,18 mil millones.
- Tasa de Crecimiento Anual Compuesta Proyectada: 12,9% entre 2024 y 2032.

Proyección de Ventas

Según mentzer2005sales<empty citation>, los métodos cualitativos son los más adecuados para hacer pronósticos en el largo plazo. Para empresas nuevas, sin datos históricos, los autores recomiendan el método Delphi, que es una técnica cualitativa de pronóstico que se utiliza para obtener un consenso de expertos sobre un tema en particular, como las estimaciones de ventas futuras. Este método es especialmente útil cuando no se dispone de datos históricos confiables o cuando se trata de predecir escenarios complejos e inciertos.

Metodología de estimación ("Delphi simulado con LLM"). Para estimar variables de demanda y unit economics realizamos un Delphi simulado asistido por IA. A diferencia del método Delphi ortodoxo, que requiere un panel de expertos humanos que itera en rondas anónimas hasta lograr consenso, aquí simulamos perfiles de expertos con un modelo de lenguaje (LLM), ejecutando múltiples rondas y consolidando métricas de consenso (mediana, IQR).

El procedimiento constó de: (i) definición de variables a estimar (nuevos clientes por año, ACV/precio, CAC, churn anual, tiempo de implementación); (ii) fijación de anclas empíricas a partir de fuentes primarias (reportes sectoriales) para acotar rangos; (iii) generación de 5 "perfiles de experto" (p. ej., VC SaaS LATAM, CTO de proveedor, Head of CS, economista macro AR, consultor pricing B2B) con ronda 1 de estimaciones; (iv) ronda 2 con retroalimentación de estadísticas agregadas (medianas y cuartiles) y pedido de revisión; (v) consolidación por mediana y IQR; (vi) validación de plausibilidad contra benchmarks publicados.

Limitación: al no involucrar expertos humanos reales, el resultado no constituye consenso experto; se interpreta como escenarios informados por el LLM bajo restricciones de datos. Por ello, los resultados se usan para armar escenarios Pesimista/Base/Optimista y para análisis de sensibilidad, no como pronóstico puntual.

Parámetros de reproducibilidad. Modelo y fecha: 'gpt-5.1-2025-11-13', ejecución: 2025-11-23

Temperatura: 0.0; top_p: 1.0; semillas distintas por "experto" y ronda

Rondas: 2 (criterio de convergencia: $\Delta\text{mediana} < 10\%$ y $\text{IQR}/\text{mediana} < 30\%$)

Detalle del método Delphi:

1. **Selección del panel de expertos:** Se eligen especialistas en el área relevante (por ejemplo, mercado, ventas, industria) que puedan aportar conocimiento y experiencia. El panel puede incluir expertos internos (empleados) o externos a la empresa.
2. **Definición del problema:** Se establece claramente el objetivo del pronóstico, como estimar las ventas en un nuevo mercado o predecir la demanda de un producto en desarrollo.
3. **Primera ronda de consultas:** Cada experto responde un cuestionario sobre el tema de interés. Las respuestas son recopiladas de forma anónima para evitar sesgos o influencias entre los participantes.
4. **Análisis de las respuestas:** Los facilitadores (coordinadores del proceso Delphi) analizan y resumen las respuestas, destacando áreas de acuerdo y desacuerdo. Se calcula un promedio o rango de las estimaciones iniciales.
5. **Segunda ronda de consultas:** Los expertos reciben un resumen de las respuestas del grupo junto con las estimaciones promedio. Se les pide que revisen sus respuestas iniciales a la luz de esta información y, si cambian de opinión, expliquen las razones.
6. **Repetición de rondas:** El proceso de consulta y revisión continúa en múltiples rondas (generalmente 2-3) hasta que se alcanza un consenso razonable o las respuestas muestran estabilidad.
7. **Presentación del pronóstico final:** El resultado final es una estimación basada en el consenso de los expertos, que puede incluir rangos o escenarios para reflejar las incertidumbres.

En este caso, al no tener empleados se seleccionan cinco perfiles de la industria latinoamericana de asistentes virtuales o relacionados con la inteligencia artificial.

Ante la imposibilidad de generar un encuentro físico con expertos reales, se ha simulado el desarrollo del método reemplazando las visiones de cada experto por simulaciones con el

modelo de lenguaje “openai o1-2024-12-17”, el más avanzado a la fecha de confección de este trabajo, que ha superado a expertos humanos en el test MMLU por un margen promedio de 2.5% (92.3% vs. 89.8%).

Justificación de métodos para predicción de ventas. Al tratarse de un producto nuevo desarrollado por una empresa emergente que ingresa en un mercado competitivo como el de los asistentes virtuales en América Latina, no es viable aplicar métodos cuantitativos tradicionales, como regresiones o promedios móviles, para la predicción de ventas. Estos enfoques requieren datos históricos confiables que no están disponibles para este caso, ya que no existen ventas previas que permitan modelar tendencias y la industria de asistentes virtuales está en rápida evolución, lo que dificulta basarse en datos históricos de otros productos como referencia. Las variables clave, como adopción tecnológica, estrategias de marketing y competitividad, introducen incertidumbre significativa en el análisis. En su lugar, es más apropiado utilizar métodos cualitativos como el método Delphi, que permite obtener proyecciones fundamentadas a través del consenso de expertos.

Simulación con el Método Delphi Simulado

Perfiles de expertos simulados. Se simularon cinco perfiles especializados:

- **CEO Startup:** CEO y Co-fundador/a de una Startup Latinoamericana de Soluciones de Inteligencia Artificial Conversacional
- **Gerente de producto:** Product Owner de una Startup Latinoamericana de Soluciones de Inteligencia Artificial Conversacional
- **Analista de mercado:** Analista de mercados emergentes con foco en Argentina
- **Consultor en tecnología:** Consultor en tecnología para software empresarial
- **Investigador académico:** Investigador académico en inteligencia artificial conversacional

Variables estimadas: nuevos clientes por año, precio anual (ACV), costo de adquisición de clientes (CAC), tasa de churn anual, tiempo de implementación.

Fundamentación Empírica para el Método Delphi. La estimación de variables críticas para la proyección de adopción de un sistema de hiperpersonalización aplicado al desarrollo de asistentes virtuales requiere el uso de evidencia sólida y verificable. En el marco del método Delphi, es fundamental que los expertos participantes cuenten con un conjunto de referencias empíricas que les permitan acotar razonablemente los valores iniciales, evitando sesgos de sobreoptimismo, heurísticas individuales o extrapolaciones no fundamentadas. Para ello, se ha realizado una revisión exhaustiva de literatura técnica, estudios de mercado, reportes industriales y análisis de consultoras globales que cubren tres dimensiones centrales: (i) adopción de IA y asistentes conversacionales en empresas; (ii) madurez de la personalización y experiencia del cliente; y (iii) métricas operativas relevantes tales como potencial de mercado, inversión proyectada, tiempos de implementación, barreras de adopción y rotación de clientes (churn).

Los reportes de [zendesk2024cxtrends](#); [zendesk2024_cxmaturity_latam](#)<empty citation> aportan datos consistentes sobre la evolución del *Customer Experience* (CX) en América Latina, incluyendo niveles de automatización, uso de asistentes conversacionales, expectativas de personalización y la disposición de las organizaciones a incorporar soluciones basadas en IA. Estos informes permiten comprender el grado de madurez regional y constituyen una referencia directa para estimar cuántas organizaciones en Argentina y la región estarían en condiciones de adoptar un middleware de hiperpersonalización.

En la misma línea, los documentos de Gartner ([gartner2024mq_conversational_ai](#); [gartner2024_marketguide_conversational_ai](#); [gartner2024_hypecycle_ai](#)) describen la estructura del mercado de plataformas conversacionales, su evolución tecnológica, y niveles de madurez en el ciclo de adopción. Si bien el acceso completo a los reportes requiere suscripción, los resúmenes públicos, las metodologías y los criterios empleados por Gartner funcionan como una guía metodológica sólida para definir escenarios de adopción y límites razonables para el crecimiento del sector.

La dimensión económica del mercado está representada principalmente por el *Artificial Intelligence Spending Guide* de IDC ([idc2024_ai_spending_latam](#)), que presenta datos de inversión proyectada en IA para América Latina. Esto es particularmente relevante, ya que el

volumen de gasto en tecnologías de IA y automatización establece un techo presupuestario realista para la incorporación de soluciones de alto valor agregado como el sistema propuesto.

Por su parte, las encuestas globales de McKinsey (**mckinsey2024stateofai**), Deloitte (**deloitte2024_state_of_ai**), Accenture (**accenture2024_ai_maturity**) y PwC (**pwc2024_global_ai_study**) proporcionan estimaciones robustas sobre adopción corporativa de IA, impacto económico, fuentes de valor, barreras de implementación y métricas de retorno. Estas fuentes funcionan como referencia para inferir la velocidad de penetración esperada, los patrones de adopción organizacional, y los factores que potencian o inhiben la expansión de soluciones avanzadas.

La perspectiva latinoamericana se complementa con estudios específicos como el *State of Personalization* de Twilio Segment (**twilio2024_personalization_latam**), que analiza comportamientos de personalización en empresas de la región; los informes de Meta sobre comercio conversacional (**meta2024_conversational_commerce_latam**); el reporte regional de Salesforce (**salesforce2024_state_of_service_latam**); y el análisis del ecosistema digital en América Latina elaborado por Google Cloud (**google2024_future_ai_latam**). Estos trabajos aportan una visión granular del comportamiento de los usuarios, la penetración de canales conversacionales y el grado de madurez en automatización dentro de industrias verticales.

Asimismo, reportes de análisis de mercado como Statista (**statista2024_latam_ai**), Omdia (**omdia2024_conversational_ai_forecast**), Everest Group (**everest2024_enterprise_conversational_ai**) y BCG (**bcg2025_personalization_index**) proveen estimaciones cuantitativas del tamaño del mercado, crecimiento anual compuesto (CAGR), expansión proyectada para plataformas conversacionales y el impacto económico de la personalización basada en IA. Estas cifras permiten estructurar límites superiores e inferiores para la cantidad potencial de clientes, su velocidad de incorporación y los ritmos de despliegue.

Finalmente, se incorporan referencias académicas revisadas por pares, como el artículo de IEEE (**ieee2024_ai_latam**), que complementa la evidencia industrial a través de análisis sistemáticos de adopción de IA en organizaciones latinoamericanas, permitiendo fundamentar las diferencias regionales respecto de mercados de mayor madurez.

El conjunto de todas estas fuentes constituye una base empírica robusta, multidimensional y coherente, que permite enmarcar adecuadamente las estimaciones del método Delphi. Cada informe contribuye a acotar una variable específica—sea adopción, inversión, madurez, impacto económico o barreras—y la combinación de todos estos elementos reduce el riesgo de estimaciones arbitrarias o inconsistentes entre panelistas.

Anexo para Expertos del Método Delphi. A continuación se presenta el inciso que acompañará las preguntas del panel Delphi:

Nota metodológica para expertos. Las estimaciones solicitadas en este cuestionario deben considerarse a la luz de la evidencia sectorial relevada previamente. Para garantizar coherencia y reducir sesgo individual, se proveen como referencia los principales reportes globales y regionales sobre adopción de inteligencia artificial, plataformas conversacionales y tecnologías de personalización, incluyendo estudios de Gartner (2024), IDC (2024), McKinsey (2024), Deloitte (2024), Accenture (2024), PwC (2024), Zendesk (2024), Twilio Segment (2024), Meta (2024), Google Cloud (2024), Salesforce (2024), Statista (2024), Omdia (2024), BCG (2025), Everest Group (2024) y análisis académicos (IEEE, 2024).

Se solicita que sus respuestas tomen en consideración los rangos, tendencias, niveles de madurez y limitaciones identificadas en dichas fuentes, aplicándolos al contexto argentino y latinoamericano según su experiencia profesional. Las estimaciones deben basarse en escenarios plausibles y justificables a partir de esta evidencia, evitando supuestos fuera de escala respecto de la literatura técnica y las proyecciones existentes.

Primera Ronda. Se distribuye la pregunta inicial a los expertos, quienes responden de forma independiente.

Cuadro 1

Ronda 1 - Proyección de clientes por perfil de experto

Ronda 1 - Clientes	Año 1	Año 2	Año 3
CEO Startup	8-15	23-45	50-70
Gerente de producto	8-15	15-30	22-50
Analista de mercado	10-20	18-44	27-75
Consultor en tecnología	8-12	20-30	40-60
Investigador académico	8-12	16-30	30-60

Análisis y Retroalimentación. Se promedian estos resultados y se vuelven a correr las consultas para dar lugar a las predicciones de la segunda ronda.

Cuadro 2

Ronda 2 - Proyección de clientes por perfil de experto

Ronda 2 - Clientes	Año 1	Año 2	Año 3
CEO Startup	10-15	20-30	35-50
Gerente de producto	10-18	18-40	30-60
Analista de mercado	10-15	20-33	30-60
Consultor en tecnología	12-15	25-35	21-35
Investigador académico	10-15	20-35	40-60

Consolidación de resultados y escenarios. Aplicando medianas y rangos intercuartílicos (IQR), se establecen tres escenarios:

Cuadro 3

Escenarios de negocio consolidados

Variable	Pesimista (P25)	Base (Mediana)	Optimista (P75)	Unidad
Clientes Año 1	8	12	16	clientes
Clientes Año 2	18	25	35	clientes
Clientes Año 3	30	45	65	clientes
Precio anual (ACV)	10,000	12,000	15,000	USD
CAC	3,500	2,800	2,200	USD
Churn anual	15%	10%	7%	%
Tiempo implementación	4	3	2	meses

Análisis de sensibilidad. Se evalúa el impacto de variaciones en las variables clave sobre los ingresos del Año 3:

Cuadro 4

Análisis de sensibilidad - Impacto en ingresos Año 3

Variable	Variación	Impacto en ingresos	Sensibilidad
Precio (ACV)	+/- 20%	+/- \$108k	Alta
Adquisición clientes	+/- 20%	+/- \$108k	Alta
CAC	+/- 20%	+/- \$25k	Media
Churn	+/- 5pp	+/- \$65k	Media-Alta

Las variables de mayor impacto son el precio y la capacidad de adquisición de clientes, seguidas por el churn. El CAC tiene menor sensibilidad relativa en los ingresos totales.

Validación contra benchmarks. Los resultados se validaron contra benchmarks publicados:

- **CAC/ACV ratio:** 0.19-0.23 (benchmark SaaS: 0.15-0.30) - Dentro del rango
- **Churn anual:** 7-15% (benchmark B2B: 5-20%) - Dentro del rango
- **Tiempo implementación:** 2-4 meses (benchmark enterprise: 3-6 meses) - Dentro del rango

Proyección Monetaria por Escenarios

Utilizando los resultados del Delphi simulado, se proyectan tres escenarios financieros. El escenario base utiliza la mediana de las estimaciones (ACV: USD 12,000; implementación: USD 10,000).

Proyección de ingresos por escenarios.

Cuadro 5

Proyección de ingresos por escenario - Primeros 3 años

Año	Escenario	Clientes activos	Nuevos clientes	Ingresos [kUSD]
1	Pesimista	8	8	160
1	Base	12	12	264
1	Optimista	16	16	400
2	Pesimista	18	10	316
2	Base	25	13	430
2	Optimista	35	19	665
3	Pesimista	30	12	480
3	Base	45	20	740
3	Optimista	65	30	1,175

Análisis de punto de equilibrio. En el escenario base, considerando costos operativos de USD 180k anuales:

- **Punto de equilibrio:** 15 clientes activos (mes 18)
- **Margen bruto:** 85% (excluyendo costos de implementación)
- **Payback de CAC:** 2.8 meses promedio

Proyección de costos

Desarrollo del sistema

A continuación se desglosan las tareas necesarias para el desarrollo de cada componente y se estima el costo con base en la complejidad y el perfil de los desarrolladores requeridos.

Se toma un costo de USD 50 para los perfiles Sr. y USD 40 Jr.

Endpoint de generación de contexto hiperpersonalizado. Este es el componente principal que usarán los clientes para mejorar las respuestas de sus bots.

Tareas:

- Diseño y desarrollo del endpoint (Sr.): 40 horas.
- Implementación de lógica de generación de contexto (Sr.): 60 horas.
- Pruebas y optimización (Sr.): 20 horas.

Módulo agencial de consulta de información personal. Funciona en segundo plano y garantiza consultas seguras de fuentes privadas.

Tareas:

- Diseño del sistema agencial (Sr.): 50 horas.
- Desarrollo de la lógica de consulta segura (Sr.): 80 horas.
- Pruebas de seguridad y optimización (Sr.): 30 horas.

Conectores a bases de datos más difundidas. Incluye conectores para MySQL, SQL Server, MongoDB y Oracle.

Tareas:

- Desarrollo de conectores para cada base de datos:
 - MySQL y SQL Server (Jr.): 40 horas cada uno.
 - MongoDB y Oracle (Sr.): 40 horas cada uno.
- Pruebas de integración y optimización (Sr.): 20 horas.

Conectores a CRMs y ERPs más conocidos. Incluye conectores para Salesforce y SAP.

Tareas:

- Desarrollo de conectores para Salesforce y SAP (Sr.): 60 horas cada uno.
- Pruebas de integración (Sr.): 20 horas.

Módulo de configuración con interfaz gráfica. Permite a los clientes configurar el sistema mediante una interfaz intuitiva.

Tareas:

- Diseño y desarrollo de la interfaz gráfica (Sr.): 80 horas.
- Implementación de la lógica de configuración (Sr.): 60 horas.
- Pruebas de usuario (Jr.): 20 horas.

Contador de uso. Monitorea el consumo de tokens para interpretar información y generar respuestas.

Tareas:

- Desarrollo del sistema de conteo de tokens (Sr.): 40 horas.
- Pruebas de precisión y optimización (Sr.): 20 horas.

Resumen.

Cuadro 6

Proyección de Costos por Componente

Componente	Costo (USD)
Endpoint de generación de contexto	6,000
Sistema agencial de consulta segura	8,000
Conectores a bases de datos	8,200
Conectores a CRM y ERP	7,000
Módulo de configuración con interfaz gráfica	7,600
Contador de uso	3,000
Total	39,800

El costo estimado para completar el desarrollo del middleware y convertirlo en un producto listo para ser vendido como licencia es USD 39,800. Este cálculo incluye todas las tareas necesarias para asegurar que el producto sea funcional, seguro y fácil de integrar en los sistemas de los clientes.

Implementación

Desglose de tareas por perfil. A continuación, se detallan los costos involucrados en cada implementación típica.

Roles involucrados en la implementación:

1. Project Manager (PM):

- Responsable de la coordinación del proyecto.
- Costo por hora: USD 40.
- Horas estimadas por implementación: 20 horas.
- Subtotal: USD 800

2. Equipo Técnico:

• Desarrolladores Sr.

- Encargados de la integración técnica compleja.
- Costo por hora: USD 50.
- Horas estimadas por implementación: 40 horas.
- Subtotal: USD 2000

• Desarrolladores Jr.

- Soporte técnico y tareas simples de configuración.
- Costo por hora: USD 30.
- Horas estimadas por implementación: 20 horas.
- Subtotal: USD 600

3. Otros Costos Técnicos:

- Incluyen herramientas, software adicional y licencias necesarias para integraciones específicas.
- Subtotal: USD 500

Costo total por implementación. Sumando los costos de los roles involucrados y los costos adicionales:

Cuadro 7

Costo Total por Implementación

Concepto	Costo (USD)
Project manager (PM)	800
Desarrollador Sr. (40 horas)	2,000
Desarrollador Jr. (20 horas)	600
Otros costos técnicos	500
Total de costos	3,900

Soporte y monitoreo

El costo de soporte y monitoreo mensual tiene en cuenta los servicios que se deben dar a quienes abonan la licencia para garantizar un correcto tratamiento de problemas que puedan surgir durante la operación.

Roles y costos asociados. a. Soporte Técnico

- Tareas:
 - Resolución de problemas técnicos.
 - Atención a solicitudes y dudas de los clientes.
- Frecuencia estimada: 5 horas mensuales por cliente.
- Costo por hora (Jr.): USD 30.

b. Monitoreo del Sistema

- Tareas:
 - Monitoreo de logs y métricas del sistema.
 - Prevención de fallos y optimización continua.
- Frecuencia estimada: 3 horas mensuales por cliente.
- Costo por hora (Sr.): USD 50.

Costo mensual. Sumando los costos por cliente:

Cuadro 8

Costo Mensual de Soporte y Monitoreo

Concepto	Costo Mensual (USD)
Soporte Técnico	150
Monitoreo del Sistema	150
Total	300

Costos fijos

Recursos humanos. El equipo administrativo y de gestión se modificará de acuerdo al volumen de negocio. Al inicio se necesitará de una figura que lidere la compañía y que se mantenga durante toda la vida del proyecto.

CEO:. USD 4,000/mes.

En el equipo de ventas se calcula una persona el primer año y luego dos hasta el año 7.

Ventas: USD 3,000/mes.

Para asistir en temas contables y de operatoria diaria se necesitará un equipo administrativo que crecerá desde una posición en el año 1 a 3 en el año 4.

Asistente Administrativo: USD 1,500/mes.

Un líder técnico a cargo de las operaciones de soporte nivel 3 y monitoreo formará parte desde el inicio del proyecto.

Líder Técnico. (para actualizaciones y soporte interno): USD 4,000/mes.

Un desarrollador se encargará de las actualizaciones de la plataforma y el soporte nivel 2.

Desarrollador: USD 1,800/mes.

Se necesitará al comienzo una persona para dar soporte nivel 1 que luego en el año 4 se convertirá en un equipo de dos personas.

Soporte: USD 1,500/mes.

Equipo Técnico de Base:

- Ingeniero de Infraestructura: USD 3,500/mes.
- Especialista en Seguridad: USD 3,000/mes.

Cuadro 9*Costos de Recursos Humanos por Año (USD)*

RRHH	Año 0	Año 1	Año 2	Año 3	Año 4	Año 5	Año 6	Año 7
CEO	0	4,000	4,000	4,000	4,000	4,000	4,000	4,000
Vendedores	0	2,000	4,000	4,000	4,000	4,000	4,000	4,000
Asist. Administrativo	0	1,500	3,000	3,000	4,500	4,500	4,500	6,000
Esp. en Seguridad	0	0	0	2,500	2,500	2,500	2,500	2,500
Líder Técnico	0	4,000	4,000	4,000	4,000	4,000	4,000	4,000
Des. Software	0	1,800	1,800	1,800	1,800	1,800	1,800	1,800
Soporte	0	1,500	1,500	1,500	3,000	3,000	3,000	3,000
Total mensual	0	14,800	18,300	20,800	23,800	23,800	23,800	25,300
Total anual	0	177,600	219,600	249,600	285,600	285,600	285,600	303,600

Infraestructura y operación

Hosting y Servidores en la Nube. Incluye servicios como AWS, Azure o Google Cloud para alojar los servicios de monitoreo y soportar las operaciones. Costo estimado: USD 800/mes.

Herramientas de Monitoreo y Gestión. Herramientas de ingestión de logs y métricas. (Splunk, New Relic, Sentry, Datadog, Bitrix24) Costo estimado: USD 300/mes.

Subtotal Infraestructura y Operación: USD 1,100/mes.

Resumen
Cuadro 10

Costos de Infraestructura y Operación

Concepto	Costo
Hosting y Servidores en la Nube	\$ 800
Herramientas de Monitoreo y Gestión	\$ 300
Total mensual	\$ 1,100
Total anual	\$ 13,200

Otros costos variables

Publicidad, prensa y marketing. Se calcula un 5% sobre las ventas para cada año para ir incrementando acorde al crecimiento de la empresa. Los primeros clientes vendrán primordialmente de las redes de contacto de los involucrados y a medida que evolucione el proyecto será necesario acceder a nuevos clientes mediante tácticas de prensa y publicidad. La dirección deberá reservar siempre este porcentaje para salvaguardar la salud de ventas del año siguiente.

Gastos generales. Por cualquier gasto oculto que pueda surgir de operar con un equipo de personas de manera totalmente remota se toma un 1% del costo total de recursos humanos. Esto no incluye la compra de computadoras ya que se evalúan como bien de uso de la empresa y se pueden amortizar.

Evaluación Financiera

Márgenes de contribución

Los ingresos se dividen en ventas de licencias e implementaciones. A continuación se calculan los márgenes de contribución de cada línea de negocio.

Margen de contribución por Implementación. El margen bruto se calcula restando los costos de implementación al precio de venta:

Precio de venta por implementación: USD 10,000

Costos totales por implementación: USD 3,900

Margen bruto absoluto por implementación: USD 6,100

Margen de contribución por licencia.

Precio de venta por licencia: USD 12,000

Costos variables asociados a gastos generales: USD 120

Costos variables asociados a marketing: USD 600

Margen absoluto por licencia anual: USD 11,280

Tasas para la valuación del proyecto

En diciembre de 2024, las tasas de interés para préstamos a PyMEs en Argentina varían según la entidad financiera y las condiciones específicas del crédito. A continuación, se detallan algunas de las tasas ofrecidas por diferentes instituciones:

Banco Nación: Ofrece una Tasa Nominal Anual (TNA) del 34% para créditos a 90 días, y 35% para plazos superiores.

Banco Provincia: Presenta tasas del 40% TNA para capital de trabajo a 12 meses.

Bancos Privados: Las tasas para préstamos de corto plazo (180 días) rondan el 49.5% TNA, mientras que para plazos de 12 meses ascienden al 51% TNA.

Considerando estas opciones, una tasa promedio razonable para el análisis financiero sería del 40% TNA.

Esto significa que la TEA es aproximadamente del 49.27% para intereses compuestos mensualmente.

Cálculo de la tasa de descuento con el método CAPM. Según el método CAPM (Capital Asset Pricing Model), el costo del capital (K_e) se calcula usando la siguiente fórmula:

$$K_e = R_f + \beta(R_m - R_f) + \text{Riesgo País}$$

Selección de la Tasa Libre de Riesgo

La tasa libre de riesgo (R_f) se toma del rendimiento a 10 años de los bonos del tesoro de los Estados Unidos de Norteamérica, hoy en torno al 4,6%.

$$R_f = 4,6\%$$

Estimación del factor Beta

Para este estudio, se propone un valor de Beta de 1.5, fundamentado en las características del sector, la naturaleza del producto y las condiciones del mercado regional.

El mercado tecnológico, particularmente en el ámbito de software empresarial y soluciones de inteligencia artificial, se caracteriza por una volatilidad superior al promedio del mercado general. Empresas maduras como Microsoft y Salesforce operan con valores de Beta entre 1 y 1.3, reflejando un riesgo moderado. Las startups tecnológicas que ingresan en mercados competitivos y en rápida evolución presentan Betas superiores, generalmente en el rango de 1.3 a 2.0, debido a su mayor dependencia del crecimiento y la incertidumbre inicial.

El middleware de hiperpersonalización que constituye el producto central de este proyecto está diseñado para integrarse con sistemas empresariales como CRMs, ERPs y bases de datos. Esto genera una menor volatilidad relativa en comparación con startups más disruptivas que operan en mercados especulativos como el de las criptomonedas. Sin embargo, el hecho de que sea un producto nuevo, con una curva de adopción inicial y competencia directa de actores consolidados (e.g., Salesforce, Amazon) aumenta el nivel de riesgo, justificando un Beta superior a 1.

La operación se desarrolla en un mercado emergente, América Latina, donde factores económicos y políticos incrementan la incertidumbre. Esto se traduce en un ajuste adicional al valor de Beta.

Según Damodaran (2024), las empresas que operan en mercados emergentes suelen experimentar un aumento de entre 0.25 y 0.50 puntos en su Beta debido a estos factores. Como startup en etapa temprana, este proyecto enfrenta riesgos inherentes a la falta de una base de clientes establecida y de ingresos recurrentes probados. Esto justifica un ajuste hacia el rango superior del valor de Beta, posicionándolo en 1,5.

Prima de riesgo para Argentina. Al obtener la prima de riesgo para Argentina, una referencia destacada es el trabajo del profesor Pablo Fernández de IESE Business School, quien publica regularmente estudios sobre las primas de riesgo de mercado a nivel mundial. En su informe de 2023, Fernández estima la prima de riesgo de mercado para Argentina en torno al 7%.

Este valor se obtiene considerando la diferencia entre el rendimiento promedio del mercado accionario argentino y la tasa libre de riesgo, ajustada por factores específicos del país.

$$R_m - R_f = 7\%$$

Riesgo País. El riesgo país para Argentina a la fecha de elaboración de este trabajo se ubica en 578 puntos básicos.

Cálculo del costo del capital. Al no tener fuentes de financiamiento y hacer frente solo con aportes de capital la tasa de descuento es igual al costo del capital.

$$K_e = 4,6\% + 1,5 \times 7\% + 5,78\% = 20,88\%$$

Evaluación monetaria del proyecto

Duración. La industria de la inteligencia artificial se encuentra en un momento de crecimiento explosivo, con cambios de paradigma que requieren repensar modelos de negocio constantemente. De esta manera se ha adjudicado una vida útil de 7 años al desarrollo del software de hiperpersonalización. Se tiene en cuenta una actualización durante el año 3 correspondiente al 50% de la inversión inicial.

Estados de resultados proforma.**Cuadro 11***Estados de resultados proforma*

Año	0	1	2	3	4	5	6	7
Ventas								
Ventas netas	\$ 220,000	\$ 362,000	\$ 582,000	\$ 542,000	\$ 602,000	\$ 684,000	\$ 756,000	
Costo de ventas								
Costo de ventas	\$-	\$ 68,000	\$ 98,800	\$ 152,400	\$ 118,371	\$ 133,145	\$ 150,538	\$ 169,145
Margen bruto	\$-	\$ 152,000	\$ 263,200	\$ 429,600	\$ 423,629	\$ 468,855	\$ 533,462	\$ 586,855
Gastos operativos								
Ventas, generales y administrativas	\$ 2,000	\$ 212,456	\$ 268,468	\$ 314,364	\$ 357,316	\$ 360,316	\$ 364,416	\$ 391,032
Gastos operativos totales	\$ 2,000	\$ 212,456	\$ 268,468	\$ 314,364	\$ 357,316	\$ 360,316	\$ 364,416	\$ 391,032
Resultado operativo	-\$ 2,000	-\$ 60,456	-\$ 5,268	\$ 115,236	\$ 66,313	\$ 108,539	\$ 169,046	\$ 195,823
Amortizaciones								
Computadoras	\$-	\$ 6,000	\$ 8,000	\$ 9,000	\$ 5,000	\$ 3,000	\$ 2,000	\$ 1,000
Otros ingresos y gastos								
Renta extraordinaria		\$-	\$ 3,000	\$ 1,000	\$ 500	\$ 1,000	\$ 500	
Resultado antes de IG	-\$ 2,000	-\$ 66,456	-\$ 13,268	\$ 109,236	\$ 62,313	\$ 106,039	\$ 168,046	\$ 195,323
IG	-\$ 700	-\$ 23,260	-\$ 4,644	\$ 38,233	\$ 21,810	\$ 37,114	\$ 58,816	\$ 68,363
Resultado neto	-\$ 2,000	-\$ 66,456	-\$ 13,268	\$ 71,003	\$ 40,504	\$ 68,926	\$ 109,230	\$ 126,960

Hasta el año 2 el negocio no es sustentable y necesita del aporte de capital para sostener los costos.

Balances proforma.**Cuadro 12***Balances proforma*

Año	0	1	2	3	4	5	6	7
Activo corriente								
Caja y bancos	\$-	\$ 20,000	\$ 8,732	\$ 85,735	\$ 125,239	\$ 197,165	\$ 308,395	\$ 433,355
Provisión de pago de IVA	\$-	\$ 4,165	\$ 6,997	\$ 11,319	\$ 10,765	\$ 11,980	\$ 13,602	\$ 15,072
Provisión de pago de IG			-\$ 4,644	\$ 38,233	\$ 21,810	\$ 37,114	\$ 58,816	\$ 68,363
Total activo corriente	\$-	\$ 24,165	\$ 11,085	\$ 135,287	\$ 157,814	\$ 246,259	\$ 380,813	\$ 516,790
Activo fijo								
Computadoras	\$-	\$ 12,000	\$ 10,000	\$ 4,000	\$ 5,000	\$ 2,000	\$-	\$ 2,000
Total activo fijo	\$-	\$ 12,000	\$ 10,000	\$ 4,000	\$ 5,000	\$ 2,000	\$-	\$ 2,000
Activo intangible								
Código propietario	\$ 39,800	\$ 39,800	\$ 39,800	\$ 54,800	\$ 54,800	\$ 54,800	\$ 54,800	\$ 54,800
Total Activo	\$ 39,800	\$ 75,965	\$ 60,885	\$ 194,087	\$ 217,614	\$ 303,059	\$ 435,613	\$ 573,590
Pasivos corrientes								
IVA a pagar	\$-	\$ 4,165	\$ 6,997	\$ 11,319	\$ 10,765	\$ 11,980	\$ 13,602	\$ 15,072
IG			-\$ 4,644	\$ 38,233	\$ 21,810	\$ 37,114	\$ 58,816	\$ 68,363
Total Pasivo Corriente	\$-	\$ 4,165	\$ 2,353	\$ 49,552	\$ 32,575	\$ 49,094	\$ 72,418	\$ 83,435
Patrimonio								
Capital social	\$ 41,800	\$ 140,256	\$ 140,256	\$ 155,256	\$ 155,256	\$ 155,256	\$ 155,256	\$ 155,256
Utilidades retenidas		-\$ 2,000	-\$ 68,456	-\$ 81,724	-\$ 10,721	\$ 29,783	\$ 98,709	\$ 207,939
Resultado del período	-\$ 2,000	-\$ 66,456	-\$ 13,268	\$ 71,003	\$ 40,504	\$ 68,926	\$ 109,230	\$ 126,960
Total Patrimonio	\$ 39,800	\$ 71,800	\$ 58,532	\$ 144,535	\$ 185,039	\$ 253,965	\$ 363,195	\$ 490,155
Total P + PN	\$ 39,800	\$ 75,965	\$ 60,885	\$ 194,087	\$ 217,614	\$ 303,059	\$ 435,613	\$ 573,590

Flujo de fondos.**Cuadro 13***Flujo de fondos*

Año	0	1	2	3	4	5	6	7
Caja de operaciones								
Ingreso neto	-\$ 2,000	-\$ 66,456	-\$ 13,268	\$ 71,003	\$ 40,504	\$ 68,926	\$ 109,230	\$ 126,960
Ajustado por:								
Amortizaciones	\$-	\$ 6,000	\$ 8,000	\$ 9,000	\$ 5,000	\$ 3,000	\$ 2,000	\$ 1,000
Cambios en cuentas	\$-	\$ 20,000	\$ 8,732	\$ 85,735	\$ 125,239	\$ 197,165	\$ 308,395	\$ 433,355
Caja de operaciones	-\$ 2,000	-\$ 40,456	\$ 3,464	\$ 165,738	\$ 170,743	\$ 269,091	\$ 419,625	\$ 561,315
Caja de inversiones								
Compraventa de bienes	\$-	-\$ 18,000	-\$ 6,000	-\$ 3,000	-\$ 6,000	\$-	\$-	-\$ 3,000
Software	-\$ 39,800			-\$ 15,000				
Caja de inversiones	-\$ 39,800	-\$ 18,000	-\$ 6,000	-\$ 18,000	-\$ 6,000	\$-	\$-	-\$ 3,000
Cambio neto en caja								
	-\$ 41,800	-\$ 58,456	-\$ 2,536	\$ 147,738	\$ 164,743	\$ 269,091	\$ 419,625	\$ 558,315
Cambio en capital de trabajo								
	\$-	\$ 20,000	\$ 8,732	\$ 85,735	\$ 125,239	\$ 197,165	\$ 308,395	\$ 433,355
CAPEX	-\$ 39,800	-\$ 18,000	-\$ 6,000	-\$ 18,000	-\$ 6,000	\$-	\$-	-\$ 3,000

Valuación y viabilidad del proyecto. Tomando una duración de 7 años, con una inversión inicial de USD 41,800 al inicio y un costo promedio del capital de 0,2088 anual procedemos al cálculo del valor actual neto de los flujos de fondo proyectados.

Flujos de fondos proyectados. Para el cálculo del VAN se necesitan los flujos de caja libre para la empresa conocidos como FCFF (free cash flow to the firm) que se computan como la suma de los resultados netos, las amortizaciones, las diferencias de capital de trabajo y los aportes de capital.

Cálculo del VAN.

$$VAN = \sum_{t=0}^n \frac{FCF_t}{(1+r)^t} - \text{Inversión inicial}$$

Cálculo del TIR.

TIR = Tasa que hace el VAN igual a cero

Cuadro 14*Flujo de caja libre para la empresa (FCFF)*

Año	0	1	2	3	4	5	6	7
Resultados								
Resultado neto	-\$ 2,000	-\$ 66,456	-\$ 13,268	\$ 71,003	\$ 40,504	\$ 68,926	\$ 109,230	\$ 126,960
Amortizaciones	\$-	\$ 6,000	\$ 8,000	\$ 9,000	\$ 5,000	\$ 3,000	\$ 2,000	\$ 1,000
Diferencia en Capital de Trabajo	\$-	\$ 20,000	\$ 8,732	\$ 85,735	\$ 125,239	\$ 197,165	\$ 308,395	\$ 433,355
CAPEX	-\$ 39,800	-\$ 18,000	-\$ 6,000	-\$ 18,000	-\$ 6,000	\$-	\$-	-\$ 3,000
FCFF	-\$ 41,800	-\$ 58,456	-\$ 2,536	\$ 147,738	\$ 164,743	\$ 269,090	\$ 419,625	\$ 558,315

VAN. Con las tasas calculadas y este flujo de fondos se arriba a un VAN USD 455,720.32 , por lo tanto al ser este número positivo se demuestra la viabilidad del proyecto.

Bibliografía